Data-driven Dynamic Simulation of Signaling Networks in Triple Negative Breast Cancer Cells

Master Thesis

Degree: Life Science Engineering – Master of Science

University: Hochschule für Technik und Wirtschaft Berlin

Faculty: Engineering – Technology and Life (F II)

Humboldt Universität zu Berlin – Department of Theoretical Biophysics

Author:

Luis Firl

15.04.2026

First Supervisor: Prof. Dr. Carsten Conradi

Second Supervisor: Prof. Dr. Dr. h.c. Edda Klipp

Acknowledgement

First and foremost, I would like to express my deepest gratitude to Prof. Dr. Dr. h.c. Edda Klipp from the Department of Theoretical Biophysics at Humboldt-Universität zu Berlin. Her exceptional supervision and unwavering support have been instrumental to the success of this thesis. I am profoundly grateful for the wealth of knowledge she shared with me and for the generous amount of time she dedicated to my guidance.

I would also like to extend my sincere thanks to Prof. (ret.) Dr. Rune Linding and Dr. James Longden for providing the essential phosphoproteomic dataset. Their expertise and the detailed information they provided regarding the experimental procedures and data processing were fundamental to the development of my model.

Furthermore, my sincere thanks go to Prof. Dr. Carsten Conradi for his invaluable guidance and the insights he provided throughout my studies. I am grateful for the academic foundation he helped me build and for his constructive feedback, which was essential in the final evaluation of this thesis.

My deepest thanks also go to my partner for her immense devotion and tireless support throughout this journey. Your presence provided the strength and balance I needed to complete this work.

Finally, my heart goes out to my family. I am infinitely grateful for the sacrifices you have made to make my education possible. Thank you for giving me the freedom and the foundation to become the person I am today. Your belief in me has been my greatest strength, and this achievement is as much yours as it is mine.

Abstract

This thesis presents a data-driven dynamic modeling approach for kinase signaling in triple-negative breast cancer cells, focusing on the EGFR/MAPK-centered network reconstructed from phosphoproteomic fold change data. The central objective is to estimate both network dependency parameters and kinetic rate parameters in order to reproduce the experimentally observed phosphorylation dynamics as closely as possible and to establish a basis for dynamic perturbation analysis.

To achieve this, dependency weights representing relations between kinases and phosphorylation sites were first estimated using two distinct optimization approaches implemented in the Julia programming language, namely a gradient-based optimizer and an Interior Point optimizer. Both methods were designed to fit the same dependency model to the experimental data and were compared with respect to their methodological properties and numerical performance. The estimated dependency parameters were subsequently integrated into an ordinary differential equation model based on mass-action kinetics, in which the kinetic parameters were optimized protein-wise before being utilized in the coupled network simulation.

The results show that both dependency optimizers produced highly similar fits to the experimental fold change data, despite differences in parameter handling and optimization strategy, indicating that multiple parameter constellations can reproduce the observed data equally well. The Interior Point optimizer achieved faster convergence for the current network size, whereas the gradient-based approach remains attractive with respect to future scalability. At the same time, the comparison of the resulting dependency parameter profiles suggests that the pure mathematical optimization in the absence of additional biophysical constraints is insufficient to identify uniquely plausible biological dependency parameters.

The signaling network model successfully integrated the estimated dependency structure and yielded stable kinetic parameter sets that reproduced the dynamic trends of the phosphoproteomic data within the coupled system. Remaining discrepancies, including offsets between simulated and measured profiles and numerical artifacts during initialization, indicate that further integration of biophysical constraints and additional data will be required to improve biological plausibility and predictive accuracy. Overall, the study establishes a computational framework for dependency estimation and dynamic simulation of signaling networks and provides a basis for future perturbation studies and expanded network reconstruction in triple-negative breast cancer.

Contents

List of Figures

List of Abbreviations

ABL2	Abelson tyrosine-protein kinase 2
ADP	Adenosine diphosphate
ATP	Adenosine triphosphate
CDK1	Cyclin-dependent kinase 1
CDK2	Cyclin-dependent kinase 2
CLK3	CDC-like kinase 3
CSV	Comma-separated values
EGFR	Epidermal growth factor receptor
EPHA2	Ephrin Type-A receptor 2
ERBB2	Erb-B2 Receptor tyrosine kinase 2
GRK2	G Protein-coupled receptor kinase 2
GRK5	G Protein-coupled receptor kinase 5
GSK3A	Glycogen synthase kinase 3 alpha
HER2	Human epidermal growth factor receptor 2
LYN	Lck/Yes novel tyrosine kinase
KKT	Karush-Kuhn-Tucker
MAP2K2	Mitogen-activated protein kinase kinase 2
MAPK	Mitogen-activated protein kinase
MAPK1	Mitogen-activated protein kinase 1
MAPK3	Mitogen-activated protein kinase 3
MAE	Mean absolute error
MET	Methylnitronitrosoguanidine receptor tyrosine kinase
MSE	Mean squared error
ODE	Ordinary differential equation
PAK2	P21-activated kinase 2
PKMYT1	Membrane-associated Y- and T-specific protein kinase
PRKCA	Protein kinase C alpha
ReLU	Rectified linear unit
TNBC	Triple negative breast cancer

List of Symbols

A	Transcription rate
α_j^i	Effect of j-th kinase on i-th phosphorylation site
α_i^0	Effect of unknown kinases on i-th phosphorylation site
B	mRNA degradation rate
β_j^k	Contribution of k-th phosphorylation site on activity of j-th kinase
β_j^0	Contribution of unknown phosphorylation sites on activity of j-th kinase
C	Translation rate
D_0	Protein degradation rate
D_i	Phosphoprotein degradation rate
P_0	Unphosphorylated protein abundance
P_i	Phosphorylated protein abundance
S_i	Phosphorylation rate
Z_i	Dephosphorylation rate

1. Introduction

The mathematical modeling of signaling pathways plays a crucial role in understanding cellular functions and regulatory mechanisms. By translating complex biological interactions into quantitative frameworks, it enables uncovering the design principles of biological systems and translating experimental data into predictive and interpretable knowledge [1].

In multicellular organisms, signaling pathways are essential for coordinating cell developments and maintaining homeostasis. Their dysregulation however is a primary driver of oncogenesis [2]. Aberrant activation of conserved signaling modules, such as the mitogen-activated protein kinase (MAPK) cascade, often induced by overactive receptors including the Epidermal Growth Factor Receptor (EGFR), can disturb the intracellular signaling balance [3]. Consequence of such dysregulation is the promotion of malignant cell behavior, including enhanced proliferation, survival, and migration.

Cell migration is a tightly regulated process that depends on the coordinated activity of signaling pathways controlling cytoskeletal remodeling, adhesion turnover, and force generation [4]. In cancer cells, the pathways controlling these processes are often persistently activated or insufficiently inhibited, resulting in accelerated and invasive migratory behavior [5]. As these phenotypes are driven by the interaction of multiple kinases rather than by isolated signaling events, their investigation requires a systems-level perspective.

The present study addresses the quantitative modeling of a kinase signaling network and the estimation of model parameters from experimental protein fold change data. The central objective of this work is the computational optimization of both network dependency parameters and kinetic parameters, in order to achieve the closest possible agreement between model predictions and observed data. Establishing a quantitatively reliable model is essential for simulating targeted perturbations, such as knockdowns, and for analyzing how these interventions affect the behavior of the signaling network. In the long term, the project goal is to identify critical regulatory nodes and support the development of therapeutic strategies aimed at restricting cancer cell migration and metastatic progression.

1.1 Signaling Pathways in Cancer Cells

The distinction between cancer cells and healthy cells is not limited to genetic alterations but also involves profound changes in the regulation and transmission of intracellular signaling pathways. Conditions of oncogenic alterations, gene amplifications and loss of regulatory control mechanisms lead to signaling pathways not transiently activating but remaining permanently dysregulated or chronically active [6]. Thereby, cancer cells gain properties of long-term proliferation, advanced survival abilities and metabolic changes as well as increased invasiveness and cell migration [5].

The malignant characteristics are mediated by several intertwined signaling pathways that are often altered in cancer cells. Frequently affected pathways include cytoplasmic tyrosine kinases, lipid kinases and nuclear receptors as well as growth factor receptor kinases like EGFR, Guanosine Triphosphatases like Ras, and serine/threonine kinases like Raf and Akt, which are responsible for triggering and maintaining the MAPK signaling cascade [7].

These pathways do not act as isolated linear cascades but form a closely linked regulatory network. They are interlinked through crosstalk mechanisms, which respond to perturbations in one branch of the network resonating through other pathways, whether directly or indirectly [8]. At the same time, many of these pathways inherit redundant functionalities [9] as different signaling cascades often control similar cellular programs like proliferation and cell migration. Additionally, feedback regulation impacts the strength of linear signals [8]. These properties make the cellular signaling system reasonably robust, but also vulnerable to oncogenic changes and mutations. Loss of regulatory control mechanisms at central nodes of the signaling network can lead to persistent dysregulation which spreads via numerous connections, thereby stabilizing malicious cellular states [6].

For this reason, mathematical modeling of signaling pathways is of particular importance, as it allows complex pathway interactions to be analyzed quantitatively and can support the prediction of how specific perturbations may affect cancerous phenotypes.

1.2 Phosphorylation mechanisms in Signaling Pathways

Protein phosphorylation is the most commonly occurring type of post-translational modification utilized for transducing cell signals [10]. It takes part in controlling a range of fundamental cellular processes including metabolism, growth, cell division and differentiation, mobility, substance transport, and neuronal and immunological functions [11].

The process of protein phosphorylation is carried out by kinases (Fig. 1), which are responsible for catalyzing the transfer of the γ-phosphate from Adenosine triphosphate (ATP) to a serine, threonine or tyrosine residue in protein substrates [11]. In contrast, phosphatase enzymes are responsible for dephosphorylation of proteins by cleaving off phosphate groups previously added by kinases [12]. Phosphorylation processes are therefore no one-time switches but dynamic, regulatory, and reversible procedures [13].

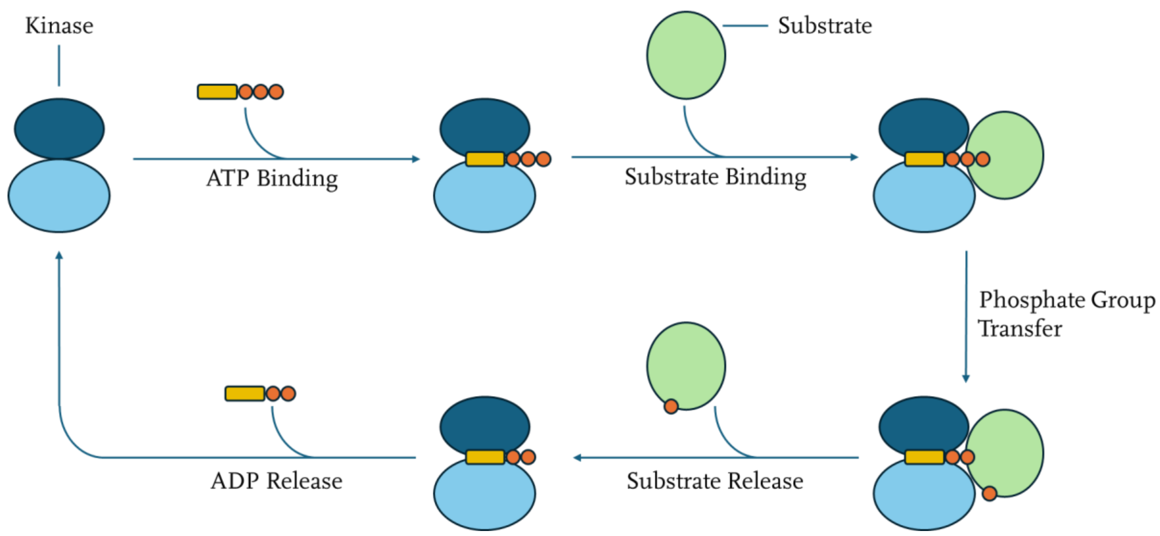

Figure 1: Cyclic process of substrate phosphorylation carried out by a kinase (adapted from Ubersax et al. [10]). In the initial step, ATP binds to the active domain of the kinase. Subsequently, the substrate is also binding to the active kinase domain. A phosphate group is split from ATP and bound to the substrate. The phosphorylated substrate is then released. The remaining Adenosine diphosphate (ADP) is also released from the active kinase domain, and the phosphorylation process can start over.

The attachment of phosphate groups to specific amino acids of proteins changes essential physicochemical properties of the protein regarding charge, conformity, and interaction surfaces [14]. The binding reaction influences enzymatic activities, molecular stability, and the ability to form complexes with other proteins. The functional shifts of a protein triggered by phosphorylation can severely impact signaling networks, as small molecular changes can lead to significant functional consequences on the entire network level [13].

In many cellular signaling pathways, phosphorylation procedures are part of multi-stage signaling cascades. Throughout the process, a kinase activates another downstream kinase or effector protein, which in turn transmits the stimulus signal [15]. The signaling cascades enable the structural flow of information from receptors to cytoplasmic and nuclear targets for initiating intra- and extracellular reactions [16]. They often do not solely consist of linear transduction chains but are embedded within complex signaling networks where multiple input signals converge on shared modules. Theoretical studies have shown that depending on the cascade architecture and kinetic parameters, different responses can arise from similar input information [17].

The specificity of cellular responses to receptor stimulation emerges from the spatial and temporal dynamics of downstream signaling networks [18]. Phosphorylation based signaling pathways are strongly dependent on time and context of the reactions. Multiple parameters determine the intensity and duration of the phosphorylation which affect the pathway outcome and cellular responses [17]. Moreover, feedbacks and nonlinear kinetics can produce bistable, pulsing or oscillating behaviors [19]. Therefore, accurately estimating parameters that represent the kinetics of the pathways is essential for establishing dynamic signaling network models with the ability to simulate realistic cellular responses.

1.3 Establishment of the Protein Dataset

The phosphorylation model established in the present study is based on an unpublished comprehensive set of protein data, which was collected by a research team lead by Rune Linding, James Longden and Gaye Saginc at the Linding Lab of the Biotech Research and Innovation Centre, University of Copenhagen, Denmark [20]. The data was collected by conducting a series of scratch assays with the MDA-MB-231 triple negative breast cancer (TNBC) cell line, where a surface layer of cells was symmetrically wounded to trigger cell migration and proliferation mechanisms within the surrounding cells.

The MDA-MB-231 cell line does not express hormone receptors of estrogen, progesterone and the human epidermal growth factor receptor 2 (HER2), and is therefore categorized as triple negative/basal-like [21]. Its pronounced invasiveness and ability to form metastatic tumors make it an ideal cell line for modeling the migratory behavior of cancer cells [22], which supports studying and understanding cancer progression and opens up possibilities for identifying strategies to inhibit its metastatic capabilities.

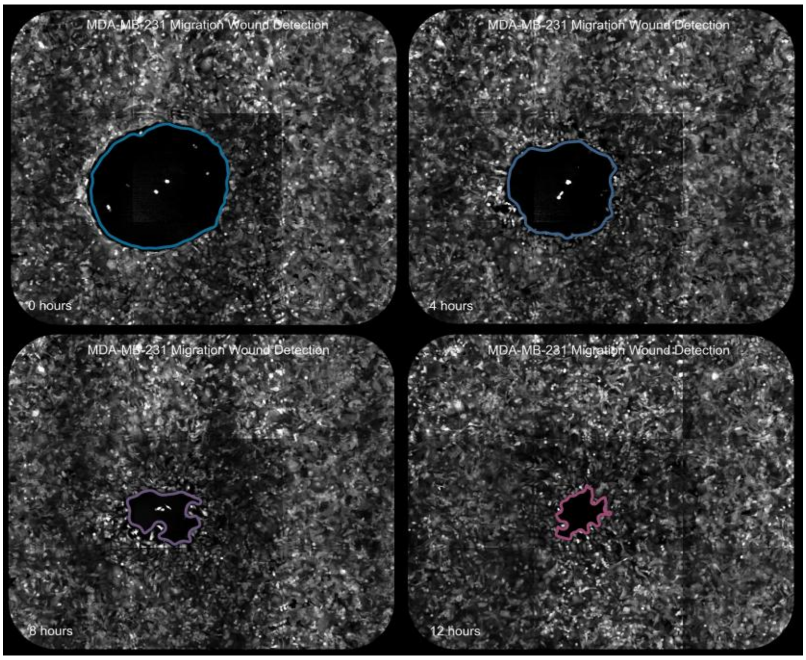

Figure 2: Cell migration process after performing a scratch assay on MDA-MB-231 TBNC cell culture. Four microscopic images illustrate the progress of cell migration at 0, 4, 8, and 12 hours. Green fluorescent protein diffuse in cytoplasm was utilized for cell visualization. Elevated cell migration and proliferation rates are evident, as the scratch wound is almost completely closed within the observed time span [20].

To obtain information and data on protein phosphorylation, the TNBC cells were cultivated in bulk and dispersed on a framed plate. A custom 96-pin studded tool was utilized at different locations on the plate to create a total of 384 scratches in the cell surface layer. Cell samples were harvested at 14 different time points at 0, 0.5, 0.75, 1, 2, 4, 8, 16, 30, 60, 120, 240, 480 and 960 minutes, respectively. A visual illustration of the cell migration progress of TNBC cells over time is depicted in figure 2.

To prepare the cells for mass spectrometry measurements, the protocol established by Santini et al. [23] was utilized. Cell samples were purified, and phosphorylated proteins were enriched with titanium dioxide to isolate them from their complex environment. Afterwards, the phosphoproteomic samples were loaded onto C18 reverse phase chromatography column for separation and eluted with 5-50% acetonitrile over a 240-minute gradient to achieve maximum accuracy and even resolve proteins only differing by a single phosphorylation site. An Orbitrap mass spectrometer was employed to identify phosphorylated proteins and measure their abundances [23].

The data captured by mass spectrometry was analyzed using a novel Bayesian method for estimating protein concentration and phosphorylation site occupancy established by Robin et al. [24]. The developed model tracks experimental and instrumental noise and calculates standard deviations for each calculated ratio. A Likelihood function was derived which models the average ratio of a peptide as a function of protein concentration and the degree of phosphorylation at various sites. By utilizing Markov Chain Monte Carlo sampling, the most likely values for concentrations and occupancy rates, as well as their uncertainties, are estimated from a distribution of values. This allows accurate quantification of protein abundances and prediction of relevant phosphorylation sites even with lower amounts of measured data [24].

By employing the Bayesian method onto the phosphoproteomic data of TNBC cells measured by mass spectrometry, a dataset comprising the mean relative logarithmic fold changes of the unphosphorylated and phosphorylated form of proteins is established. The abundances of 1768 proteins were measured in unphosphorylated and phosphorylated states over 3333 respective target sites.

$$Fold\ Change = \frac{Abundance_{current}}{Abundance_{initial}} \quad \text{Eq. 1}$$

The fold changes represent the relative change of protein abundances depending on the initial abundance measured at time point 0 (Eq. 1). Essentially, logarithmic fold change values for all 14 time points were calculated for every measured protein and its identified phosphorylation sites in the dataset. An example of the measured courses of fold change values for the MAP2K2 protein is depicted in figure 3.

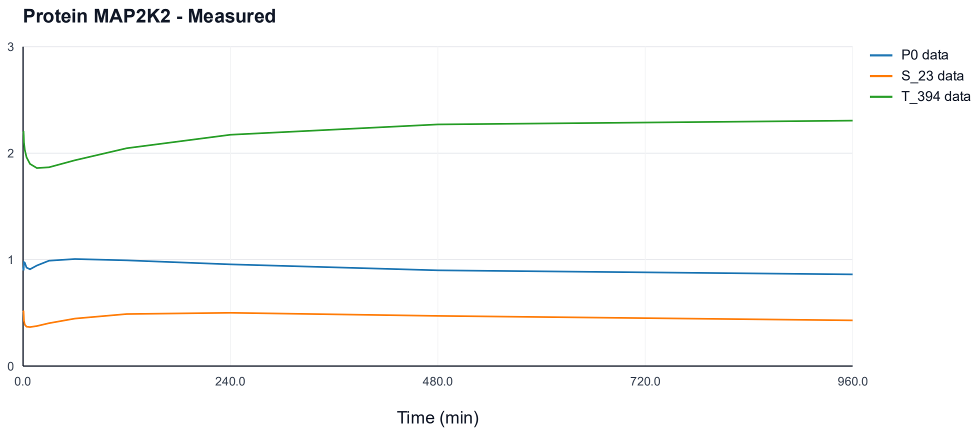

Figure 3: Trajectories of measured fold change values of the MAP2K2 protein plotted between 0 and 960 minutes. The plot shows the trajectories for the states of the unphosphorylated protein P_0, and the phosphorylated protein at sites S_23 and T_394.

1.4 Modeling the EGFR/MAPK Network

The kinase network established in the present study is based on an extended version of the EGFR/MAPK pathway. As described in section 1.1, mutations within this signaling cascade are a frequent driver of oncogenesis, as aberrant signaling causes extensive alterations in physiological activities. These include dysregulated cell proliferation, differentiation, and migration, with the latter being of particular interest regarding the scratch assay experiments described in section 1.3 [25]. Consequently, the signaling environment surrounding the central nodes of the pathway, specifically EGFR and MAPK, is reconstructed as a simulated network model to investigate its dynamics.

To systematically analyze the dynamic interactions between the proteins in the pathway, the KinomeXplorer platform was utilized, which presents a computational tool for modeling kinase signaling networks by combining sequence specificity with cellular context [26]. Specifically, relationships between kinases and phosphorylation sites are evaluated through a unified likelihood ratio that combines motif-based predictions with network proximity. This provides a probabilistic score that decides the potential systematic inclusion of respective interactions into the network model based on a kinase-specific confidence threshold of 1,0 [26].

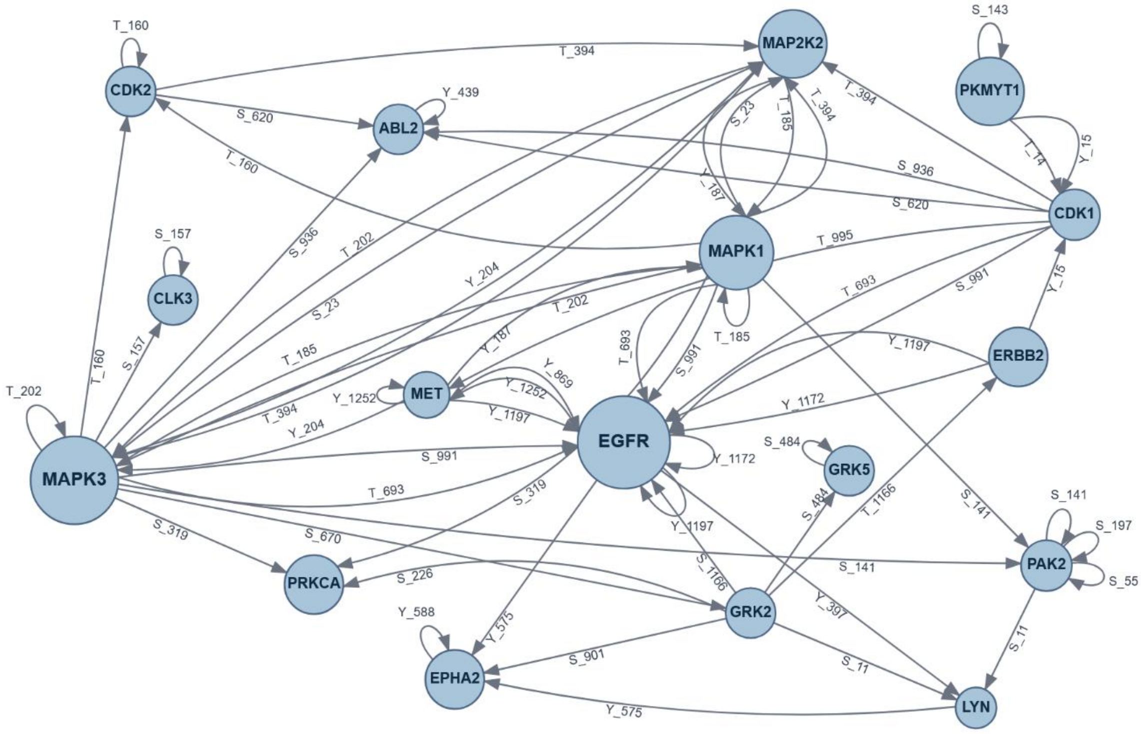

Figure 4: Schematic illustration of signaling network representing predicted dependencies (edges) between kinases (nodes) and phosphorylation sites (labels).

By applying the probabilistic calculations provided by the KinomeXplorer platform onto the observed data retrieved from the phosphoproteomic experiments described in section 1.3, a network comprising of 17 kinases, 35 phosphorylation sites, and 65 edges is predicted. A schematic representation of this network is depicted in figure 4. The predicted kinase dependencies are further utilized to establish the ODE model for simulating the signaling network.

1.5 Construction of ODE Network Model

In the modeling approach, a combination of basic mass action kinetics for gene expression and phosphorylation processes is employed. These kinetics represent microbiological progressions of the central dogma by incorporating transcriptional and translational processes for genes and proteins as well as phosphorylation and dephosphorylation processes into the model.

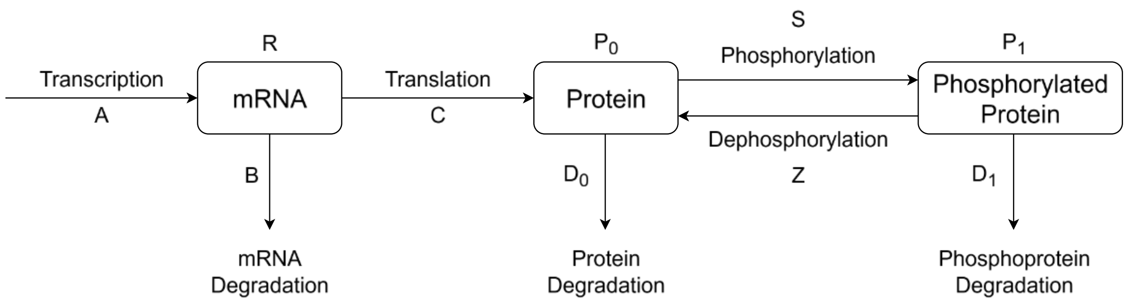

Figure 5: Schematic representation of mass action kinetics relations between mRNA (R), proteins (P_0) and phosphoproteins (P_1). Kinetic rate parameters for transcription (A), mRNA degradation (B), translation (C), protein degradation (D_0), phosphorylation (S) and dephosphorylation (Z), and phosphoprotein degradation (D_1) are established.

Proteins often harbor multiple phosphorylation sites that are not necessarily modified in an arbitrary order. Instead, the phosphorylation of a first site can create the conditions that a second site is recognized and phosphorylated efficiently [27]. The first modification can change conformation, electrostatic charge, and binding of other co-factors to facilitate subsequent phosphorylation of other sites. This first modification is therefore referred to as priming phosphorylation which represents an essential concept of signaling cascades, since many pathways are initiated by secondary phosphorylation of substrates [14]. Based on the streamlined mass action kinetics model (Fig. 5), three distinct approaches regarding phosphorylation processes and their dependencies can therefore be interpreted.

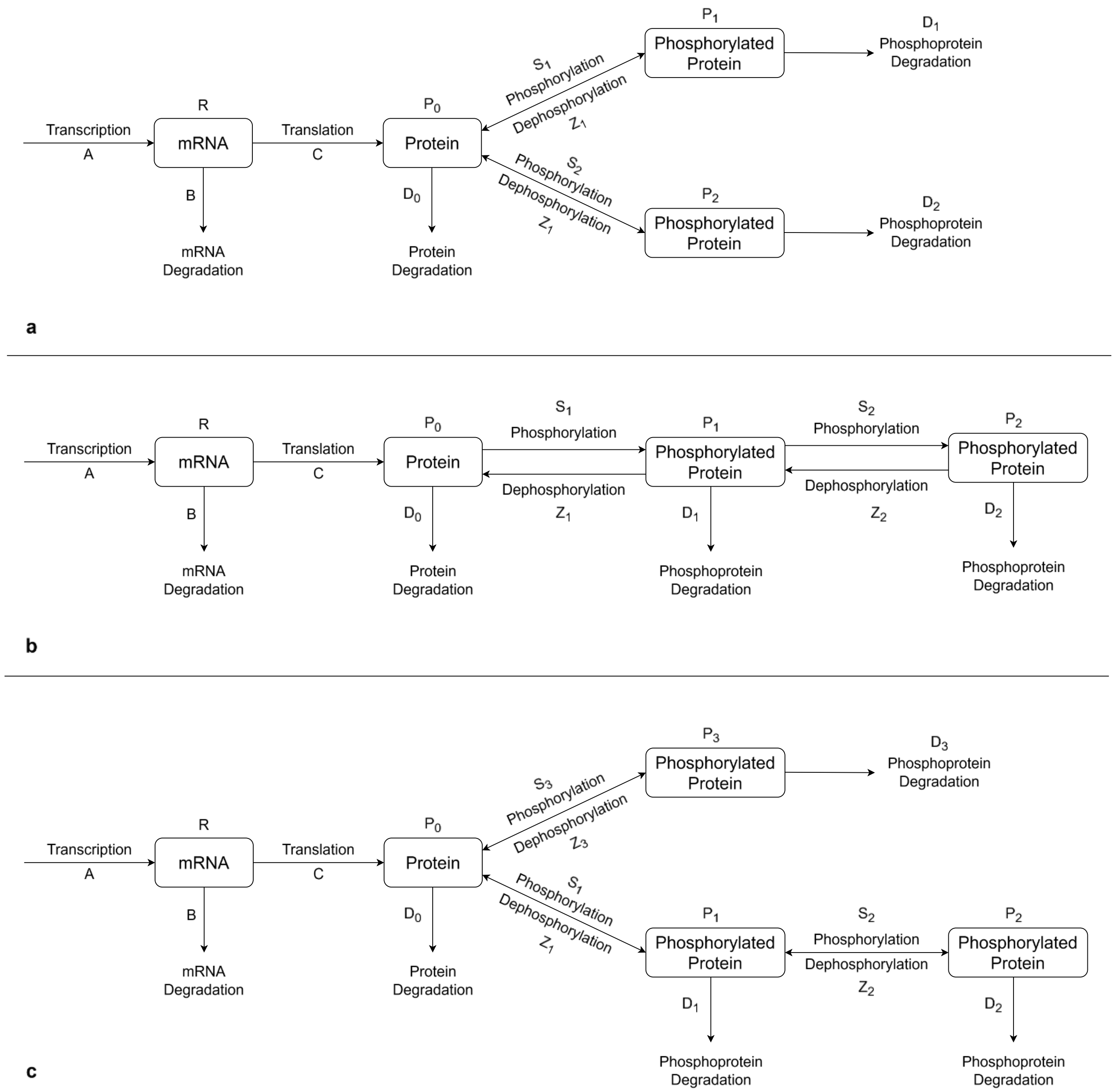

Figure 6: Schematic representation of mass action kinetics relations with distributive (a), successive (b), and random (c) phosphorylation dependencies. In the distributive approach, proteins are independently phosphorylated at multiple target sites, whereas in the successive approach, proteins must be primarily phosphorylated in order for a second phosphorylation process to be initiated. If random phosphorylation dependencies occur, proteins can be phosphorylated independently or require successive processes of multiple phosphorylations.

In figure 6 (a), the mass action kinetics of the distributive phosphorylation model are depicted which represent independent relations between multiple phosphorylation reactions at distinct sites of one respective protein. In contrast, the successive phosphorylation model (Fig. 6, b) shows the relations of protein states and kinetic parameters if a secondary phosphorylation is dependent on the execution of a primary protein phosphorylation. As the dependencies of protein modification procedures are difficult to identify, a third approach representing random phosphorylation is introduced (Fig. 6, c).

Since the dataset utilized in the present study comprises proteins which inherit only one primary phosphorylation, the distributive mass action kinetics model is utilized to establish the equations of the phosphorylation network model. From the mass action kinetics depicted in figure 5, ordinary differential equations (ODEs) can be formulated which represent the rate of change for the mRNA (R) and protein (P) abundances (Eq. 2 and 3).

$$\frac{dR}{dt} = A - BR \qquad \text{Eq. 2}$$

$$\frac{dP}{dt} = CR - D_0 \qquad \text{Eq. 3}$$

To implement the effects for a protein being phosphorylated and dephosphorylated at one respective site as illustrated in figure 5, equation 3 can be supplemented with the appropriate terms (Eq. 4). Additionally, another equation is introduced to represent the abundance of the phosphorylated protein P_1 (Eq. 5).

$$\frac{dP_0}{dt} = CR - P_0(D_0 + S_1) + Z_1P_1 \qquad \text{Eq. 4}$$

$$\frac{dP_1}{dt} = S_1P_0 - P_1(D_1 + Z_1) \qquad \text{Eq. 5}$$

Building on the mass action kinetics depicted in figure 6, the ODEs governing the distributive phosphorylation processes can be derived. These equations ultimately serve to simulate the protein network investigated in this study.

$$\frac{dR}{dt} = A - BR \qquad \text{Eq. 2}$$

$$\frac{dP_0}{dt} = CR - P_0(D_0 + \sum_{i=1}^{n} S_i) + \sum_{i=1}^{n} Z_iP_i \qquad \text{Eq. 6}$$

$$\frac{dP_i}{dt} = S_iP_0 - P_i(D_i + Z_i) \quad \forall i \qquad \text{Eq. 7}$$

The final equations represent the abundance rates of the mRNA (Eq. 2), the unphosphorylated protein (Eq. 6), and the phosphorylated protein (Eq. 7) where i represents each respective phosphorylation site.

1.6 Establishment of Network Dependencies

The ODE equations formulated to model the protein network represent the abundances which can be fitted to the protein dataset. However, the relations between the kinases need to be integrated into the equations to establish the network dependencies. For this reason, additional weight parameters are introduced to depict the protein activities and phosphorylation effects.

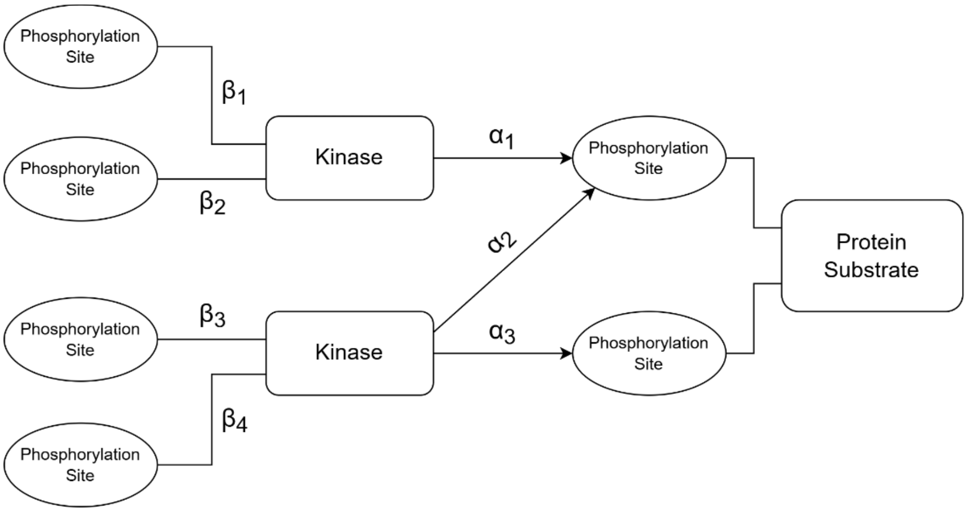

Figure 7: Schematic example of network dependencies between substrates, phosphorylation sites, and kinases. Protein substrates inherit phosphorylation sites, which are modified by kinases. The relative effect of kinases on phosphorylation sites is represented by α parameters. Kinases also inherit phosphorylation sites, and the relative impact of these sites on the kinase activity is represented by β parameters. In this example, a protein substrate is shown which inherits two phosphorylation sites. These sites are modified by two kinases present in the network, with one kinase phosphorylating one site, and the other kinase phosphorylating both sites. The two kinases inherit phosphorylation sites on their own, therefore making them substrates on which other kinases can act within the network.

The protein network structure incorporated in the present study contains reaction dependencies between protein substrates, their respective phosphorylation sites, and kinases phosphorylating these sites. An example of the structural network dependencies is depicted in figure 7. As currently only limited information is available on the relations between the network nodes in terms of relative intensities of the phosphorylation effects and the impacts on kinase activities, the present study suggests mathematical estimation of these dependencies by introducing additional activity parameters and optimizing their values.

The introduced α parameter displays the relative effect of a kinase on the phosphorylation of a respective protein site of the substrate. Two kinds of α parameters are established. On the one hand, the α_j^i parameter represents the effect of the j-th kinase on the i-th phosphorylation site of the substrate. The values of α_j^i therefore partly display the relations employed in the current network state described in section 1.4. However, the established protein dataset utilized in the present study does not cover all kinases and phosphorylation sites acting within the pathway the investigation focuses on. That is why, on the other hand, the α_i^0 parameter is introduced to represent the effects of unknown kinases on the phosphorylation site of the respective substrate.

To ensure relativity in contribution of effects between the kinases, a secondary condition for the α parameters is established stating that the sum of α_j^i values as well as the α_i^0 value equals 1 (Eq. 8). Accordingly, constraints for the α parameters were set between 0 and 1.

$$\alpha_i^0 + \sum_j \alpha_j^i = 1 \qquad \text{Eq. 8}$$

The β parameters are introduced to represent the contribution of phosphorylation sites of a kinase to its activity. Similar to the α parameters, two kinds of β parameters are established for the same reasons. The β_j^k parameters display the contribution of the k-th phosphorylation site to the activity of the j-th kinase, while the β_j^0 parameter represents the contribution of unknown phosphorylation sites currently not employed in the network state on the activity of the j-th kinase. As for the α parameters, the same secondary condition is established stating that the sum of β_j^k values as well as the β_j^0 value equals 1 (Eq. 9) to maintain relative contribution.

$$\beta_j^0 + \sum_k \beta_j^k = 1 \qquad \text{Eq. 9}$$

Since phosphorylation processes can catalyze but also inhibit enzymatic activity [28], it was decided to set constraints of all β parameters between -5 and $+5$ to give the parameter estimation process a wider optimizing space.

The established α and β parameters are implemented into an equation utilized as the fit function in the optimization process (Eq. 10). On the left-hand side, $P_i(t)$ represents the abundance of phosphorylated protein as measured in the dataset (Section 1.3). The right-hand side of the equation illustrates the dependency term comprised of the inner β activity term and the outer α activity term.

$$P_i(t) = \alpha_i^0 + \sum_j (\alpha_j^i (\beta_j^0 + \sum_k \beta_j^k P_j^k(t))) \qquad \text{Eq. 10}$$

The β_j^k parameters are multiplied with $P_j^k(t)$ term, which represents the respective protein abundance of the kinase whose activity is impacted by the β parameters. The term is therefore the main driver behind the network dependencies, as it establishes the link between the protein substrate $P_i(t)$ and the kinases $P_j^k(t)$ under the influence of the α and β parameters.

In the optimizing processes developed in the present study, all α and β parameter values are estimated with the target of minimizing the residual between the right-hand side of the fit function and the experimentally observed abundances of the phosphorylated proteins $P_i(t)$ from the dataset. The dependency parameters are estimated as time-invariant, as they represent constant biochemical effects on kinases and phosphorylation sites that remain stable throughout the simulated time course. The right-hand side activity term including the optimized parameter values is implemented into the ODE functions established in section 1.5 to employ dependencies between the chosen kinases of the EGFR/MAPK pathway.

$$\frac{dR}{dt} = A - BR \qquad \text{Eq. 2}$$

$$\frac{dP_0}{dt} = CR - P_0\left(D_0 + \sum_i^n S_i \left(\alpha_i^0 + \left(\sum_j \alpha_i^j \left(\beta_j^0 + \sum_k \beta_j^k P_j^k\right)\right)\right)\right) + \sum_{i=1}^n Z_i P_i \qquad \text{Eq. 11}$$

$$\frac{dP_i}{dt} = S_i P_0 \left(\alpha_i^0 + \left(\sum_j \alpha_i^j \left(\beta_j^0 + \sum_k \beta_j^k P_j^k\right)\right)\right) - P_i(D_i + Z_i) \quad \forall i \qquad \text{Eq. 12}$$

The final ODE functions utilized to simulate the protein network are employed as equations 2, 11, and 12. Within the functions representing the abundances of the unphosphorylated and phosphorylated protein substrate (Eq. 11 and 12), the activity term containing the α and β parameters is multiplied with the kinetic parameter S_i which represents the phosphorylation rate of the unphosphorylated protein substrate P_0.

Similar to the estimation of α and β parameters, the kinetic rate parameters are optimized by utilizing the ODE functions representing the protein abundances P_0 and P_i (Eq. 11 and 12) as fit functions and adjusting the kinetic parameter values for all 14 time points respectively to estimate fold change courses that align with the dataset.

2. Methods

2.1 Parameter Estimation of Network Dependencies

In order to establish biological dependencies between the observed kinases and their phosphorylation sites and implement them in the network model, it is required to estimate the parameter values of the activity weights α_i^0 and α_j^i as well as β_j^0 and β_j^k (Eq. 10).

To facilitate the implementation of dependencies between the observed kinases, it was assumed that the weight values obtained through optimization at mathematical level also reflect the corresponding real values at biological level. This simplification was adopted as limited information was available on the effects of the kinases and phosphorylation sites represented in the network. An accurate estimation of biologically plausible parameter values could potentially be achieved by introducing additional weights for the enzyme activity, provided that more comprehensive information on the mutual influences of enzymes and phosphorylation sites becomes available at a later stage.

Parameter optimization was carried out by defining equation 10 as the objective function. The term $P_i(t)$ represents the fold change measurements provided in the dataset for the respective protein P phosphorylated at phosphorylation site i. The activity weights were optimized by minimizing a loss function that quantifies the deviation between the model predictions and the experimental measurements.

The present study benchmarks two optimizers representing distinct classes of optimization algorithms by assessing their performance across different experimental conditions and modeling constraints. The comparison is intended to provide guidance on which optimization paradigm is preferable in specific scenarios, especially with respect to future network expansions and their implications for scalability and robustness.

$$MSE = \frac{1}{N \cdot T} \sum_{i=1}^{N} \sum_{t=1}^{T} (\hat{P}_i^{model}(t) - P_i^{data}(t)) \qquad \text{Eq. 13}$$

For both optimizers, equation 10 was utilized as the objective function to estimate the dependency parameters. Additionally, equation 13 is defined as the true fit function to calculate the Mean Squared Error (MSE) as representative difference between the results of the estimated parameters ($\hat{P}_i^{model}$) and the measured data (P_i^{data}). Here, N represents the number of proteins P_i currently featured in the network, while T represents all the measured time points.

2.1.1 Gradient-Based Optimization

The first optimizer developed in the present study is based on gradient descent parameter estimation algorithms employed through Flux [29] and Zygote [30] packages as well as the Adam optimizer in the Julia programming language [31]. In this approach, minimization of the MSE between model predictions and measured data across all proteins included in the network is achieved by calculating parameter updates through automatic differentiation, which are iteratively applied in the optimization progress. In the following, the script for gradient-based parameter optimization written in Julia will be referred to as Flux optimizer.

2.1.1.1 Initialization of Flux Optimizer

In the initial step of the parameter estimation process, the Comma-separated values (CSV) data file comprising information on the proteins considered in the network is uploaded into the cache [32]. More precisely, it includes fold change values for every time point from the original dataset, information on the network dependencies, and which phosphorylation site of the proteins is phosphorylated by which specific kinase. This information is converted into a data frame within the cache [33].

Subsequently, a numeric matrix P is generated, which contains the fold changes of every protein in the data file for every measured time point. For the network dependencies, two data lists are established. The first list comprises a vector containing information on which phosphorylation site i belongs to which protein/kinase j, whereas the second list comprises a vector that states which kinases j react on which phosphorylation site i. Since the two generated vectors differ in length, it is mandatory to establish pointer arrays to accurately map the α_j^i parameters to the correct phosphorylation site i and β_j^k parameters to the correct kinase j, respectively. By setting up pointer arrays, expansion of the protein network is later facilitated while maintaining numerical efficiency of the vector optimization.

2.1.1.2 Establishment of Parameter Boundaries

The true optimization parameter vectors ps can afterwards be established as a tuple for α_i^0 and α_j^i as well as β_j^0 and β_j^k. In this tuple, the parameters are categorized as raw variables θ for α and ρ for β, which are passed towards the optimization process. This is advantageous for applying the constraints mentioned in section 2.1 to the parameters, as otherwise they would need to be explicitly forced onto α and β in each optimization step. Instead, the Adam optimizer can operate in the free space of real numbers $\mathbb{R}$ and fulfil the constraints indirectly when transforming the raw variables θ and ρ into valid α and β values.

$$\text{softmax}(\theta^i) = \frac{e^{\theta^i}}{(\sum_1^m e^{\theta^i})} \qquad \text{Eq. 14}$$

In the Flux optimizer script, applying the parameter constraints for α is realized by implementing a softmax function [34], where $\sum softmax(\theta^i)$ equals 1 (Eq. 14). With integration of the bias term α_i^0, the equations 15 and 16 for the estimation of α values are employed onto the vectors θ by the softmax function integrated in the Flux package [29]. This ensures positivity of values in the vector contents as well as the result of summing up α values of a protein equaling $+1$.

$$\alpha_j^i = \frac{e^{\theta_{\alpha_j^i}}}{\left(\sum_1^m e^{\theta_{\alpha_j^i}}\right) + e^{\theta_{\alpha_i^0}}} \quad \text{for } j = 1, \ldots, m \qquad \text{Eq. 15}$$

$$\alpha_i^0 = \frac{e^{\theta_{\alpha_i^0}}}{\left(\sum_1^m e^{\theta_{\alpha_j^i}}\right) + e^{\theta_{\alpha_i^0}}} \qquad \text{Eq. 16}$$

For applying the parameter constraints to the β values, the hyperbolic tangent defined as tanh function [35] in Julia is utilized and employed onto the vectors ρ (Eq. 17).

$$\tanh(\rho) = \frac{e^{\rho} - e^{-\rho}}{e^{\rho} + e^{-\rho}} \qquad \text{Eq. 17}$$

Since the raw values in these vectors can take on any real number $\mathbb{R}$, the tanh function restricts greatly positive numbers towards a value of $+1$, while restricting greatly negative numbers towards a value of -1. Because constraints for the β values were chosen at $+5$ to -5, a multiplier depicted in equation 18 is introduced into the code.

$$\beta = 5 * \tanh(\rho) \qquad \text{Eq. 18}$$

With the tanh function implemented, the constraints for the β values can be realized. Normalizing the sum of β_j^k values and β_j^0 value to a result of $+1$ must be achieved by introducing another function for λ, which shifts all β values by the same amount (Eq. 19).

$$\lambda = \frac{1 - \sum \beta_j^k + \beta_j^0}{m} \quad \text{with } m = \text{amount of } \beta \text{ values} \qquad \text{Eq. 19}$$

The numeric value of λ is added to each β_j^k value as well as the β_j^0 value, thereby achieving a result of $+1$ when summing up all β values for one protein. However, by adding λ, it is possible for the final β values to settle outside the target constraints of $+5$ to -5.

$$viol = \max(0, |\beta|, -5) \qquad \text{Eq. 20}$$

Therefore, a penalty function (Eq. 20) is employed to track the violation of constraints, whose results will be considered and brought into the calculation of the overall loss at a later stage.

2.1.1.3 Forward-Pass

After parameter vector setup and integration of constraints for the α and β parameters, the Forward-Pass of the optimizing process is initiated for calculating a tangible model prediction. In order to calculate the fold changes for every protein, the Forward-Pass is first divided into two steps. The model equation (Eq. 10) is nested with two sum levels, with the inner layer calculating phosphorylation site activity of the kinases, and the outer layer calculating the activity of kinases on specific phosphorylation sites of a protein (Section 1.6). Therefore, two matrices can be established for each layer calculation of the Forward-Pass.

The first is an intermediate matrix B, which stores results of the phosphorylation site activity calculation. For that, the β_j^k and β_j^0 values are passed from constraint appliance to the Forward-Pass. Fold change values for every kinase are imported from the matrix P (Section 2.1.1.1). By applying the sub equation for the inner layer of the model equation (Eq. 10), results for terms comprising the β values can be calculated and stored within the matrix B.

Subsequently, values of the α_j^i and α_i^0 parameters are imported along with the results from matrix B to calculate the model output P_i. The output is afterwards stored in the matrix $\hat{P}$ for the outer layer of the model equation.

In the Forward-Pass, both matrices are established through the buffer function integrated in the Zygote package [30]. By utilizing buffers, the script automatically differentiates the matrices, therefore supporting the looping process of the iterative optimization, as standardized arrays might cause calculation problems.

2.1.1.4 Loss Function

For calculating the loss as difference between the observed data from the model and the measured data from the experiments, both matrices $\hat{P}$ and P were imported. Subsequently, observational masks W_{obs} and *site_has_kin* were established. The former verifies all the measured data points in matrix P while flagging missing values, whereas the latter examines the assignments of kinases to the phosphorylation sites of the proteins within the network. With introduction of W_{train} as a final training mask, both observational masks are evaluated and their information combined. Mathematically, W_{train} mask now corresponds to the number of indices *i,* that will be summated. In the *obs_count* mask, entries of W_{train} that are recognized as true will be counted.

$$MSE = (\hat{P} - P)^2 \qquad \text{Eq. 21}$$

The difference between observed and measured data can be calculated with the result being squared (Eq. 21). Utilizing the MSE in loss calculation is sensible for penalizing greater differences between model and data with stronger effects. Finally, the MSE for the entire model can be calculated under utilization of *obs_count* and summation of all entries within the W_{train} mask (Eq. 22).

$$MSE = \frac{1}{obs\_count} \sum_{(i,t)\in W_{train}} (\hat{P}_i(t) - P_i(t))^2 \qquad \text{Eq. 22}$$

In addition to the MSE calculation, the penalty term *beta_pen* for β values located outside the set constraints is calculated and included in the overall loss of the optimization.

2.1.1.5 Gradient-Based Optimization Algorithm

After establishing the matrices as well as the loss function, the initiation of the iterative training algorithm is approached. First, the currently employed set of parameters *ps* is utilized to calculate the values of the $\hat{P}$ matrix, which is then used to calculate the initial loss value of the optimization process.

With the initial loss value calculated, the optimizer can start determining the gradients of the current parameter set by calculating the derivatives of the parameter change over the change in loss function value by applying the gradient function implemented in the Zygote package [30].

$$grad\_ps = \left(\frac{\partial loss}{\partial \theta_{\alpha_j^i}}, \frac{\partial loss}{\partial \theta_{\alpha_i^0}}, \frac{\partial loss}{\partial \rho_{\beta_j^k}}, \frac{\partial loss}{\partial \rho_{\beta_j^0}} \right) \qquad \text{Eq. 23}$$

The calculation of the Zygote gradient function results in generation of a tuple $grad\_ps$, which contains four vectors representing the gradients of each parameter (Eq. 23). The loss derivatives in the tuple are then passed on to the Adam optimizer along with the initial parameters *ps* and the current state of iterations. For every entry in the parameter vectors, Adam stores a first momentum *m* representing the smoothed mean trend direction of the gradient, and a second momentum *v* representing the squared mean gradient value for measuring gradient size [36]. With the introduction of these quantities, the optimizing procedure of Adam differs from standard gradient-based algorithms, as the gathered information helps analyzing which step size can be chosen and how careful the parameters should be updated. The true optimization and update of parameters is carried out according to equation 24, with ω as the parameter to be updated, ε as a small stabilization constant, and η as learning rate [36].

$$\omega^{i+1} = \omega^i - \eta * \frac{m_i}{\sqrt{v_i} + \varepsilon} \qquad \text{Eq. 24}$$

By utilizing the update equation, direction of gradient and scale of step size are impacting the generation of the new set of parameters, as these quantities act inversely proportional. In general, the Adam optimizer is utilizing adaptive step sizes per parameter to increase loss minimization per iteration and stabilize oscillating gradients as well as noisy gradient direction [36]. The updated set of parameters is fed back into the Forward-Pass of the iteration loop to start the next optimization cycle, while the state of optimization with Adam is updated as well.

In the final step of the gradient-based optimization progress, the raw parameters are imported from cache once the last of the manually defined iterations is reached. The true α and β values for every protein, kinase, and phosphorylation site are calculated from the tuples and stored in data files for later use in the ODE network simulation.

2.1.2 Interior-Point Optimization

The second optimizer developed in the present study is based on Interior Point optimization algorithms of parameters employed through the JuMP [37] and Ipopt [38] packages. In contrast to the gradient-based approach, the interior point optimizer defines parameter estimation as a constrained nonlinear optimization problem in which the parameters are treated as explicit decision variables and optimized directly while applying bound parameter constraints. The model parameters are obtained by minimizing the MSE between predicted and measured data, while applying structural conditions of the network within the optimization procedure itself. In the following, the script for the interior point–based parameter optimization written in Julia will be referred to as the JuMP optimizer.

2.1.2.1 Initialization of JuMP Optimizer

In first instance, the CSV data file containing information on the measured fold change values of all proteins within the network as well as their phosphorylation dependencies is uploaded into the cache of Julia [32] and subsequently converted into a data frame [33] while verifying the existence of predefined columns for proteins, phosphorylation sites and kinases. The contents of the data frame are cleaned to be converted into strings. For every protein and phosphorylation site measured, rows are inserted into vectors of a tuple, which now contains all the information of the measured data. Subsequently, dictionary *P_data* is defined from this tuple to act as an accessible data storage from which information can later be easily retrieved for building arrays and matrices.

In the following step, *alpha_list* is established with every entry corresponding to one α_j^i parameter term. In the JuMP optimizer script, α parameters are not reconstructed as pointer arrays, but modelled as explicit optimization variables. Simultaneously, *beta_set* is established as a quantity of tuples containing kinases and phosphorylation sites. By comparing and matching the kinase entries from *alpha_list* with the measured phosphorylation sites in the *P_data* dictionary, candidates for β_j^k can be identified and stored in a *unique_betas* list. To identify which α parameters act on the phosphorylation sites *i*, the *targets* dictionary is established storing lists of all indices in *alpha_list* that act on the respective target site. By establishing the *beta_groups* dictionary, a similar assignment is conducted to map which β_j^k belong to the kinases j and convert these dependencies to indices. In general, converting string identifiers of proteins, phosphorylation sites, and kinases into index assignments is mandatory for the JuMP optimizer to achieve higher computational power in the later stages of parameter estimation.

With introduction of the *Pval* 3D array, a great numeric data structure consisting of the α index *i* (first dimension), the β index *j* (second dimension), and the time point *t* (third dimension) is established, which can be utilized to calculate the needed sums in the JuMP optimization. In preparation to compare parameter estimation results with the measured data, the matrix *Y_obs* is created to store every measured fold change value of every protein at the respective time points.

2.1.2.2 Prediction Process

After establishing and formatting the data structures needed for the optimization, prediction of parameters and loss calculation is initiated. Similar to the Forward-Pass of the gradient-based optimization (Section 2.1.1.3), the function *predict_and_losses* is implemented to take on the tasks of calculating model predictions, compare them with measured data, and calculate the MSE to later update the estimated parameter values. As an input, the function receives four vectors containing values of α_j^i and α_i^0 as well as β_j^k and β_j^0 as final and tangible model parameters instead of raw parameters.

$$prediction = pred \in \mathbb{R}^{k_{sites} * t} \quad \text{Eq. 25}$$

In the initial step of the prediction, a subfunction *pred* (Eq. 25) is introduced which creates the prediction matrix containing the predicted values for the respective target site *k* at time point *t*. As the site-specific structure of the prediction is established, the bias term α_i^0 is expressed first, since it represents the consistent part of the outer model equation calculating the activity of kinases on specific phosphorylation sites of a protein (Section 1.6). Afterwards, values are summed across all respective α_j^i parameters belonging to the target site. For every respective α_j^i parameter, it is determined which kinase represents each term to subsequently initiate the establishment of the inner layer for calculating phosphorylation site activity of the kinases (Section 1.6). In the script, calculation of the inner layer is initiated by establishing the bias term β_j^0. Afterwards, values are summed across all respective β_j^k parameters belonging to the respective kinase. All α_j^i and β_j^k values for prediction of known dependencies are retrieved from the *Pval* array. Once creating the inner and outer layer of the model equation (Eq. 10) by summation of respective terms is completed, the final prediction value $\hat{P}_i(t)$ for every protein is calculated and stored as an entry in the *pred* prediction matrix. Subsequently, the MSE can be calculated by utilizing equation 13 (Section 2.1).

2.1.2.3 Formulating the Optimization Problem

With the helper function *predict_and_losses* established, the true optimization model can be formulated using the JuMP [37] and Ipopt [38] packages. The vectors containing α_j^i and α_i^0 as well as β_j^k and β_j^0 represent the desired model parameters with secondary conditions and boundary constraints (Section 1.6) implemented directly within the optimization problem.

The target function *@NLobjective* is initially defined similar to the objective function for the gradient-based optimization (Eq. 10). When the *optimize!* function is initiated, starting point values are assigned to each set of parameters, with α_j^i and β_j^k vectors receiving values of $+1$ and α_i^0 and β_j^0 vectors receiving values of 0. Initiating the function already meets the summation requirements in form of the fit function for the respective proteins to set a valid starting point for the Interior Point optimizer. Just before the iterative optimization process starts, an initial loss value is calculated with the *predict_and_losses* function.

2.1.2.4 Optimization Process

Once the requirements for optimization are fulfilled, the Interior Point optimizer starts the iterative solution process. In this context, the solver interface provided by the MathOptInterface package [39] is used to implement the Ipopt callback for logging optimization progress. The optimizer then searches for a parameter vector that minimizes the objective function while satisfying the defined secondary conditions. To prevent violation of these conditions during optimization, barrier functions are incorporated into the solution process [40].

$$v_{\alpha} = -\mu \log(\alpha) - \mu \log(1-\alpha) \quad \text{Eq. 26}$$

To set constraints to the α parameters, a boundary function is applied as depicted in equation 26. With this function implemented, values for α are positive and lie between 0 and $+1$. Moreover, the implementation of the barrier weights μ serves to penalize parameter values pushing towards the boundary limits.

$$v_{\beta} = -\mu \log(\beta - (-5)) - \mu \log(5-\beta) \quad \text{Eq. 27}$$

By implementing the second barrier function depicted in equation 27, boundaries for the β values can be applied. With use of logarithmic conversion as well as setting barrier weights μ, values of the β parameter are limited within constraints between -5 and $+5$.

$$\Phi(\alpha_j^i, \alpha_i^0, \beta_j^k, \beta_j^0) = \hat{P}_i(\alpha_j^i, \alpha_i^0, \beta_j^k, \beta_j^0) + B(\alpha_j^i, \alpha_i^0, \beta_j^k, \beta_j^0) \quad \text{Eq. 28}$$

With setup of the barrier functions being completed, the sub equation 28 can be formulated. In context of the equation, $\hat{P}_i$ represents the term aiming to fit the model output as close as possible to the measured data, while B represents the sum of implemented barrier functions to keep the variables within the permissible range.

Although boundary constraints are now established through utilization of barrier functions, other secondary conditions must be fulfilled as well. For the implementation of the condition that the sum of respective α_j^i and α_i^0 values as well as β_j^k and β_j^0 values of a protein must equal a value of $+1$, a Lagrange function is utilized as depicted in equation 29 [40].

$$\mathcal{L}(\alpha_j^i, \alpha_i^0, \beta_j^k, \beta_j^0, \lambda) = \Phi(\alpha_j^i, \alpha_i^0, \beta_j^k, \beta_j^0) + \sum_{i=1}^{i_{sites}} \lambda_i \left(\alpha_i^0 + \sum \alpha_j^i - 1\right) + \sum_{j=1}^{j_{kinases}} \lambda_j \left(\beta_j^0 + \sum \beta_j^k - 1\right) \quad \text{Eq. 29}$$

The Φ term of the Lagrange equation represents the first part fitting the model to the measured data under consideration of parameter constraints (Eq. 28). The second part of the equation works towards fulfilling the secondary conditions regarding the sums of respective α and β values by introducing a Lagrange multiplier λ for penalizing the violation of the conditions with adaptive strength. The stronger the constraint violation, the higher the numeric penalty flowing back into the result of the Lagrange function [40].

The Interior Point optimizer integrated in the Ipopt package [38] searches for the optimal result of the Lagrange function where the best fit the model to the data is achieved while satisfying all the secondary constraints by calculating derivatives of $\mathcal{L}$ over every parameter to be optimized (Eq. 30).

$$\frac{\partial \mathcal{L}}{\partial \alpha_j^i} = \frac{\partial \mathcal{L}}{\partial \alpha_i^0} = \frac{\partial \mathcal{L}}{\partial \beta_j^k} = \frac{\partial \mathcal{L}}{\partial \beta_j^0} = 0 \quad \text{Eq. 30}$$

From the derivatives, the Interior Point optimizer utilizes the gradients and furthermore constructs a system of Karush-Kuhn-Tucker conditions (KKT) to calculate the optimum that meets them. Calculation is initiated by establishing a vector function $F(z)$ containing the adjustable parameters (Eq. 31) [40].

$$F(z) = \begin{pmatrix} \nabla\mathcal{L}(\lambda) \\ (\alpha_j^i, \alpha_i^0) \\ (\beta_j^k, \beta_j^0) \end{pmatrix} \quad \text{Eq. 31}$$

The optimizer searches for parameter values leading the calculative result of $F(z^*) = 0$. Since $F(z)$ is a nonlinear equation system, it is only iteratively solvable by performing linear approximation around the current point z in each iteration.

$$F(z + \Delta z) \approx F(z) + J\,\Delta z \quad \text{Eq. 32}$$

$$J = \frac{\partial F}{\partial z}(z) \quad \text{Eq. 33}$$

Linearization is applied locally utilizing equation 32, where J represents the Jacobi matrix of the KKT system (Eq. 33) [40]. This matrix contains the derivatives of the KKT equations with respect to all unknown variables. By calculating the first derivatives of the equations as well as the second derivatives by applying Hessian matrices, the Interior Point optimizer receives information on the direction of gradient as well as curvature of the optimization landscape. This information enables calculation of a much more precise optimizing step known as Newton step by rearranging equation 32, which gives rise to a linear equation system with unknown correction Δz (Eq. 34).

$$\Delta z = (\Delta\alpha_j^i, \Delta\alpha_i^0, \beta_j^k, \beta_j^0, \Delta\lambda) \quad \text{Eq. 34}$$

When solving this linear equation system, the Interior Point optimizer receives a direction in which violations of the steady state and the constraints are reduced with regard to the previous state. The optimizer then analyses if the new state point inherits smaller KKT residuals while improving barrier function results and constraint violations, before updating the parameters to calculate a new state point in the iterative optimizing approach. Once $F(z)$ inherits a value of (nearly) 0, the optimizer stops, since all conditions in terms of parameter adaptations are met. The accuracy of this estimate can be adjusted manually within the script [38].

Once the optimization process finishes, the values of the optimized α and β parameters are extracted from the cache and inserted into the *predict_and_losses* function one last time to receive the final model prediction and error measure. The final α and β values are stored in data files for later use in the ODE network simulation.

2.1.3 Methodological Comparison of both Optimizers

The two optimizers established in the present study were developed to estimate the dependency parameters α_j^i, α_i^0, β_j^k and β_j^0 of the signaling network in such a way that the model predictions conform to the experimentally observed fold change data of the network proteins as close as possible. Despite relying on different optimization paradigms, both optimizers share the same biological objective and modelling approach as well as distinct parts in the structure of their scripts developed in Julia, thereby allowing methodological comparison.

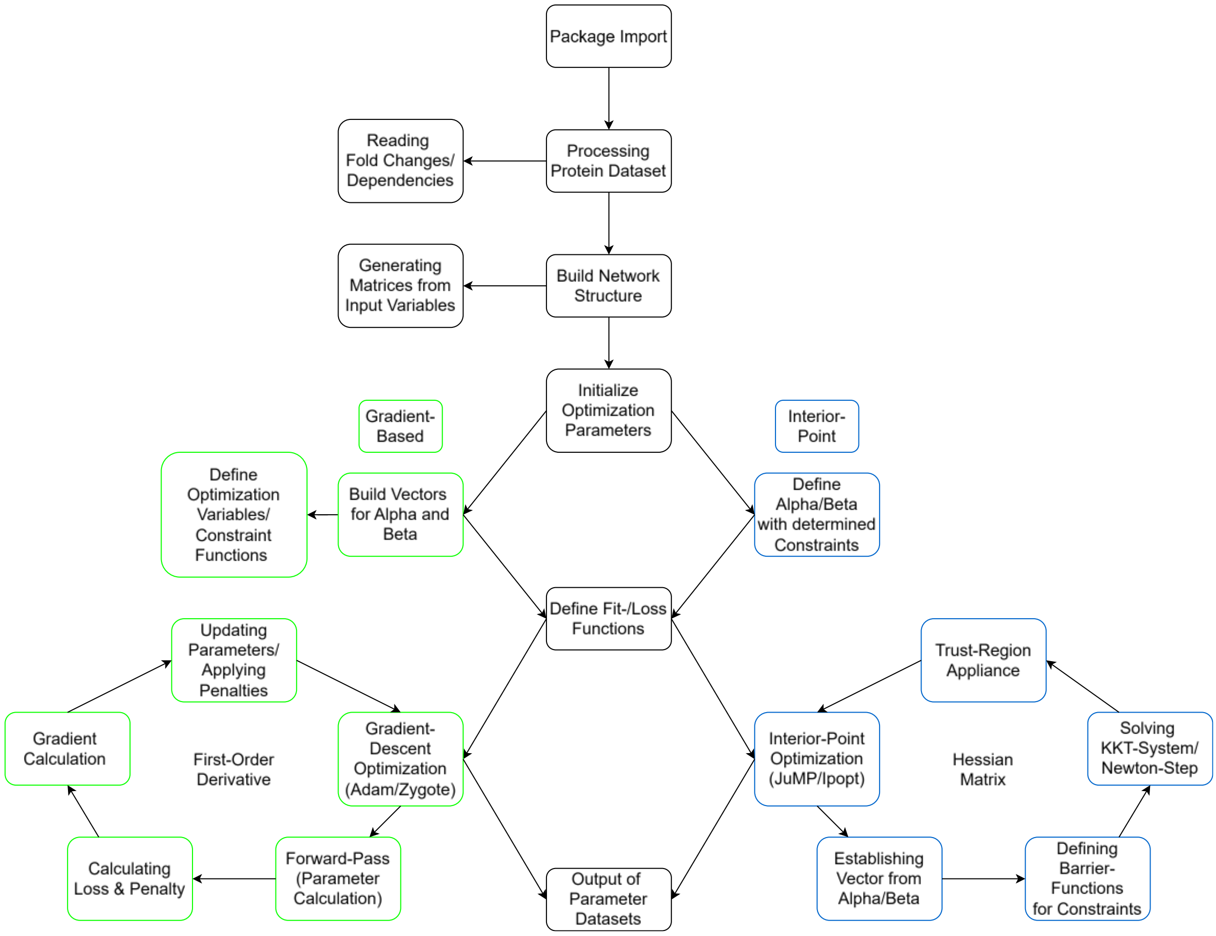

Figure 8: Schematic representation of code structure for the Network dependency parameter Optimizer. The flowchart shows similarities in the structure of the scripts (Black Boxes) as well as differences between Gradient-Based Optimization (Green Boxes) and Interior-Point Optimization (Blue Boxes).

Within both optimizers, processing the information from the incorporated measured protein fold change dataset as well as analyzing the dependencies between kinases and phosphorylation sites progresses in similar ways by creating arrays, dictionaries and tuples to transfer and build the network structure (Fig. 8). Moreover, estimating parameter values is initiated in similar ways by establishing the inner β kinase term as well as the outer α site term by the Forward-Pass in the Flux script and the *predict_and_losses* function in the JuMP script, respectively. Predicting the loss by calculating the MSE value in the iterative approach of the optimization is also identical between the two scripts.

However, the central methodological difference between the optimizers is handling of the parameters whose values are estimated in the optimization process. In the Flux script of the gradient based optimizer, the biologically interpretable parameters α and β are only indirectly optimized, as they are embedded in raw parameters θ and ρ. From these raw parameters, the α and β values are derived under appliance of constraint functions. Therefore, the Flux optimizer operates in a reparametrized, unconstrained raw parameter space, from which the biological weights are generated only as a secondary step.

In contrast, the biologically interpretable parameters α and β themselves pose the optimization variables in the JuMP script of the Interior Point optimizer. Constraints are applied directly on these variables by utilizing barrier functions, while secondary conditions appear as linear equations in the optimization model. The JuMP script operates within the original variable space of the biologically interpretable parameters.

The main distinction between the two optimizing approaches is posed by the different algorithms utilizing in the parameter estimation process. In the Flux script of the gradient-based optimizer, a first-order method is utilizing, where every training iteration proceeds by calculating the current loss, estimating the gradient of the loss for every raw parameter, and applying it in the Adam optimizer to update the parameter values. Adam utilizes every block of parameters for estimating the first and second gradient momentums to generate adaptive step sizes. Training therefore involves many small and iteratively adjusted step sizes that are based only on gradient information.

In the JuMP script of the Interior Point optimizer, constraints and secondary conditions are integrated in the parameter estimation process as linear equations, which are solved by establishing KKT systems, which are linearized for every iteration. From there, Newton steps are calculated that update all variables simultaneously. In contrast to Adam, the Interior Point optimizer utilizes the gradient as well as curvature information from Lagrange functions for accurate parameter estimation.

2.2 Kinetic Parameter Optimization & Network Simulation

The ODE network comprising the selected kinases (Section 1.4) is established by integrating the optimized dependency parameters into the model equations, estimating the values of the kinetic parameters, and solving the ODE system.

In this framework, the optimized α and β parameters are incorporated into the time-dependent ODE network model established from the distributive mass action kinetics phosphorylation approach (Section 1.5). The model equations 11 and 12 for calculation of the abundances of the unphosphorylated and phosphorylated proteins $P_0(t)$ and $P_i(t)$ are utilized as the objective functions to calculate the proteomic fold change values. The kinetic parameters are optimized within their predefined constraints to replicate the measured fold changes from the protein dataset (Section 1.3).

The model simulation is comprised of three scripts written in Julia. The main script is defined to control and coordinate the overall modeling procedures, while the optimizing script contains the protein-specific optimization logic for the kinetic parameters. The network script incorporates the definition of the kinase model, the ODE simulation and the output functions.

2.2.1 Initialization of Kinetic Optimizer

In the initial step of the estimation process for the kinetic parameters, the input CSV data files are retrieved by the main script [32]. The separate files contain the optimized dependency parameters α and β, the network dependencies between kinases and phosphorylation sites, the observed fold change values from the protein dataset, and the blueprint of kinetic parameters slated for optimization.

In the network script, the information is converted into data frames [33]. Based on the information retrieved on the dependencies within the protein network, a list is generated containing every substrate that will represent a network node. Additionally, a dictionary is established which stores corresponding phosphorylation sites per substrate. Another dictionary stores the assigned kinases which react on the respective phosphorylation sites. The data from the optimization of the α and β parameters is analyzed and translated into structures that make the α_i^0 and α_j^i values as well as the β_j^k and β_j^0 values accessible for importing the respective dependency parameters later.

After the topology and parameters are converted into data frames [33], a model cache is created in the main script, which includes protein substrates, phosphorylation sites, kinases, dependency parameters, observed fold changes, time points, and the network layout. The cache thereby combines all the information needed for optimization and simulation.

2.2.2 Generating the internal Model State Structure

For every protein substrate, a state block is established which comprises of three different substates. The first substate represents the mRNA abundance R calculated by equation 2, where the upstream mRNA production state is synthesized by A and degraded by B. The second substate represents the abundance of the unphosphorylated protein P_0 calculated by equation 11. P_0 is produced by R, degraded on a basal level by D_0, consumed by phosphorylation S_i, and regained through dephosphorylation D_i. The third substate represents the phosphorylated protein P_i calculated by equation 12, which is produced by phosphorylation of P_0, degraded by D_i, and dephosphorylated by Z_i.

The ODE solver does not process isolated proteins but instead integrates them into a unified global state vector which contains the state blocks for every protein substrate in the network. Since the network dynamics are time-dependent, an index dictionary is created to assign time point indices to every substate in the global vector.

For the implementation of the α and β parameters, a dependency term is established to be integrated in the calculations of the ODE functions. This term resembles the right-hand side of equation 10 and is defined as the activity term $Act_i(t)$ to represent the network dependencies in the ODE solving process (Eq. 35).

$$\alpha_i^0 + \sum_j(\alpha_j^i(\beta_j^0 + \sum_k \beta_j^k P_j^k(t))) = Act_i(t) \qquad \text{Eq. 35}$$

In the process of optimizing the kinetic parameters, the term $P_j^k(t)$ in equation 35 embodying the kinases phosphorylating the respective protein substrate is represented by the fold change values of the observed protein dataset rather than the kinase states calculated withing the optimization process to ensure global convergence and stabilize the optimization.

2.2.3 Establishment of ODE Function

The central equation in the global network is embedded in the *f_ode!* function, in which an ODE state block is calculated for every protein substrate. Within this function, global kinetic parameters A, B, C, and D_0 as well as site-specific parameters S_i, D_i, and Z_i are initialized via a subfunction which interpolates between the measured time points making the parameters not static but time-dependent. Subsequently, the specific equations for mRNA, protein, and phosphoprotein calculations are established.

Since the β parameters can inherit negative numeric values, the entire activity term established in section 2.2.2 can become negative. This can result in the calculation of non-physical negative fold change values for the protein abundances. Moreover, the fold changes tend to diverge further into the negative domain in the process of solving the ODE functions, reaching conditions that render numerical solvers unstable. Consequently, this issue compromises the integrity of the simulation process. For this reason, a gating function ϕ is implemented into the calculation of the activity term (Eq. 36).

$$\phi(Act_i) = \begin{cases} Act_i, & Act_i \geq 0, \\ 0, & Act_i < 0. \end{cases} \qquad \text{Eq. 36}$$

Integration of the gating function in the optimization process prevents negative inflows of the activity term from generating negative fold change values. When the activity term assumes positive values, they remain unaffected by the gating function. However, if the term assumes a negative value, the gating function resolves to 0. To provide flexibility, two different gating functions were implemented, and the desired method can be selected within the configuration settings of the main script.

$$\mathrm{ReLU}(\mathrm{x}) = \max(0, x) = \frac{x + |x|}{2} \qquad \text{Eq. 37}$$

The rectified linear unit (ReLU) function is a strict threshold function, as negative inputs are immediately eliminated by being pushed towards a value of 0, whereas positive values are passed on without changes (Eq. 37) [41].

$$\phi(Act_i) = \frac{Act_i + |Act_i|}{2} \qquad \text{Eq. 38}$$

Regarding the gating of the activity term (Eq. 38), the ReLU function is numerically efficient but not smoothed at $Act_i = 0$, where it exhibits a non-differentiable inflection point. The abrupt change of derivative behavior at the inflection point potentially complicates numeric handling of the system for the differential solver [42].

$$\mathrm{softplus}(x) = \log(1 + e^x) \qquad \text{Eq. 39}$$

As an alternative to the hard threshold of ReLU, the Softplus function is introduced which offers a smooth, continuous approximation for gating the activity term (Eq. 39) [43].

$$\phi(Act_i) = \frac{1}{k} \log(1 + e^{k(Act_i)}) \qquad \text{Eq. 40}$$

When applied to the activity term, an additional smoothing variable k is introduced (Eq. 40), which controls the steepness of the transition and can be adjusted in the configuration with the condition of $k > 0$.

$$\frac{dP_0}{dt} = CR - P_0(D_0 + \sum_i^n S_i\, \phi(Act_i)) + \sum_{i=1}^n Z_i P_i \qquad \text{Eq. 41}$$

$$\frac{dP_i}{dt} = S_i P_0 \phi(Act_i) - P_i(D_i + Z_i) \quad \forall i \qquad \text{Eq. 42}$$

For integration of the respective gating function into the calculation of protein abundances, the equations 11 and 12 (Section 1.6) can be rearranged which gives rise to the equations 41 and 42 with the integrated activity term as well as the employment of the gating function ϕ. These final equations are utilized in the ODE solving process.

2.2.4 Optimization of Kinetic Parameters

With the concluded establishment of the model state structure and the ODE function, the optimization process of the kinetic model parameters is initiated. As explained in section 2.2.2, the parameters are not globally estimated across the entire network, but protein-wise. The optimization is performed against the experimental fold change time series of proteins and their respective sites. Only after this protein-specific optimization are the resulting parameters integrated into the interdependent global simulation of the complete ODE network.

To identify the isolated parameters that need to be estimated for every time point, the parameter rows are retrieved from the data frame depending on the substrate-, time- and phosphorylation site index. Thereby, each optimizable parameter inherits a unique combination of indices. The kinetic parameters are subsequently stored in a time-dependent vector x.

$$A(t), B(t), C(t), D_0(t), S_i(t), D_i(t), Z_i(t) \in [0,2]$$

Global constraints are defined as dictionaries and applied to all kinetic parameters. As the fold change values are calculated as relatives in the ODE functions, parameter constraints were set between 0 and 2 to limit the rates of change and align them with real biological conditions.

In the initial phase of the optimizing process, the measured fold change data is mapped to be utilized as an estimation target and for retrieving the fold change values of the kinases $P_j^k(t)$ in the independent protein-wise optimization. Subsequently, the final ODE functions represented by the equations 2, 41 and 42 are retrieved to establish an ODE subsystem for every protein substrate. This subsystem is solved by a numerical algorithm selected in the configuration of the main script. In the present study, the Tsit5 solver integrated in the OrdinaryDiffEq package [44] was utilized for calculating the solution of the ODE problem, as it is the recommend default solver for non-stiff differential equation problem providing fifth-order accuracy and excellent efficiency [45].

$$\mathcal{L}_{protein} = MAE_{protein} + \lambda * \mathcal{L}_{prior} \qquad \text{Eq. 43}$$

The optimization process is driven by minimizing the loss term defined in equation 43 which comprises of two sub terms, namely the calculation of the Mean Absolute Error (MAE) and the prior loss term.

$$MAE_{protein} = \frac{1}{N_{protein}} \sum_{(i,t)\in P} \left| P_{protein,i}^{model}(t) - P_{protein,i}^{data}(t) \right| \qquad \text{Eq. 44}$$

The MAE is obtained by calculating the absolute difference between the fold change values estimated in the model and the measured fold changes from the dataset (Eq. 44). Therein, P comprises the unphosphorylated protein state $P_0(t)$ as well as the phosphoprotein states $P_i(t)$.

The prior loss term integrated in equation 43 is established to prevent specific issues regarding the assignment process of the values for parameters with opposite effects on the mRNA- and protein abundances. When a parameter governing the synthesis of mRNA, protein, or phosphoprotein shares the same value as its corresponding degradation parameter, the two offset each other. This results in a steady state where their combined effect on the abundance is nullified.

$$\mathcal{L}_{prior} = \sum_{v \epsilon A,B,C,D,S,Z} (\log(v + \varepsilon) - \log(p_0))^2 \quad \text{Eq. 45}$$

To prevent this issue, the prior loss term is implemented into the optimization process as defined in equation 45 [46]. The term applies a logarithmic transformation to the kinetic parameters represented by v, to which the variable ε is previously added. This variable contains a small numeric value to prevent the logarithmic term from becoming 0 if the respective kinetic parameter inherits a value of 0. The variable p_0, representing a manually determined configuration value, is log-transformed and subtracted from the kinetic parameter term. This mechanism biases the kinetic parameters toward the value of p_0. In this study, p_0 was set to 1, as this represents the unit value at which the derived rate of change is zero. Consequently, the kinetic parameters are encouraged to fluctuate around unity, which is consistent with biological plausibility. The λ variable implemented in the overall loss function in equation 43 is utilized as a weight parameter to manually control the impact of the prior term on the calculation of the overall loss [46].

2.2.5 Initialization of Iterative Optimization

The process of parameter estimation is initiated by importing the kinetic parameter entries for every respective protein from the global parameter vector x described in section 2.2.4. Subsequently, the Adam optimizing algorithm is employed by creating two additional vectors m and v for storing data regarding the iteration progress and gradient state.

In the beginning of the iteration loop, a perturbational vector δ is established which stores randomly generated values for defining a stochastic direction within the local parameter domain. The appliance of the perturbation creates variants of parameters with small positive as well as small negative perturbations, namely x^+ and x^-, which are immediately clamped within the determined constraints between 0 and 2 [47]. The applied perturbations are utilized as an alternative to calculating the derivatives for the entire ODE simulation, which is more complex and therefore consumes time and computational power. Perturbing the parameters in a positive and negative direction causes a gradient change which indicates the course expressing a lower loss value. As described in section 2.2.4, the Tsit5 solver [45] is applied to simulate the ODE system and gain results for both perturbation variants of the global parameter vector. These results are compared to the observed fold change values to obtain the MAE, the prior loss, and finally the overall loss for both parameter variants l^+ and l^- [47].

$$P_{scale} = \frac{l^+ - l^-}{2c} \quad \text{Eq. 46}$$

From the difference between l^+ and l^-, a gradient approximation is constructed. First, a scale vector is calculated according to equation 46, where the variable c represents the difference between the parameter variants and the currently employed set of parameters. The vector is then utilized to calculate the gradient $\hat{g}$ as expressed in equation 47.

$$\hat{g} = P_{scale} * \delta \quad \text{Eq. 47}$$

The estimated gradient is passed on to update the Adam optimizer. First, the vectors m and v containing information regarding the gradient state are recalculated to be utilized in the subsequent calculation of the step size of the corresponding time point according to equation 48 with η representing the manually adjusted learning rate [36].

$$step_t = \eta * \frac{\hat{m}_t}{\hat{v}_t + \varepsilon} \qquad \text{Eq. 48}$$

The local set of employed parameters is then updated by subtracting the calculated step (Eq. 49).

$$x_{t+1} = x_t - step_t \qquad \text{Eq. 49}$$

After this update, the new set of parameters is clamped one more time to ensure that all values are located within the determined constraints. The iteration loop is then reinitiated to further optimize the set of parameters until the maximum number of iterations defined in the configuration of the main script is reached. The set of parameters which achieved the lowest overall loss is subsequently stored in a CSV file for utilization in the dependent ODE simulation.

2.2.6 Simulation of dependent ODE Network

The simulation of the ODE Network comprising active dependency relations between the kinases in initiated once the optimized kinetic parameters are established. The parameter set is immediately retrieved and imported into the dependent run of the network script along with the model structure (Section 2.2.2), the experimental fold change data, and the dependency parameters α and β. Within the dependent simulation, the same state vector containing the ODE system for every protein is utilized to simulate not the isolated protein subsystem but the entire network. The significant difference between the independent and dependent simulations is the employment of the $P_j^k(t)$ kinase term located in the activity equation (Eq. 35). While $P_j^k(t)$ is representing the fold change values from the experimental set of data in the independent simulation, it now incorporates the kinase terms calculated by the global ODE systems in real-time. Thereby, proteins become recursively interlinked as the activity of a kinase now affects the phosphorylation rates of their downstream target sites, which in turn influence other kinases within the network.

The dependent ODE problem is formulated globally and solved using the algorithm specified in the main configuration script. In contrast to the protein-wise optimization, the complete time course between 0 and 960 minutes is calculated in a single ODE simulation run for every kinase simultaneously. The calculated fold change values are stored in CSV files to facilitate subsequent data processing.

3. Results & Discussion

3.1 Parameter Estimation of Network Dependencies

Performing the optimization utilizing the gradient-based and Interior Point optimizer results in generation of two respective datasets containing the optimized values for the α and β parameters. In the following section, results are presented and performance of both optimizers is analyzed by benchmarking certain criteria and comparing them.

3.1.1 Results of α Parameter Estimation

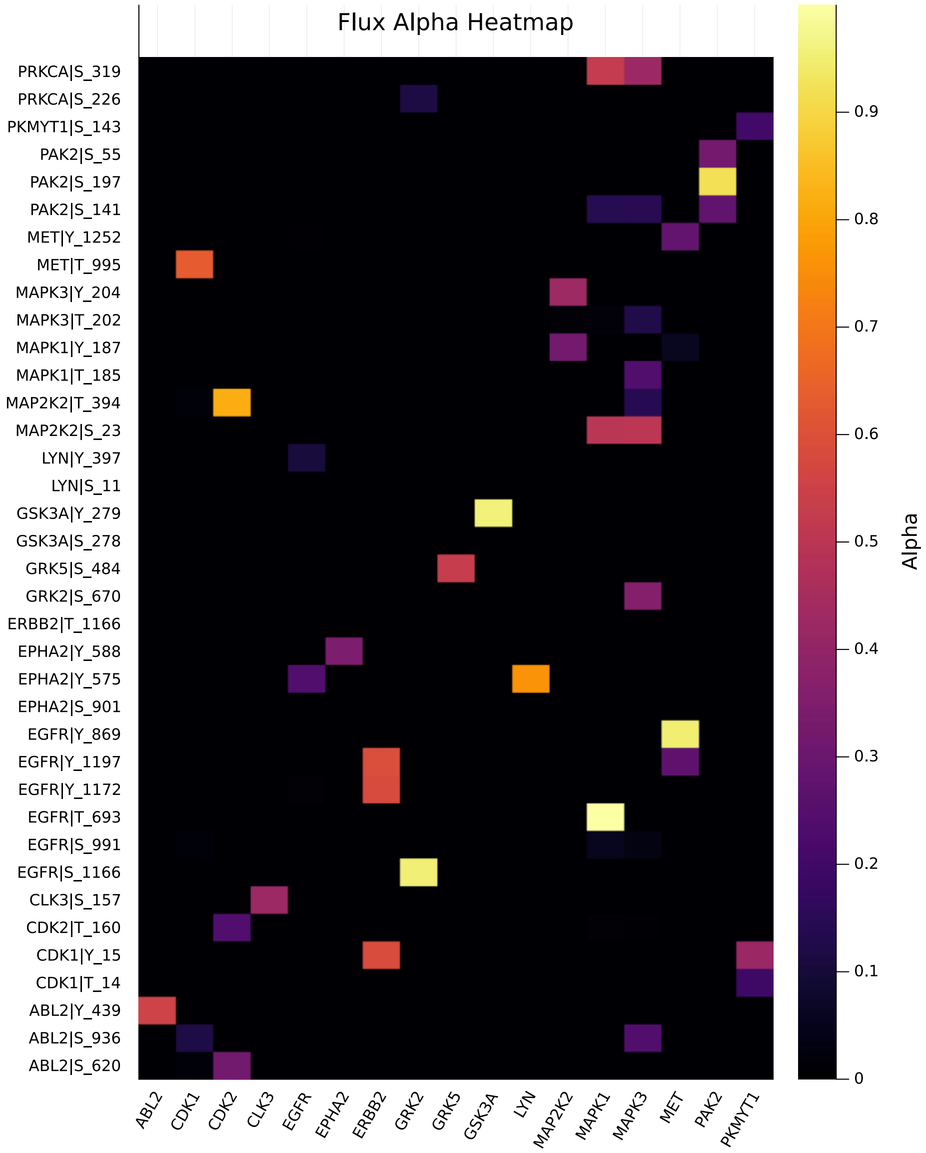

Figure 9: Heatmap depicting estimated values of α_j^i calculated by gradient-based optimization (Flux). Rows represent phosphorylation sites and columns represent their associated upstream kinases. Color scale indicates magnitude of estimated α_j^i values and therefore the effect of a kinase on phosphorylation of a protein.

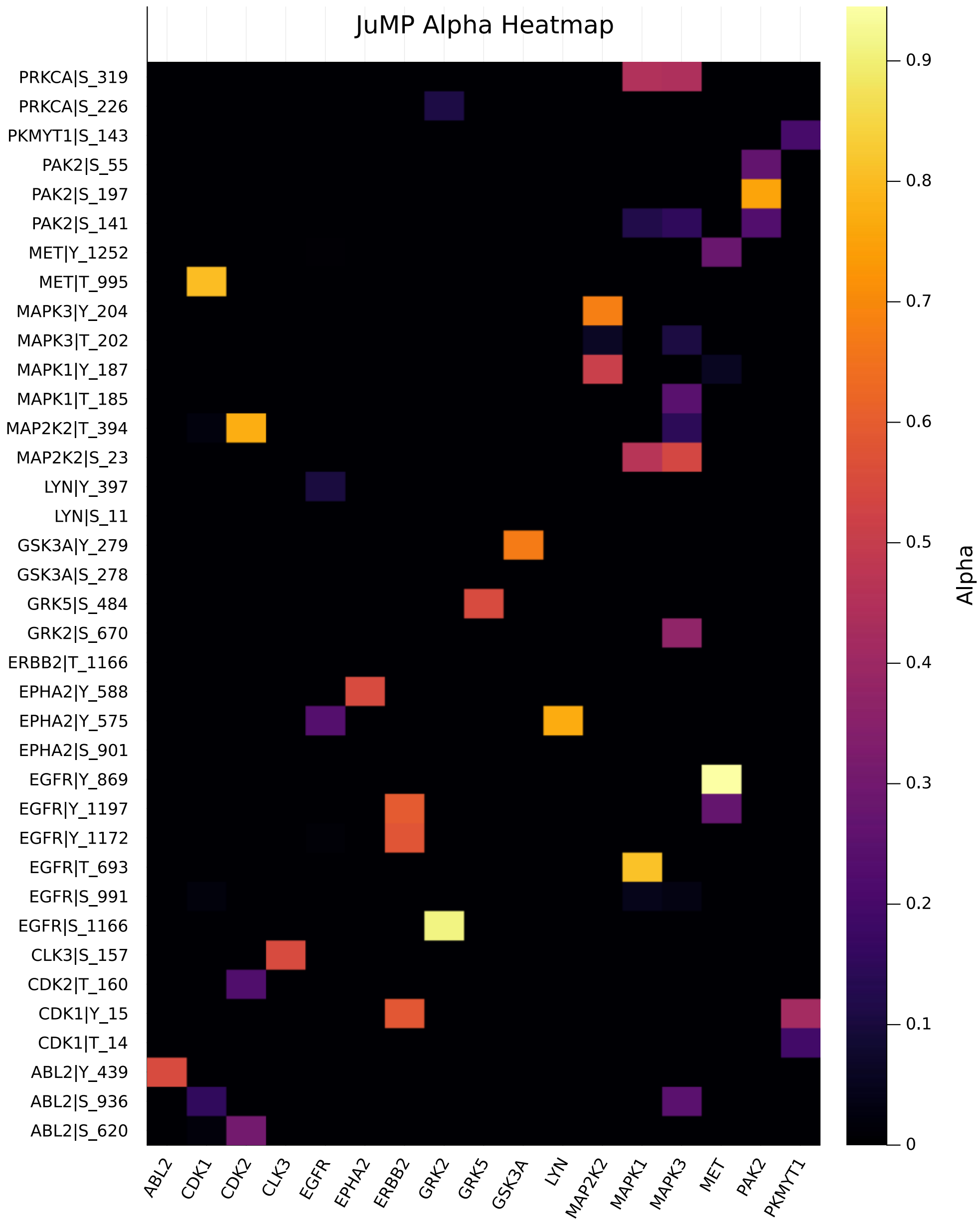

Figure 10: Heatmap depicting estimated values of α_j^i calculated by Interior Point optimization (JuMP). Rows represent phosphorylation sites and columns represent their associated upstream kinases. Color scale indicates magnitude of estimated α_j^i values and therefore the effect of a kinase on phosphorylation of a protein.

The overall sparsity of the α_j^i heatmaps is largely imposed by the predefined kinase and phosphorylation site relations integrated in the input dataset of the optimizers and is therefore no emergent property of the optimization itself. Instead, the informative aspect of the heatmaps is the relative distribution of α_j^i values within the set of permitted kinase candidates for each phosphorylation site and their differing magnitude depicted by the color scale in figures 9 and 10. Within both approaches, it is evident that the patterns of the heatmaps are very similar and show large consistency with only minor differences, indicating that regulatory assignments are robust throughout the optimization processes. The observed differences in relative distribution of values mainly concern more dominant phosphorylation effects on very specific kinase to protein site relations within the gradient-based optimization of the Flux script. Figure 9 depicts results with certain α_j^i parameters inheriting values close to the relative maximum of +1, while the heatmap pattern of the Interior Point optimizer (Fig. 10) suggests a more balanced profile with broader distributions of weights. These differences are likely related to the distinct handling of boundaries and secondary conditions within the two optimizing approaches.

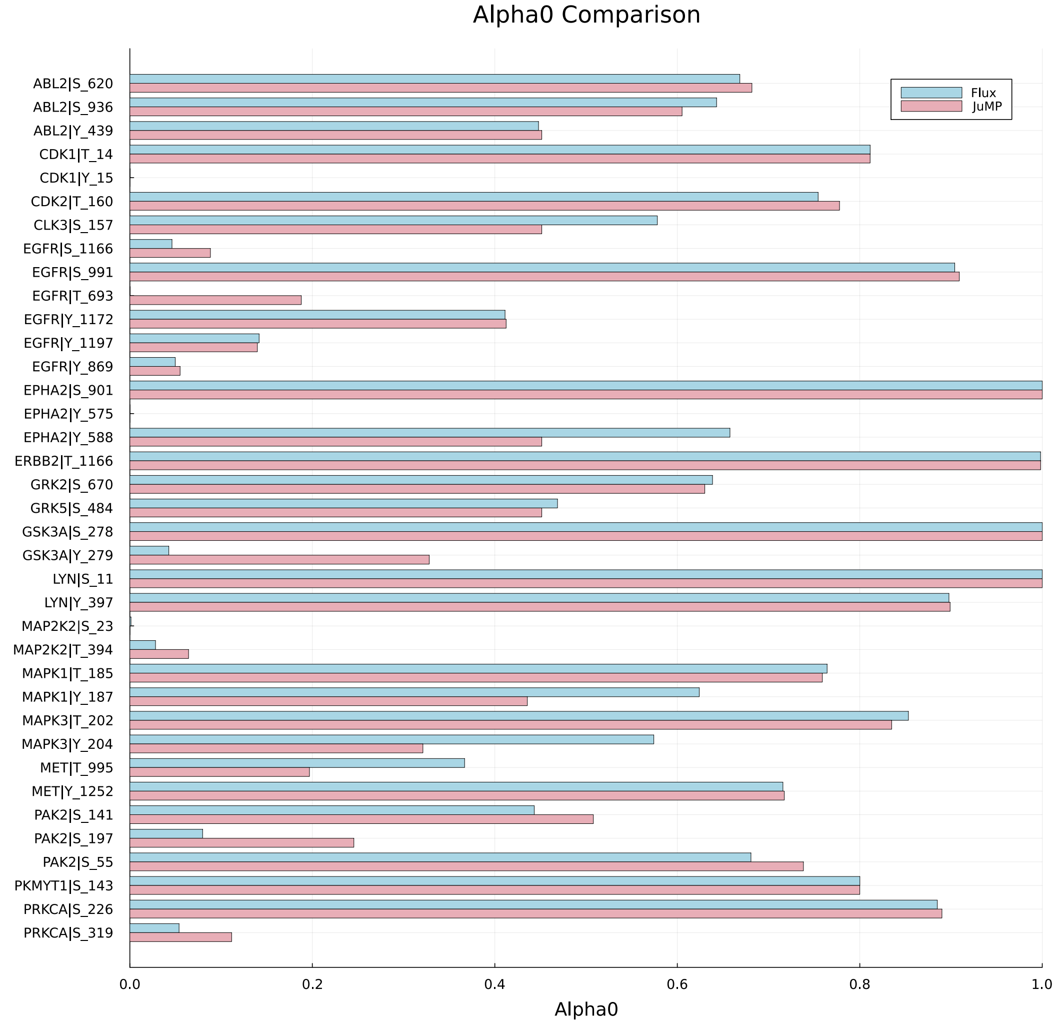

Figure 11: Comparison of results for estimation of α_i^0 parameter values between gradient-based optimization of Flux script (blue) and Interior Point optimization of JuMP script (red). Phosphorylation sites of protein network are plotted over numeric value of α_i^0 parameters representing the impact of unknown kinases on protein phosphorylation.

The values of the estimated α_i^0 parameters represent the effects of kinases currently not characterized in the network dependencies on the phosphorylation of respective protein sites. Estimation results of α_i^0 values are depicted in figure 11. It is evident that phosphorylation of protein sites is strongly influenced by unknown kinases rather than by kinases covered in the network due to higher average values of α_i^0 in comparison to α_j^i values of respective phosphorylation sites. For instance, the results indicate an

α_i^0 value of 1,0 for the phosphorylation sites S_901 of EPHA2 and T_1166 of ERBB2, both of which are exclusively phosphorylated by GRK2. This suggests that GRK2 exerts no primary influence on the phosphorylation of these specific sites, which is a finding that contradicts the scoring results provided by the KinomeXplorer platform. Despite this discrepancy, the results derived solely from the mathematical estimation of α values remain biologically plausible. This is because kinases covered in the current network state represent only portions of the entire pathway, which cannot be mapped completely yet due to limited data availability. Accordingly, the assumption that other, as yet unknown kinases are involved in the phosphorylation of the network proteins is justified.

The comparison of α_i^0 values shows that both optimizers estimate similar baseline contributions for many phosphorylation sites, while clear differences are observed for selected sites. It is evident that the gradient-based optimization produces results in which the differences between α_i^0 values of respective phosphorylation sites exhibit larger amplitudes, whereas the Interior Point optimizer estimates a more balanced parameter set for unknown kinase effects. This is consistent with the results of the estimation of α_j^i values (Fig. 9 and 10), as the Interior Point optimizer finds parameter sets that inherit more balanced profiles regarding the effects of known and unknown kinases in the network, while the gradient-based optimizer estimates results that inherit lower α_j^i values on average but with specific kinases peaking at higher α_j^i values than the Interior Point optimizer. Since optimizing results cannot be verified at this time, it is not yet possible to determine which optimizing approach produces results that reflect the actual biological conditions more closely.

3.1.2 Results of β Parameter Estimation

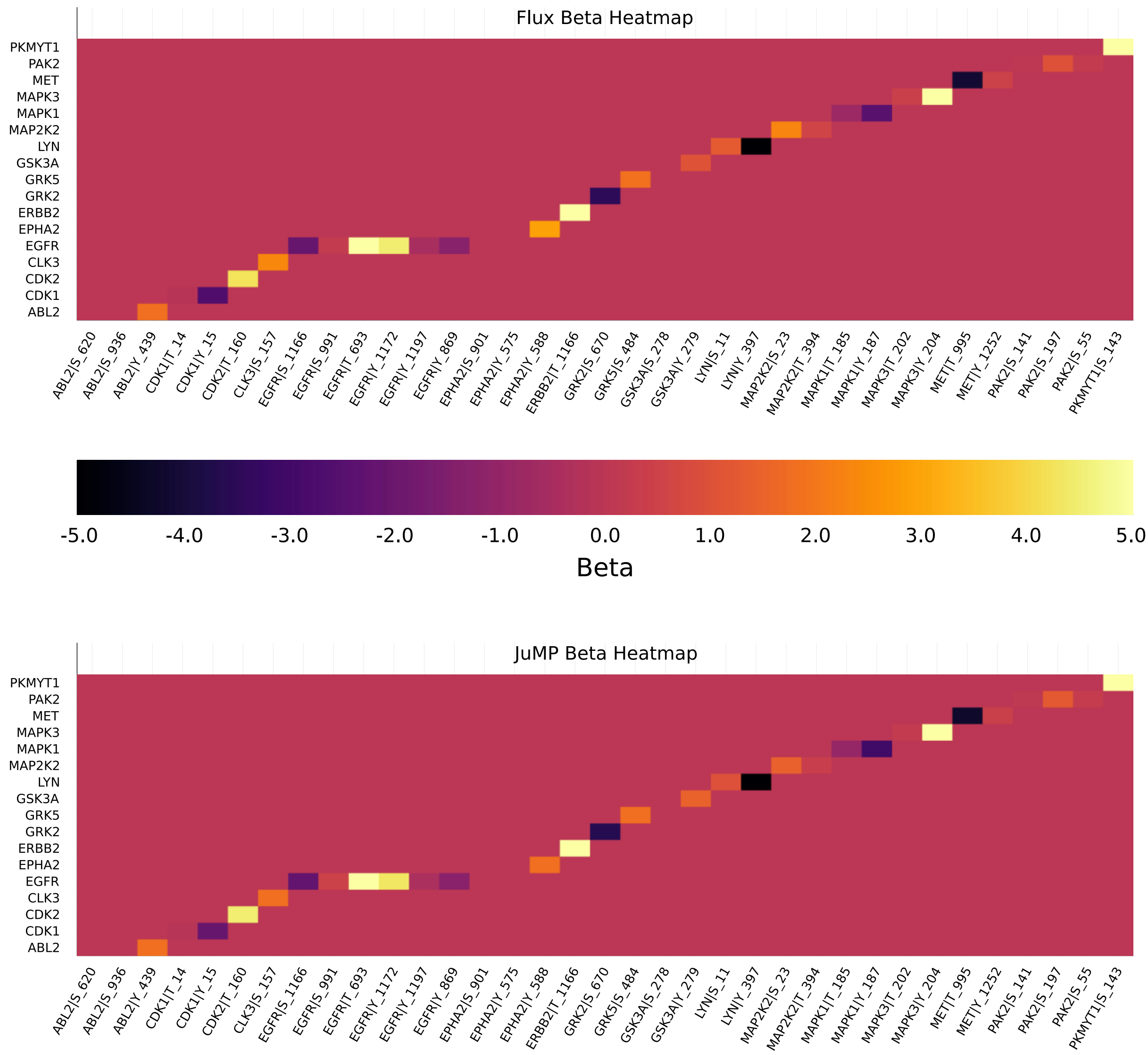

Figure 12: Heatmaps depicting estimated values of β_j^k calculated by gradient-based optimization (Flux, top) and Interior Point optimization (JuMP, bottom). Rows represent kinases and columns represent their associated upstream phosphorylation sites. Color scale indicates magnitude of estimated β_j^k values and therefore the effect of a phosphorylation site on the activity of a kinase.

The β_j^k parameters represent the effect of known phosphorylation sites on the activity of kinases integrated in the network. Since phosphorylation of these sites can have catalytic as well as inhibitory effects on the activity of the respective kinase, boundaries of β_j^k values were chosen from -5 to $+5$. This enables the optimizer to simulate both effects in the parameter estimation process.

Figure 12 depicts results of both optimizers estimation values for the β_j^k parameters. Similar to the α_j^i heatmaps, the apparent sparsity of the β_j^k heatmaps is largely imposed by the given network structure, since every kinase is only associated with known phosphorylation sites composed in the dataset. The patterns of the heatmaps from the two optimizing approaches are highly similar, indicating that both optimizers almost calculate the exact same set of parameter values. Only a subset of phosphorylation sites highly contributes to the corresponding kinase term by inheriting β_j^k values that have potentially strong catalytic effects on the kinase activity. However, majority of sites show weaker contributions with β_j^k values located in the approximate range of 0 to $+1$. It is also evident that specific phosphorylation sites inherit strong inhibitory effects on kinase activity, as depicted by the darker squares in the heatmaps of figure 12.

The close agreement between both heatmaps indicates that the estimation of β_j^k values is highly robust across optimization approaches. This suggests that the internal composition of the kinase-specific terms is more strongly constrained by the data and model structure than the distribution of α parameters (Section 3.1.1), which showed greater sensitivity to the choice of optimizer.

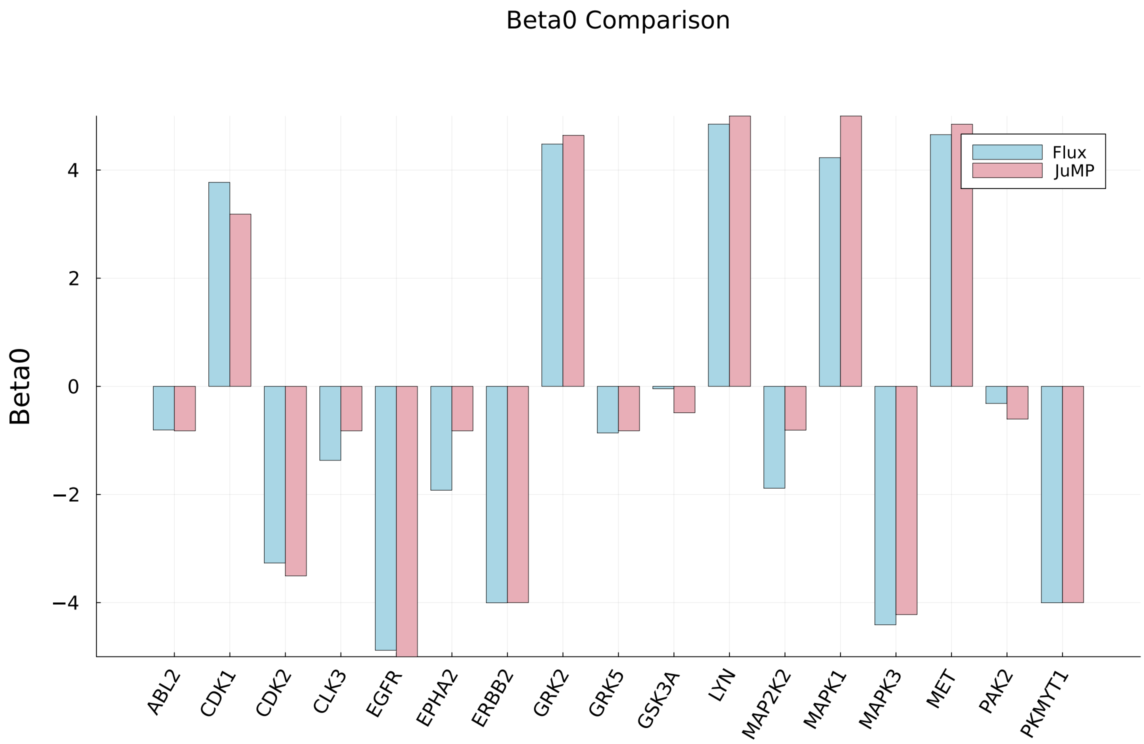

Figure 13: Comparison of results for estimation of β_j^0 parameter values between gradient-based optimization of Flux script (blue) and Interior Point optimization of JuMP script (red). Numeric value of β_j^0 parameters representing the effect of unknown phosphorylation sites on the activity of respective kinases is plotted over kinases integrated in the network.

The estimated β_j^0 parameter values represent the impact of phosphorylation sites currently not covered in the network dataset on the activity of respective kinases. Results of the estimation of β_j^0 parameter values are depicted in figure 13. It is evident that estimated values of β_j^0 are very similar between the two optimizing approaches, continuing the behavioral patterns of the heatmaps representing the estimation of β_j^k parameters. Baseline contributions depicted in figure 13 are similarly assigned with matching sign and comparable magnitude for both optimizers. This is a strong indication that the estimation of the kinase-specific contributions is also robust with respect to the chosen optimization method.

A substantial proportion of estimated β_j^0 parameters frequently attain both strongly positive and strongly negative values, indicating that the kinase-specific inner terms are not determined solely by the weighted phosphorylation site-dependent contributions, but also by a substantial constant baseline component. This corresponds to results of the β_j^k parameter estimation, in which many site-specific values remain comparatively moderate.

Since only a limited subset of kinase phosphorylation sites is represented in the dataset, the measured β_j^k-associated sites cannot be interpreted as a complete description of kinase activity. The corresponding term for β_j^0 therefore captures additional regulatory influences that are not resolved by the available phosphorylation site measurements. On the biological level, this may reflect that the measured phosphorylation sites currently employed in the network do not fully capture the effective activity state of a kinase, which is why both optimizers compensate this behavior by shifts part of the kinase-specific contribution into β_j^0. Although this is biologically plausible, β_j^0 parameters inheriting greatly higher and lower numeric values cannot be verified as representative for the impact on the activity of respective kinases, since the number of relevant phosphorylation sites is much higher than the number of sites currently covered in the network. Therefore, additional measurements of phosphorylated kinase abundances are required to achieve a more balanced and biologically interpretable separation between β_j^k- and β_j^0-associated contributions during parameter estimation to gain plausible parameter results for the impact of every phosphorylation site.

3.1.3 Performance Comparison between both Optimizers

Completing the estimation of α and β parameters allows for comparison between the two optimizing approaches regarding their performance and accuracy. As described in the methodological comparison of the two optimizers in section 2.1.3, main differences between the parameter estimation procedures are only of mathematical origin especially regarding the handling of constraints, secondary conditions and training algorithms. The effects of these differences on the optimization process and the final model quality can now be examined. Since both optimizers utilize the same model structure and loss function, it can be hypothesized that their final predictions should, in principle, be similar. As both optimizers inherit different parameter estimation approaches, differences in results should be evident primarily in the nature and speed of the convergence of the optimization.

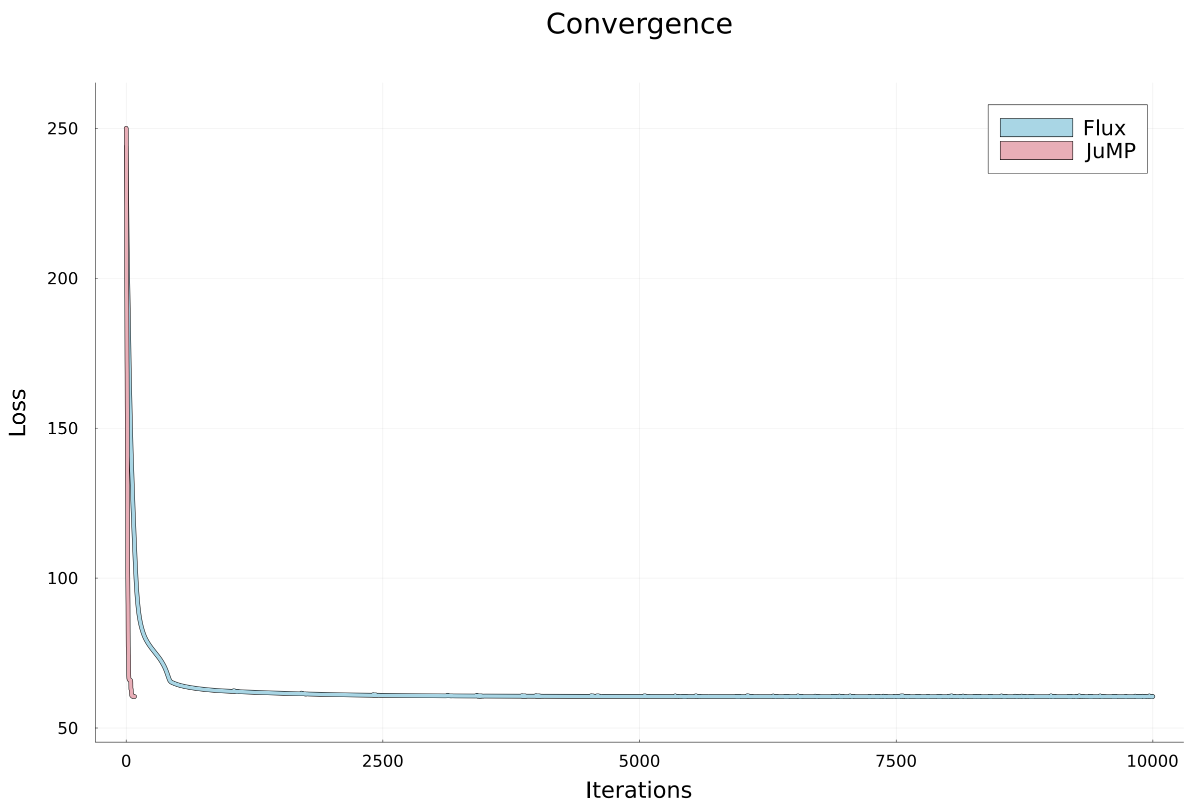

Figure 14: Comparison of iterative performance between the gradient-based optimizer (Flux, blue) and the Interior Point optimizer (JuMP, red). The numeric loss value is plotted over the number of iterations both optimizers need to reach similar losses of approximately 60.5 in the parameter estimation process.

As described in the methodological comparison of the two optimizing approaches (Section 2.1.3), the gradient-based optimizer only utilizes first-order derivative methods to calculate the gradient while the Interior Point optimizer uses first- and second-order derivatives to calculate gradient and curvature, and employ Newton steps with adaptive iteration step size. The effects of the different optimizing approaches are visualized in figure 14, where the iterative convergence of the two optimizers is depicted by plotting the loss value of the number of iterations. It is evident that the JuMP optimizer reaches a stable and low loss rate, while the Flux optimizer also significantly reduces the loss, but requires many more iterations to adjust the parameters and reach the same loss value as the JuMP optimizer. While approximately 10.000 iterations are needed in over 2 minutes for the Flux optimizer to reach a minimum loss value of 60.49, the JuMP optimizer reaches a minimum loss value of 60.48 in 75 iterations and about 4 seconds. In terms of performance, the Interior Point optimizing approach consumes less time and computing power to calculate a loss result lower than the result of the gradient-based approach. This confirms the methodological hypothesis, that that the main difference between the two methods lies in the numerical optimization strategy. At the same time, it is demonstrated that the Interior Point approach is significantly more efficient in the iteration space for this problem.

However, the currently employed network comprises 17 kinases and therefore only a part of the entire pathway. If the network is later extended across multiple pathways potentially containing hundreds of kinases, performance of the Interior Point optimizer might be less dominant over the performance of the gradient-based optimizer. The reason for this is the exponential rise of parameter number, as the amount of α and β parameters as well as secondary condition variables increases, and with it the size of the KKT system matrix that needs to be solved. Therefore, every iteration will become more costly in terms of calculation time and computing power. In contrast, single iteration steps of the gradient-based optimizer are calculated more easily with fewer costs, as they do not depend on solving second-order derivative systems.

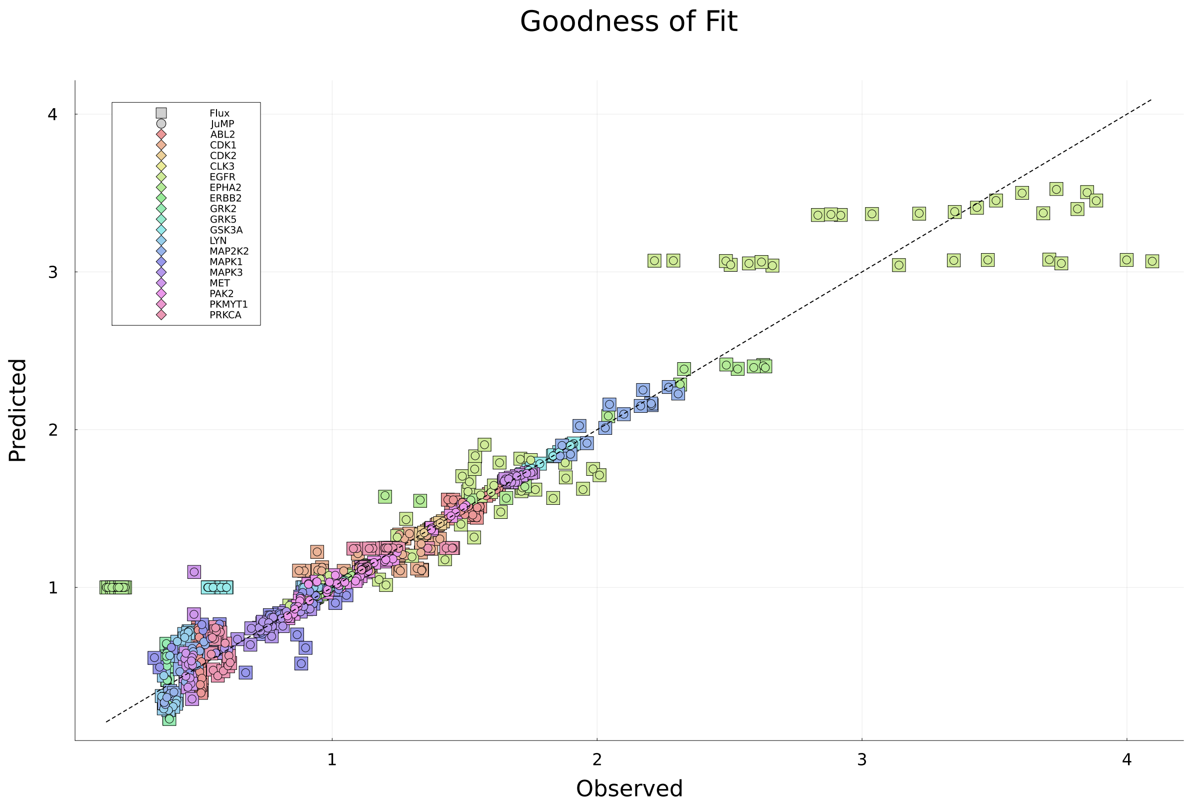

Figure 15: Comparison of goodness of fit between the gradient-based optimizer (Flux, squares) and the Interior Point optimizer (JuMP, dots). The predicted protein fold change value from both optimizing approaches is plotted over the observed protein fold change value.

When comparing both optimizing approaches in terms of goodness of fit, it is expected that their model predictions should inherit similarities, since they utilize the same model equations and MSE objective function. This hypothesis is confirmed by plotting the predicted fold change values of the model over the measured fold changes in the dataset (Fig. 15). The plot reveals that the predicted data points align closely around the linear graph representing the optimum fit, especially around lower fold change values. A few deviations can be observed at higher fold change values for the EGFR protein, which comprises the highest amount of measured phosphorylation sites currently employed in the network and therefore the most parameters required to be fitted. This leads to partially worse fit of specific predicted parameter values, as both optimizers make trade-offs since flawless modelling is more unattainable with higher amounts of parameters involved.

The plot in figure 15 confirms that the differences between the two optimizing approaches described in the methodological comparison (Section 2.1.3) do not lead to fundamentally different modelling results, but to distinct numerical paths within the same model architecture. As visualized in the plot, the data point fits of both optimizers are almost exactly congruent. This is supported by depiction of the error distribution between both optimizers in figure 16.

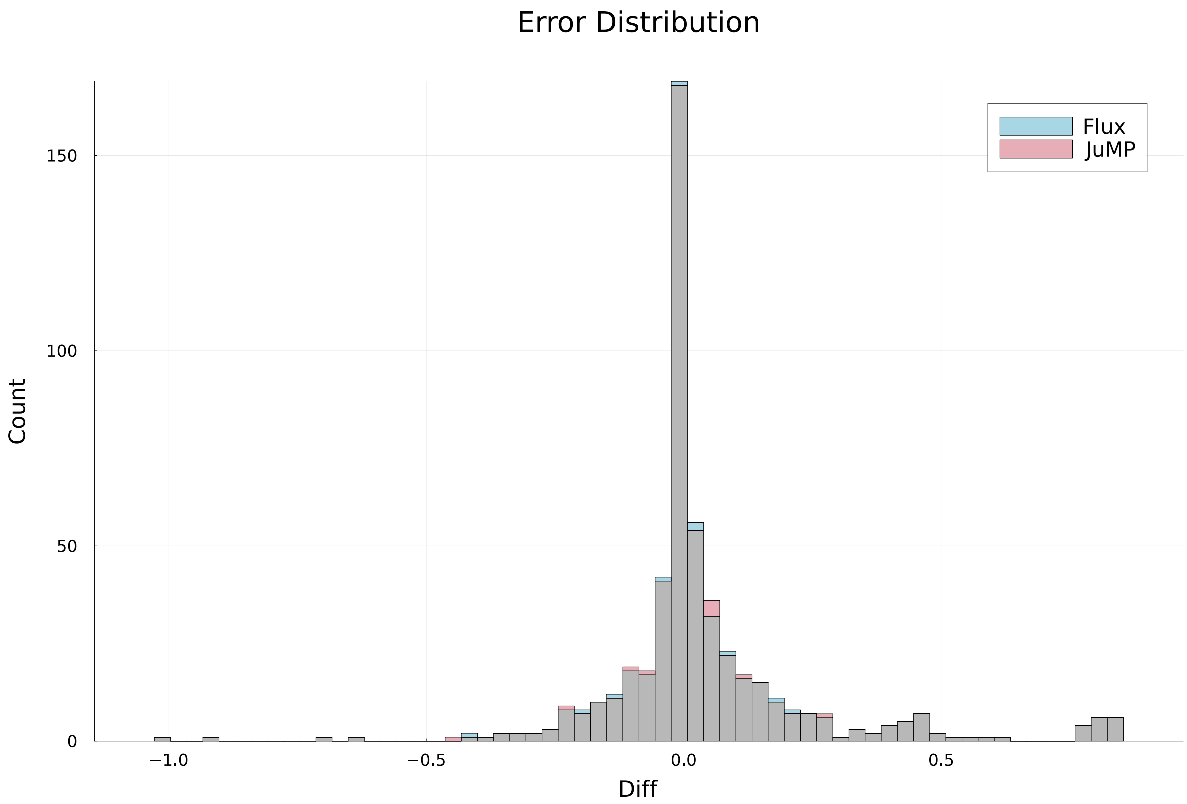

Figure 16: Comparison of error distribution between the gradient-based optimizer (Flux, blue) and the Interior Point optimizer (JuMP, red). The numeric count of parameters is plotted over the difference between predicted and measured data values.

Total counts of fold change data points are plotted over the absolute difference between the values predicted in the optimization process and the measured protein data. As visualized in figure 16, the overall parameter fit is close to the measured data for both optimizers, since almost all counts of parameters inherit only absolute differences mostly between values of -0.5 and $+0.5$. It is also evident that both optimizers share the almost exact same error distribution profile, confirming congruence of the data points fits depicted in figure 15.

The plotting of error distribution and goodness of fit demonstrates that both optimizing approaches calculate closely aligned model profiles despite their different handling of parameter constraints and secondary conditions as well as distinct optimizing algorithms. However, this exposes an issue concerning the parameter set values estimated by the two optimizers. As covered in section 3.1.1 and 3.1.2, the estimated parameters values between the optimizers show different profiles and baseline contributions, especially regarding the α_j^i and α_i^0 values. With both optimizers calculating an almost identical fit of fold change values despite utilizing different sets of parameters, it is assumed that different variants of parameter sets exist which would lead to a high goodness of fit. Therefore, finding the true biological parameters for the activities of kinases and effects of phosphorylation sites in the network solely by utilizing mathematical optimization can be considered hardly possible. It is reasonable to gather more data about the network dependencies and introduce more secondary conditions into the optimization process, for example by prophylactically analyzing if phosphorylation of respective protein sites has catalytic or inhibitory effects on kinase activities. In general, integrating biologically verified data and information into the optimization process would support the parameter estimation to receive accurate values for the network dependencies.

3.2 Results of Network Modeling Approach

Following the estimation of the dependency parameters α and β, it is evaluated whether the resulting network structure can accurately reproduce the temporal dynamics of the signaling system within the ODE framework. To this end, the kinetic parameters A, B, C, D, S and Z were optimized based on the experimental fold change data and subsequently integrated into the global simulation of the complete ODE network. The following section presents the results of the kinetic parameter estimation and examines the extent to which the resulting parameter set enables the model inheriting the coupled dependencies to replicate the observed dynamic fold change profiles at the system level.

3.2.1 Results of Kinetic Parameter Estimation

The estimation of the kinetic parameter values was conducted by performing the protein-wise optimization to independently fit the predicted fold change values of the model to the measured values of the dataset. Initially, a global estimation of all parameters was attempted using the Interior Point optimizer described in section 2.1.2. This approach was subsequently discontinued, as the optimizer was required to derive and estimate approximately 2,600 parameters simultaneously. Due to the underlying algorithm of the Interior Point method, which involves the construction of Hessian matrices, the computational overhead in terms of time and processing power became prohibitive, as the matrix sizes rise exponentially with the increasing number of parameters. Consequently, the strategy was shifted to the present protein-wise parameter estimation, utilizing the Adam optimizing algorithm presented in section 2.2.4 in combination with the ODE solver. The optimization procedure was executed for a total of 300,000 iterations, requiring approximately 224 hours of computation time.

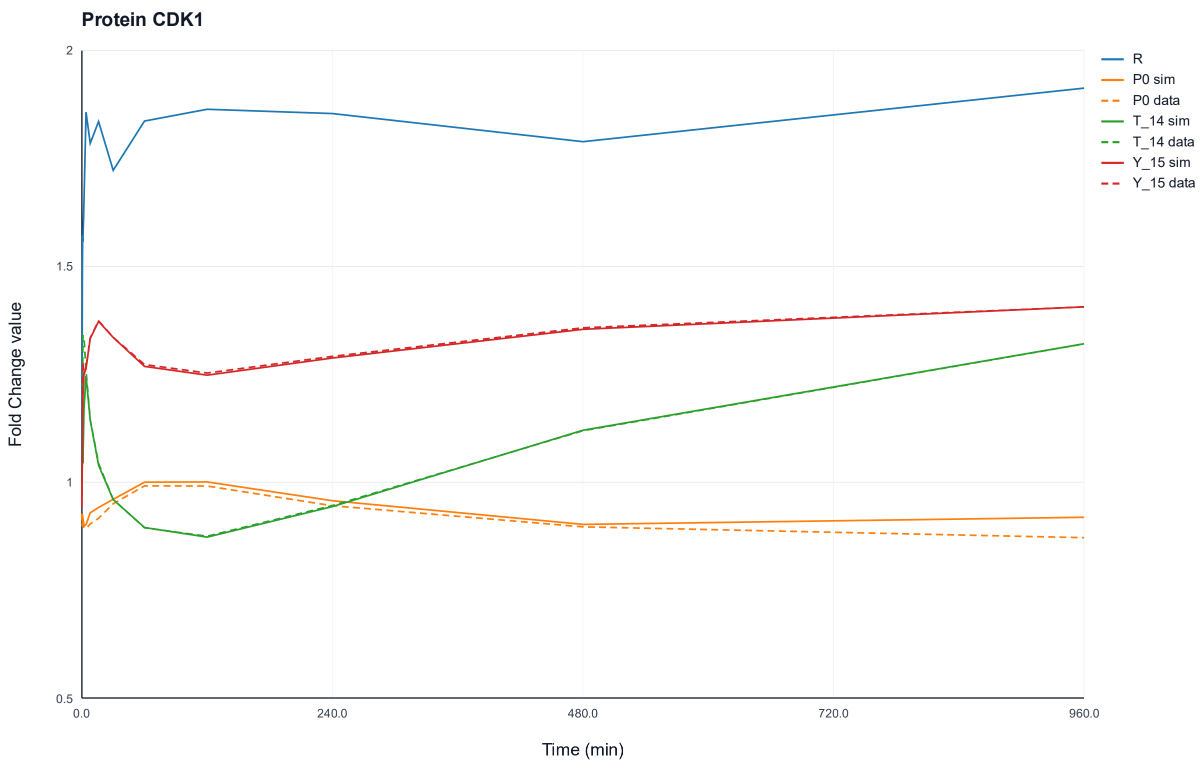

Figure 17: Protein-wise simulated and experimentally observed fold change profiles of protein CDK1 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites T_14 and Y_15. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

The transition to a protein-wise optimization strategy proved to be effective, providing a robust framework for the independent estimation of parameters. The approach was successfully validated, as the majority of simulated fold change profiles exhibited a trajectory that aligned sufficiently with the values from the experimental dataset. This is exemplarily illustrated in figure 17 for the protein CDK1, where the simulated trajectories for both the unphosphorylated protein P_0 and its phosphorylated state at sites T_14 and Y_15 closely track the experimental data points. The results for the remaining proteins in the network can be found in the appendix (Fig. 27 – 41) along with the plots of the estimated values of the kinetic parameters (Fig. 42 – 58) and the values of combined rate terms (Fig. 59 – 75).

An evaluation of all protein plots reveals that the simulated mRNA fold change values fluctuate significantly and frequently exceed a value of 1. This is because the current simulation is exclusively focused on fitting the phosphoprotein data, while mRNA values have not yet been incorporated into the objective function. Consequently, the mRNA fold change values are determined solely by the adjustment of the transcription rate A and the mRNA degradation rate B. The elevated fold change values however remain biologically plausible, as sufficient amounts of mRNA are required to sustain the translation of proteins in both their unphosphorylated and phosphorylated states.

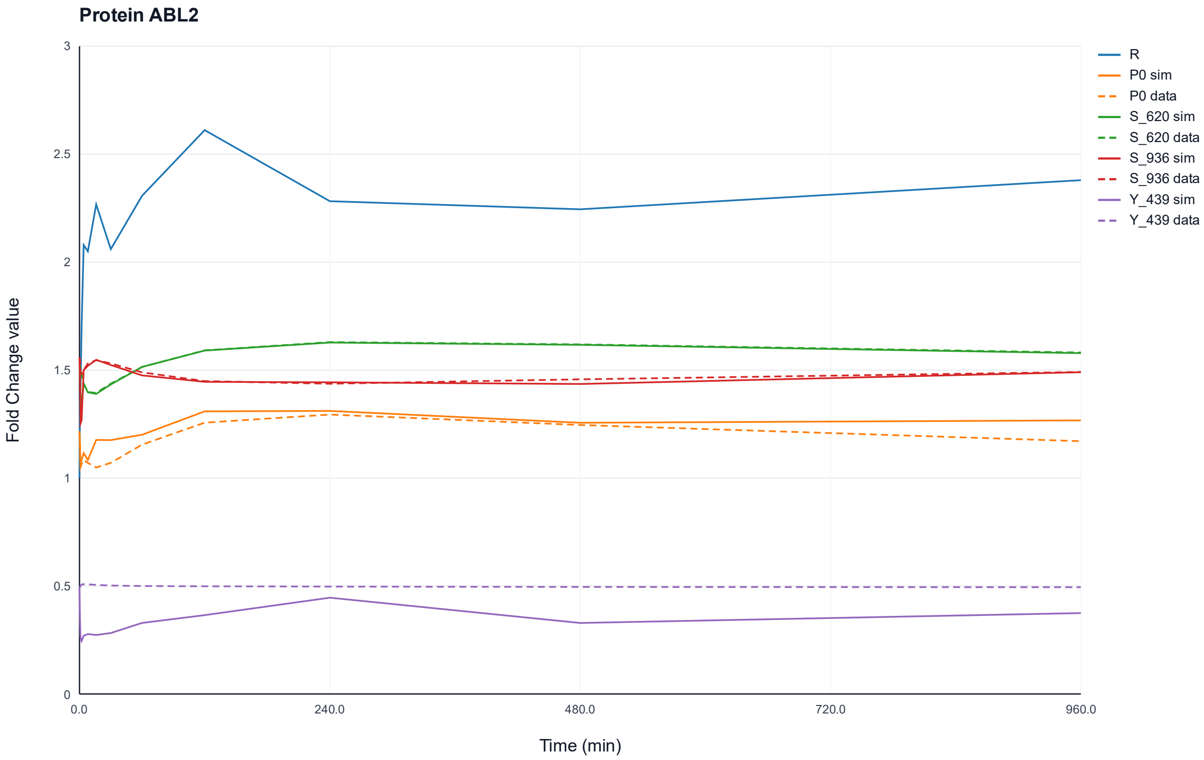

Figure 18: Protein-wise simulated and experimentally observed fold change profiles of protein ABL2 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_620, S_936, and Y_439. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

Further evaluations also reveal distinct discrepancies in the behavior of specific phosphoprotein plots which lead to inaccurate fits and offsets between the simulated fold change values and the experimental data. For example, the fold change plot of the phosphorylated state of the ABL2 protein at site Y_439 illustrates simulated values that fall significantly below the observed experimental data (Fig. 18).

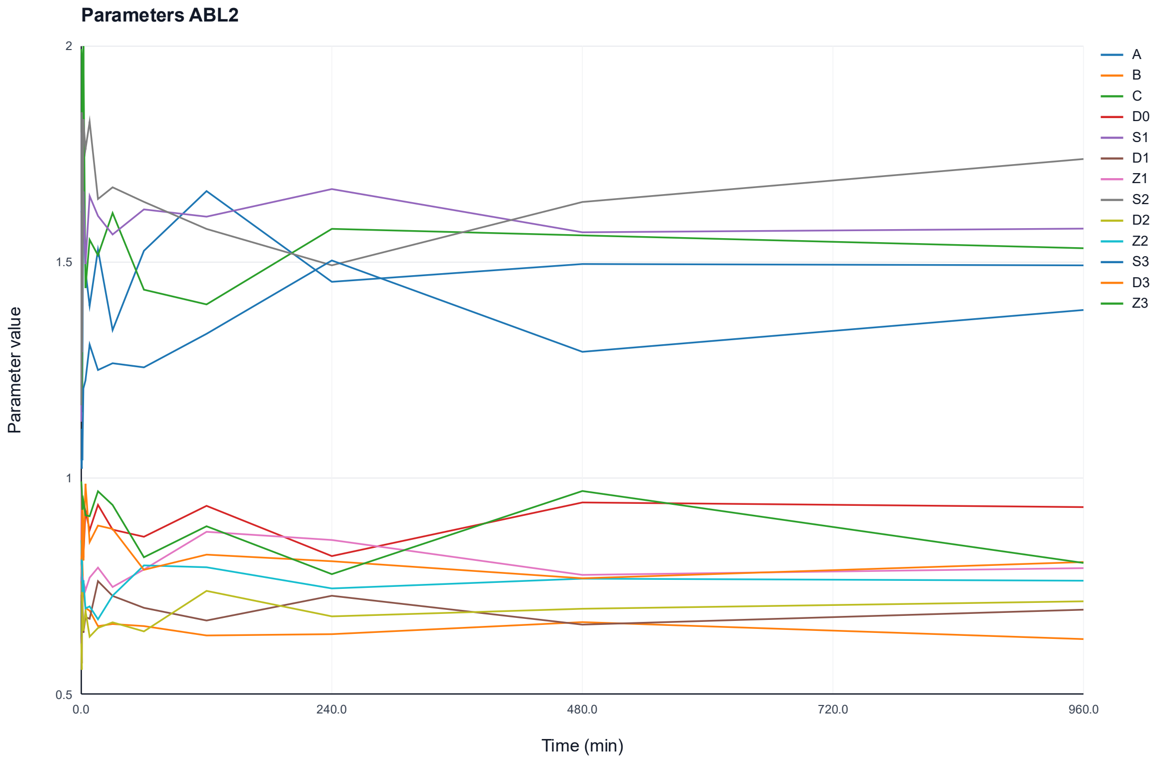

Figure 19: Temporal dynamics of the protein-wise estimated kinetic parameters of protein ABL2. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

In this context, the offset between the simulated and observed fold change values is most likely attributable to the translation parameter of the ABL2 protein reaching its upper constraint of 2 (Fig. 19). This limitation leads to insufficient supply of the protein, which results in partly decreased trajectories for the phosphorylated protein states. Adjusting the constraints to a higher value could mitigate this issue in future parameter estimation attempts, as it would provide the optimizer with a broader search space to refine the parameters for a more accurate fit.

Further investigations of the optimized kinetic parameter dynamics in figure 19 and figures 42 – 58 (Appendix) unveil, that the estimated parameters exhibit only minor oscillatory behavior. This successful suppression of numerical artifacts is attributable to the implementation of the Prior Loss term in the optimization process, which acts against strong fluctuations by driving the parameter values toward the predefined reference value of 1. Consequently, the introduction of Prior Loss can be considered successful, as it actively contributes to maintaining biologically plausible parameter values.

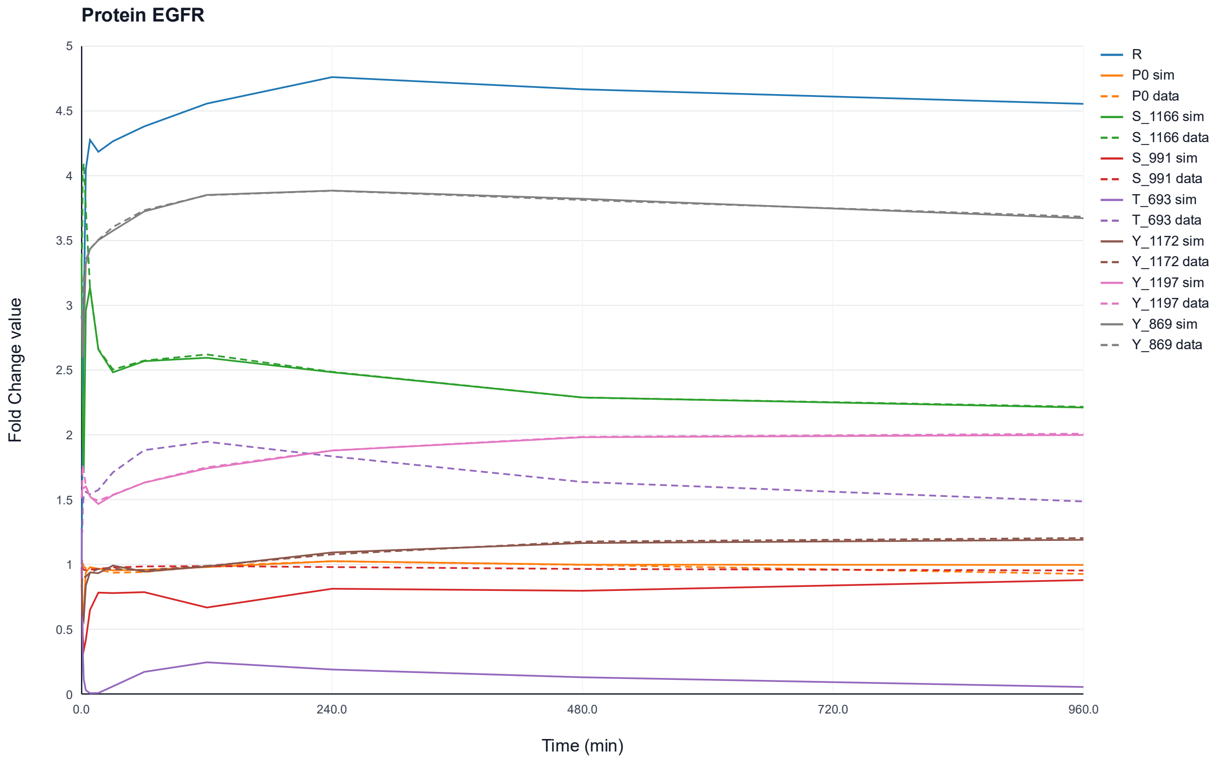

Figure 20: Protein-wise simulated and experimentally observed fold change profiles of protein EGFR over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_1166, S_991, T_693, Y_1172, Y_1197, and Y_869. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

Furthermore, additional discrepancies between simulated and measured data can be observed. In these cases, the trajectories of the fold change profiles exhibit a similar dynamic shape, yet a significant offset remains between the absolute simulated values and the experimental data. An example is depicted in figure 20, which illustrates the fold change courses of the EGFR protein. The predicted profile for the phosphorylation state at the T_693 site exhibits significantly lower values than the experimental data. However, the underlying dynamics of both profiles follow a very similar trajectory.

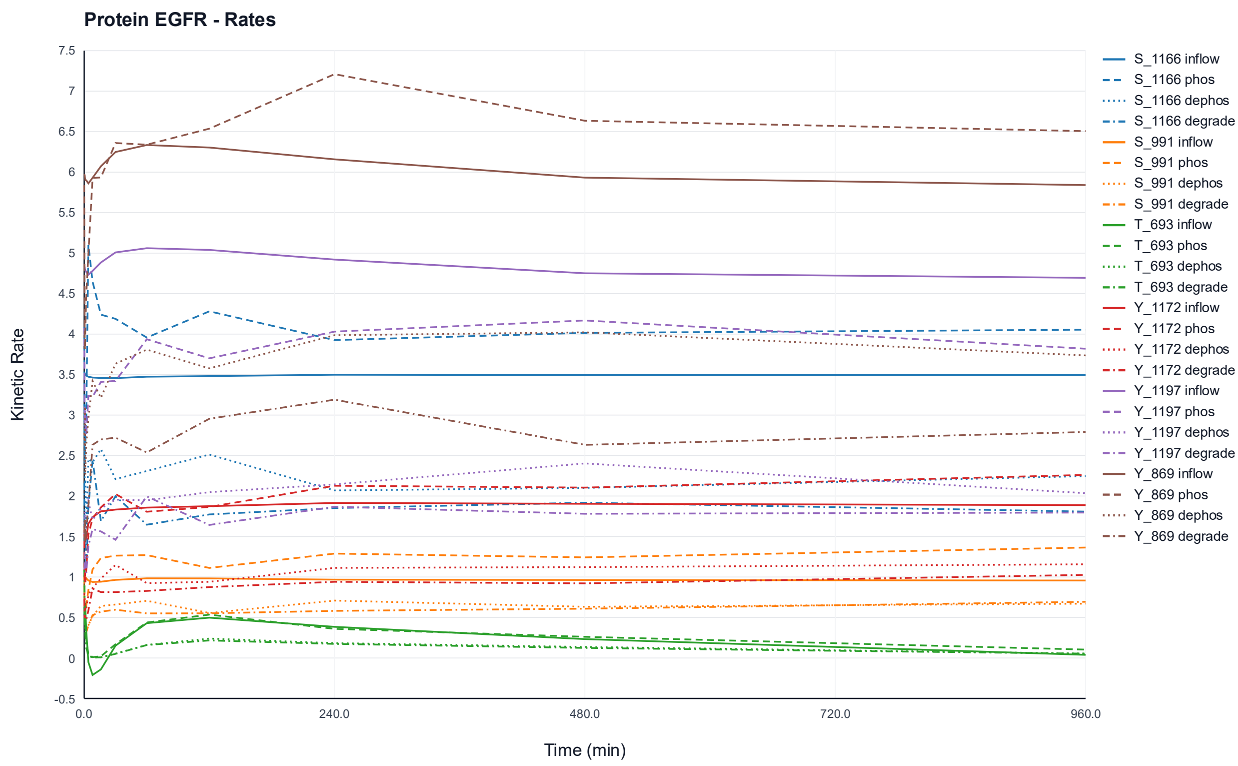

Figure 21: Time-resolved kinetic rate profiles of protein-wise estimation for the EGFR protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states of EGFR at sites S_1166, S_991, T_693, Y_1172, Y_1197, and Y_869.

The reason both trajectories follow these specific paths is that the rate of the inflow variable, which represents the values of the activity term inheriting the dependency parameters, gains significantly decreasing values in the beginning of the simulation (Fig. 21, solid green trajectory of T_693 inflow). As described in section 2.2.3, this is caused by negatively estimated values of the β parameter affecting the entire inflow term to also inherit negative values. The negative inflow causes a significant decrease of fold change values for the T_693 phosphoprotein, which cannot be counteracted by other upstream terms. However, the optimizer still manages to estimate kinetic parameter values which closely replicate the dynamics of the experimental data. By further adjusting constraints for the kinetic parameters and improving the estimation of dependency parameters, the issue concerning distinct offsets between predicted and observed fold change profiles can be countered.

Upon analyzing the gradients of the inflow terms in figures 21 and figures 59 – 75 (Appendix), it is evident that the implementation of the gating function to prevent negative fold change value calculations which could potentially compromise the solving process of the ODE model proved effective. As depicted in figure 21, the decreasing inflow term of the phosphorylation state at the T_693 site below a value of 0 leads to the fold change value of the respective phosphoprotein to remain stable at a small positive value (Fig. 20). This proves the implementation of the gating function successful.

3.2.2 Results of ODE Network Simulation

The estimated set of kinetic parameters is utilized in the final ODE simulation, in which the network phosphorylation dependencies are activated by establishing the $P_j^k(t)$ variable as a representation of kinase abundances that are calculated in real-time within the simulation.

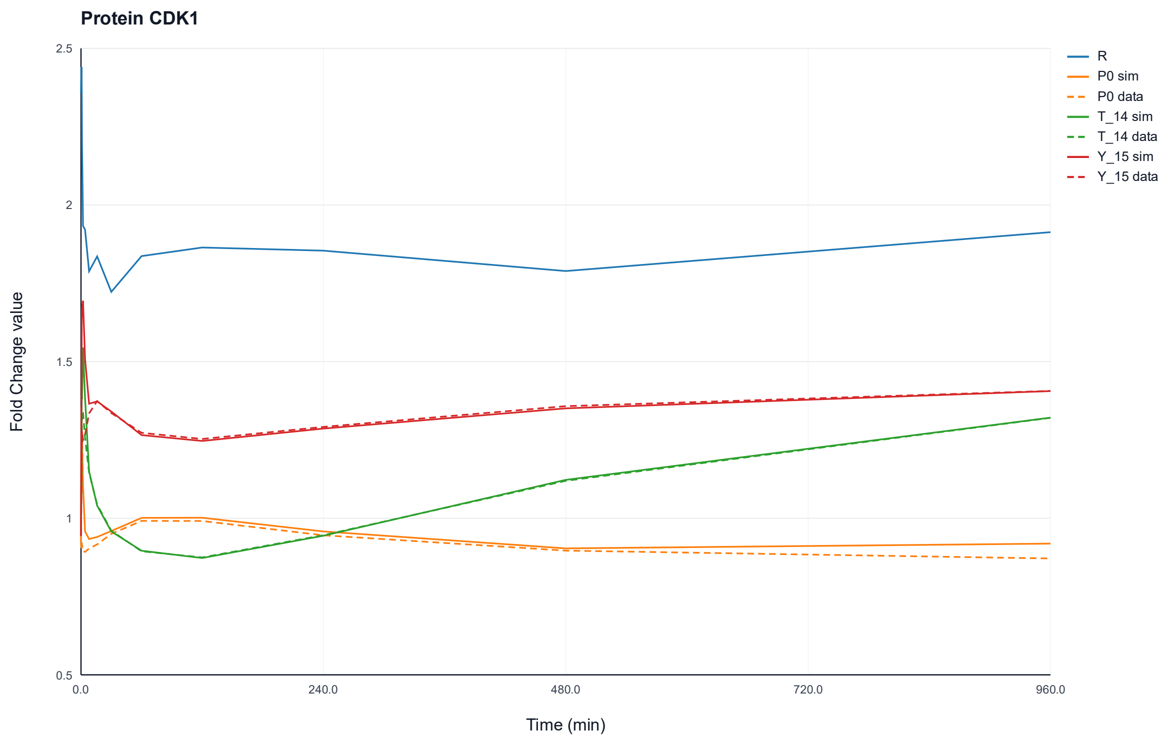

Figure 22: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein CDK1 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites T_14 and Y_15. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

The results for the fold change trajectories of the dependent ODE network simulation demonstrate that the simulated profiles closely align with the experimental data, despite the implemented phosphorylation dependencies between the kinases. This is attributable to the successful optimization process of the kinetic parameter values. As depicted in figure 22 as well as figures 76 – 92 (Appendix), the profile dynamics are almost identical to the dynamics observed in the parameter optimization process (Fig. 17), despite a pronounced numeric impulse occurring in the initial phase of the simulation. This impulse can presumably be caused by different reasons, notably because of algebraic dependencies as the α and β dependency parameters account for strong impacts on the phosphorylation processes, the absence of a steady-state which is answered by a strong reconciliation reaction at $t = 0$, or an inherent stiffness of the ODEs as kinetic processes often operate across different time scales. However, it is also evident that the initial impulses may be caused because of great fluctuations in the mRNA fold change profiles. As described in section 3.2.1, fitting the mRNA abundances to experimental data is not yet implemented in the network simulation process. Integrating this into the ODE optimization of kinetic parameters once sufficient data is provided may improve the stabilization of initial profile trajectories. Given the overall satisfactory fits of the fold change profiles across the entire network, both the optimization of the kinetic parameters and their integration into the dependent ODE simulation can be considered successful.

3.2.3 Metabolic Control Analysis

In the process of the ODE network simulation, a local elasticity analysis is performed to further investigate the roles of the kinases within the EGFR/MAPK network. The foundation of the coefficient calculations are the simulated dependent trajectories of the kinases. The impact of the respective source site $P_j^k(t)$ on the regulatory inflow and the resulting phosphorylation flux is analyzed by calculating local elasticities for every combination of source and target phosphorylation site. The inflow elasticities are determined from the linear coupling of α and β parameters and represent the sensitivity of target activity term values to changes at the source phosphorylation site. Using symmetric difference quotients, the local derivative of the flux is subsequently estimated at the source site and normalized to a dimensionless phosphorylation elasticity.

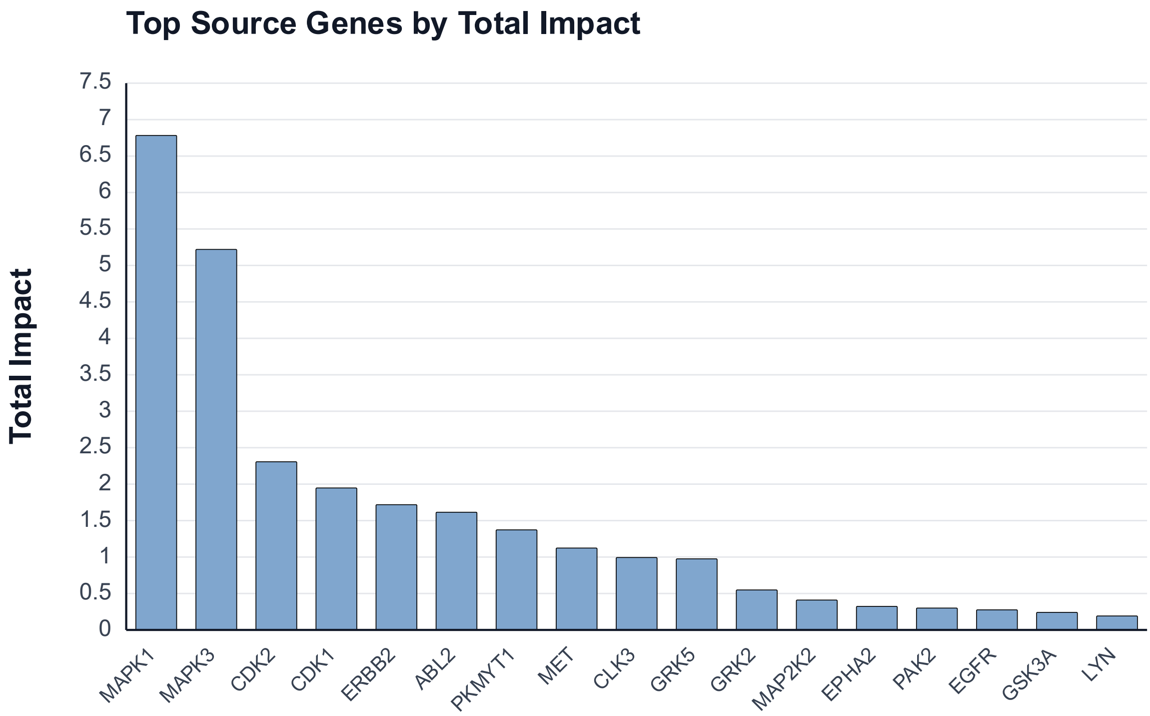

Figure 23: Ranking of total impact of upstream source genes on target phosphorylation sites. The influence of source genes is calculated by summing up the absolute values of time-dependent elasticities.

The plot depicted in figure 23 illustrates the aggregated ranking of the upstream source genes from the metabolic control analysis, showing which gene in the model inherits the strongest controlling impact on the fluxes and enzyme activities of all respective phosphorylation sites. As evident from the plot, MAPK1 and MAPK3 exhibit the greatest cumulative influence on downstream target sites, followed by CDK2 and CDK1. The ranking states that the ODE model establishes a hierarchic impact structure, in which a few kinases dominate the global signaling flux while others only contribute moderately to it.

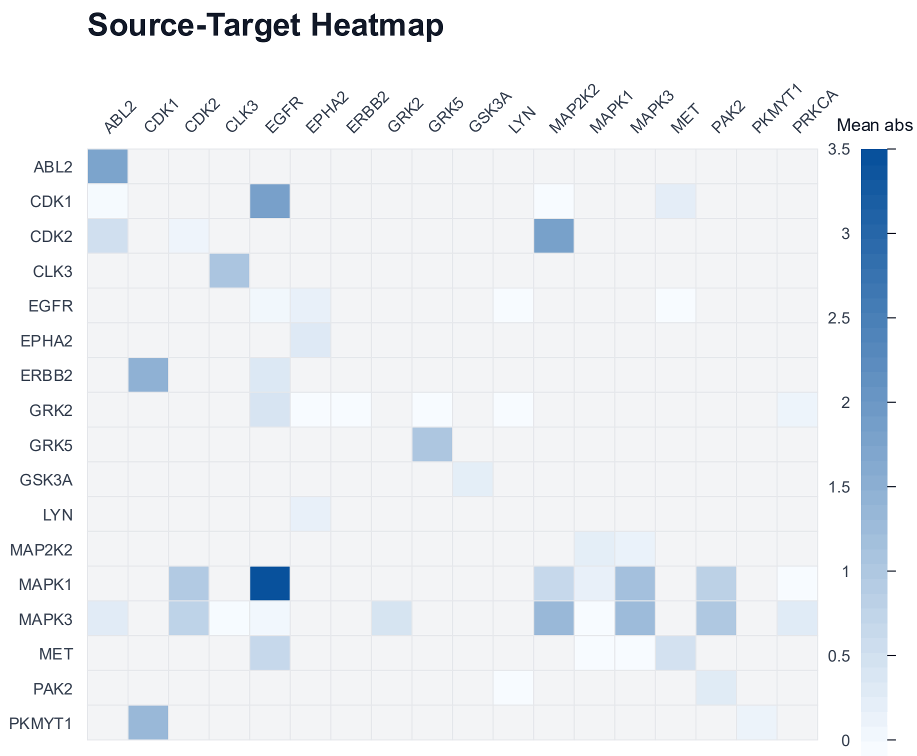

Figure 24: Heatmap of phosphorylation dependency influences. Rows represent source kinases while columns represent targets that are phosphorylated with respective effects.

The dominant role of MAPK1 and MAPK3 is attributable to them phosphorylating the highest number of target sites within the network, as illustrated in the heatmap in figure 24. Moreover, it is evident that their absolute impact on respective targets is higher than for the majority of other kinases in the network. Although their governing influence on the signaling flux is biologically plausible, the ranking does not reveal whether they exhibit catalytic or inhibitory effects on the signaling network, as it is based solely on the sum of absolute elasticity values.

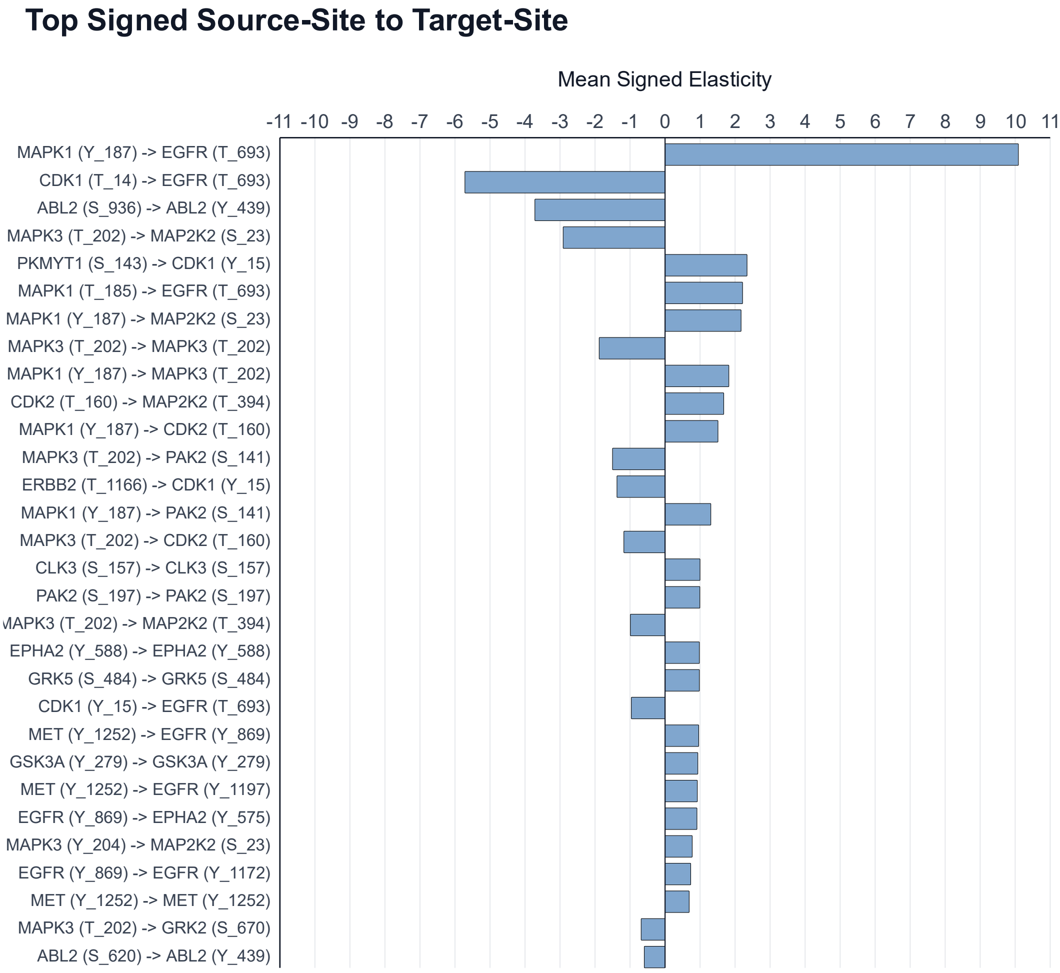

Figure 25: Ranking of mean signed elasticity of upstream phosphorylation sites of source genes on target phosphorylation sites. The elasticities are calculated by summing up the numeric values of time-dependent elasticities and calculating the mean signed value.

For this reason, the mean signed elasticity is plotted (Fig. 25) which illustrates the positive or negative impact of a source phosphorylation site of a kinase on a respective target site. Positive values increase the phosphorylation flux or the local activation of the target site in the model, while negative values have a damping effect on the target site. In contrast to the general ranking depicted in figure 23, the concrete relations between source sites and target sites are investigated.

Central observations include the dominating catalytic effect of the Y_187 source site of MAPK1 on the T_693 target site of EGFR, and the strong inhibitory effect of the T_14 source site of CDK1 on the same target site. Moreover, it is evident that the MAPK kinases as well as the EGFR receptor occupy especially sensitive positions within the dynamic signaling structure, as their activity exhibits the highest impact on the phosphorylation fluxes of the network. As described in section 1.4, this is biologically plausible as these kinases play a crucial role in the signaling cascade of their pathway, especially in cancer cells. The signed elasticity analysis in figure 25 however reveals that the implementation of secondary conditions for the optimizing process of dependency parameters described in section 1.6 supports relativity for the estimated parameters but also causes the issue that the dependency effects must inevitably balance out. This leads not only to particularly dominant catalytic effects on the phosphorylation flux but also strong inhibitory artifacts which cannot be justified on a biological level.

3.2.4 Results of Knockdown Procedures

In order to simulate signaling perturbations in the network, a protein knockdown function was implemented into the ODE model. The function allows for the selection of specific kinases to undergo a targeted knockdown, with the degree of inhibition being manually adjustable. More precisely, the upstream terms controlling the transcription and translation processes as well as the activity state can be modulated to various extents, ranging from partial reduction to a complete protein knockout.

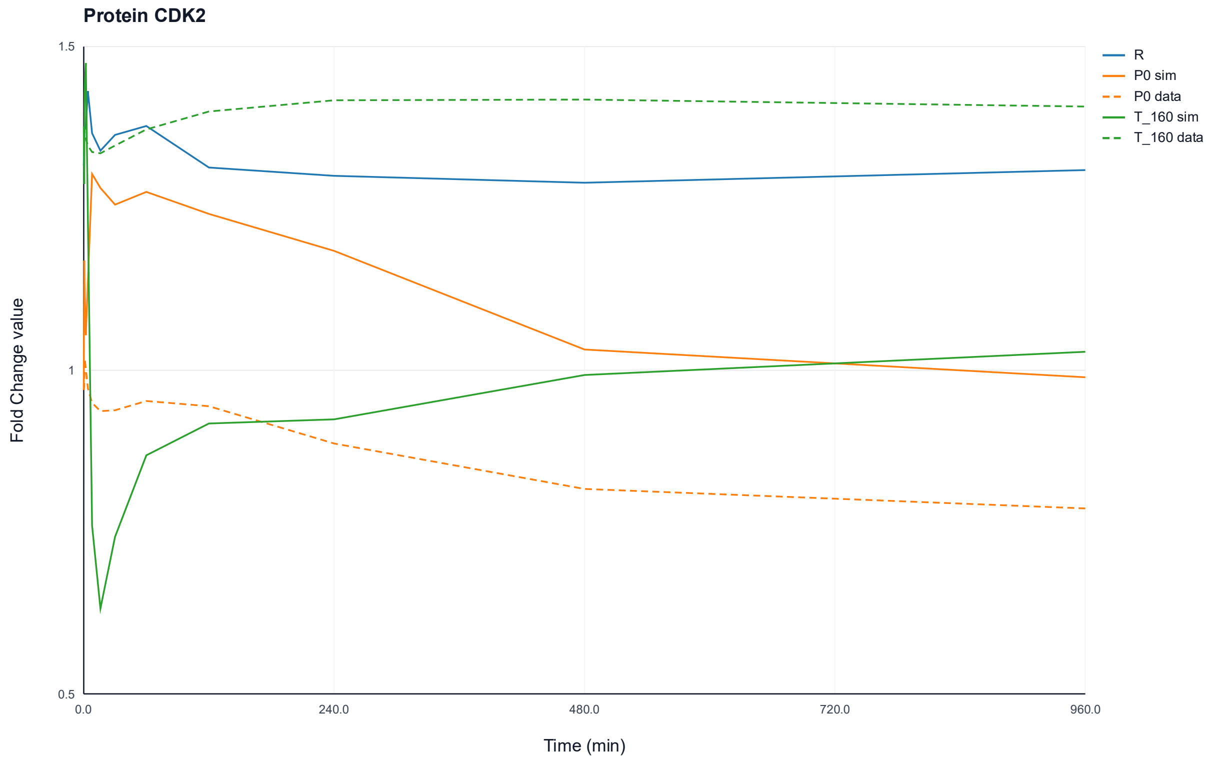

Figure 26: Comparison of fold change profiles of the CDK2 protein under appliance of MAPK3 knockout. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site T_160. Solid lines represent knockout simulations, whereas dashed lines indicate the measured data.

The knockdown function was utilized to perform full knockout procedures with an inhibition magnitude of 100 % applied for each kinase employed in the current state of the ODE network. As an illustrative example, figure 26 displays the fold change profiles of CDK2 following a knockout of the MAPK3 kinase. The successful implementation of the knockdown function is evident, as fold change profiles of CDK2 in phosphorylated state are significantly reduced while corresponding profiles of the protein in unphosphorylated state rise.

The outcomes of the full knockout procedures are summarized in the knockout matrix presented in figures 93 – 95 (Appendix). While the technical implementation of the knockdown function was successful, the resulting profiles for specific key nodes do not align with the strictly linear interpretation of the signaling pathway. For example, a knockout of EGFR is expected to cause a complete cessation of the downstream MAPK signaling. However, the knockout matrix (Appendix, Fig. 93 – 95) shows that the MAPK phosphorylation remains intact. Rather than interpreting these results as a model failure, the observation is consistent with the principle of functional redundancy and cellular context within signaling networks as described in section 1.1. In complex signaling networks, kinases are embedded in dynamic structures that allow downstream signals to be maintained through alternative receptors or crosstalk mechanisms [8]. This suggests that the network inherits a strong robustness, where the loss of a single node like EGFR is effectively buffered by the interconnected nature of the kinase network.

4. Conclusion & Outlook

Overall, the estimation of dependency parameters by developing two different optimizing approaches as well as establishing the dynamic ODE model for simulating the signaling network can be considered successful. Thereby, the research objective of the present study was achieved.
With regard to the dependency parameter optimization, the methodology of both developed optimizing approaches could be validated, as their estimated results show great similarities. The Interior Point optimizer demonstrated superior performances during the parameter estimation process for the currently employed network state comprising a total of 17 kinases. However, this performance comparison should be revisited in future expansions of the signaling network, as gradient-based optimizers often achieve a more favorable trade-off between optimization accuracy and computational cost as the number of parameters increases.

Based on the estimation of the α and β parameters, it can be concluded that a purely mathematical optimization approach, lacking the integration of additional biophysical constraints, is insufficient for deriving plausible dependencies between kinases and phosphorylation sites. This limitation arises, in part, from the distinct ways in which the two optimization algorithms handle input data and weighting, leading to a landscape where different parameter sets yield nearly identical loss values. Such ambiguous solutions are biologically implausible, as the dependencies should in theory remain consistent. Furthermore, the current implementation of constraints and secondary conditions results in unrealistic catalytic or inhibitory kinase properties, which is attributable to the forced balancing of α and β values. Consequently, integrating further biophysical conditions, such as estimating enzymatic potency through the inclusion of referential counts, is essential in the future to accurately replicate the interactions between kinases and phosphorylation sites of real cancer cells within the simulated network.

The construction of the ODE-based signaling network was successfully realized through the integration of dependency parameters, thereby effectively capturing the interactions between kinases and their respective phosphorylation sites. The protein-wise optimization strategy yielded stable parameter sets that accurately reproduce the experimental fold change dynamics within the coupled system. Observed numerical phenomena, such as the initial spikes in the dependent model, have been identified as artifacts arising from system initialization or algebraic coupling. To counteract these effects, the integration of genomic expression levels is proposed by integrating the measured mRNA fold change data parallel to the phosphoprotein data into the optimization process of the kinetic parameters.

As described in section 3.2.4, the network model demonstrated high sensitivity to *in silico* perturbations. However, the resulting knockdown and knockout profiles require future experimental validation by conducting targeted perturbation assays in live TNBC cells.

Looking forward, the developed Julia code is designed to facilitate an expansion of the currently employed network, allowing for the integration of additional kinases and participating proteins of the EGFR/MAPK signaling network. To enhance the biological plausibility and predictive power of the model, further phosphoproteomic experiments could help refining the signaling dependencies and validating the results of the simulations. Additionally, provided that larger datasets become available, the implementation of Physics-Informed Neural Networks will be explored as an alternative to the current machine-learning-based optimization. This approach offers the advantage of implementing biophysical constraints and conditions directly into the training process, potentially yielding more accurate kinetic parameters [48].

Ultimately, upon successful validation, the simulated signaling fluxes will be coupled with phenotypic data from the performed scratch assays described in section 1.3. This integration aims to link intracellular signaling dynamics with observed cell migration rates, thereby establishing a robust predictive model for cell migration and metastasis in triple-negative breast cancer.

Appendix

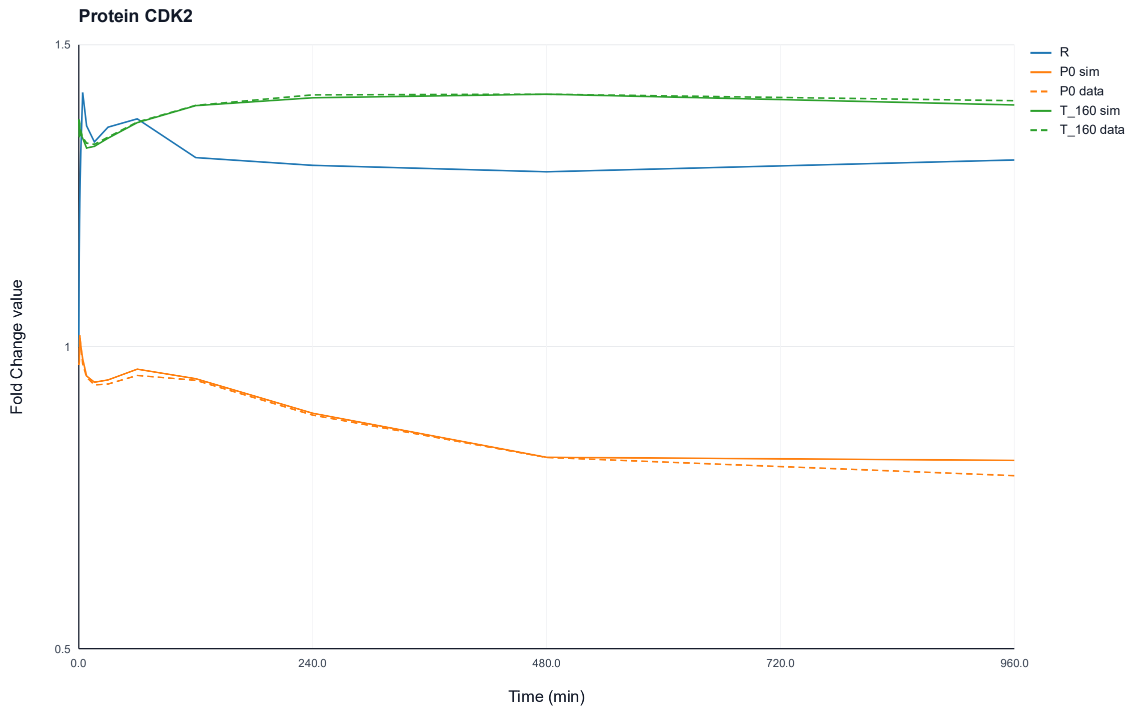

Figure 27: Protein-wise simulated and experimentally observed fold change profiles of protein CDK2 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site T_160. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

Figure 28: Protein-wise simulated and experimentally observed fold change profiles of protein CLK3 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site S_157. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

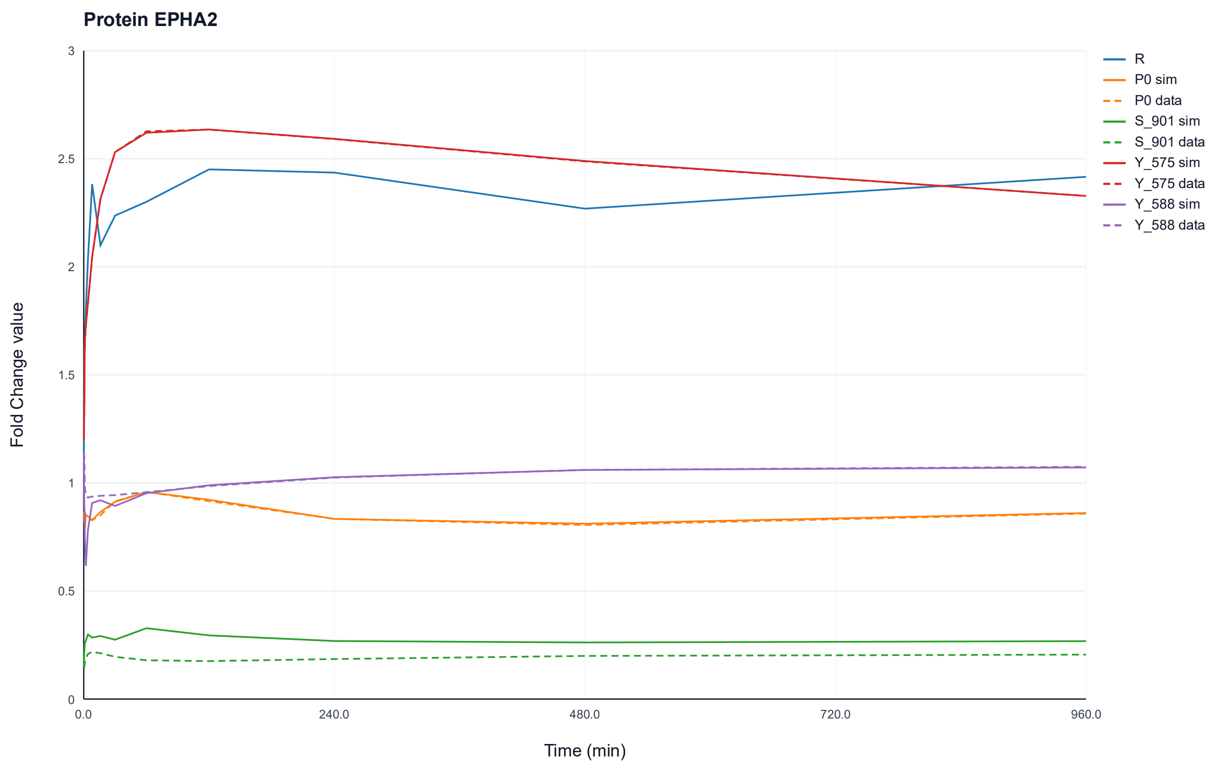

Figure 29: Protein-wise simulated and experimentally observed fold change profiles of protein EPHA2 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_901, Y_575 and Y_588. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

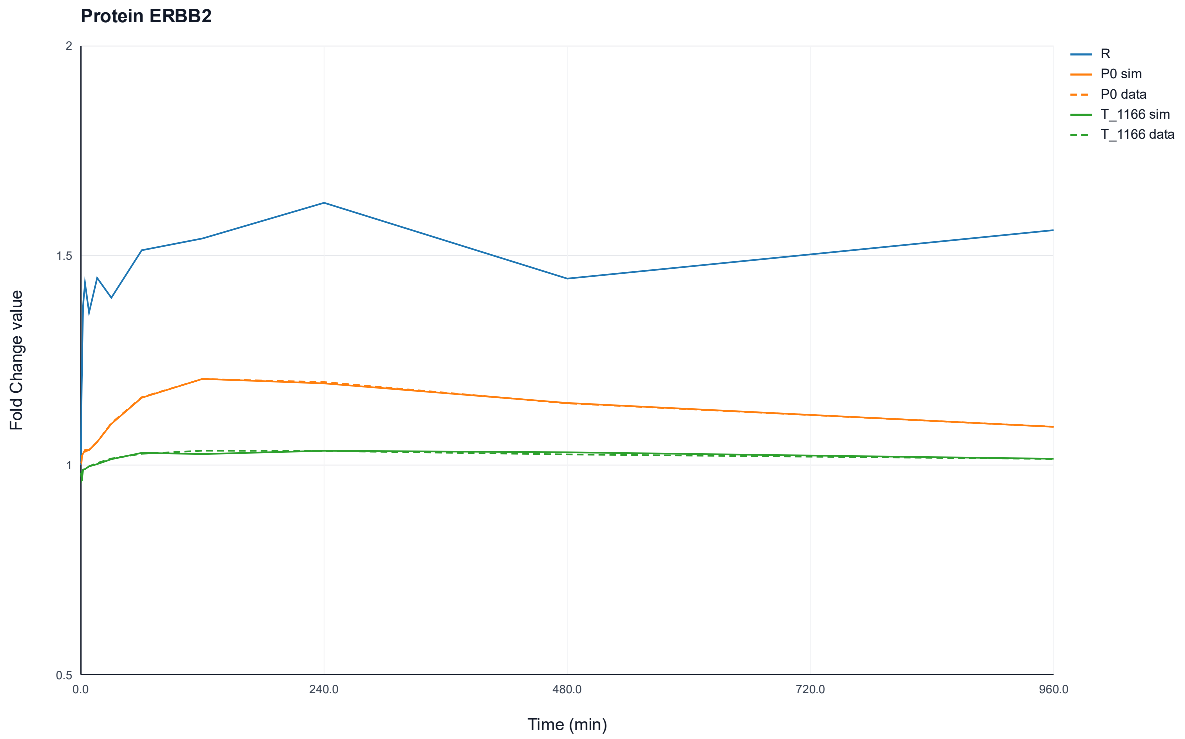

Figure 30: Protein-wise simulated and experimentally observed fold change profiles of protein ERBB2 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site T_1166. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

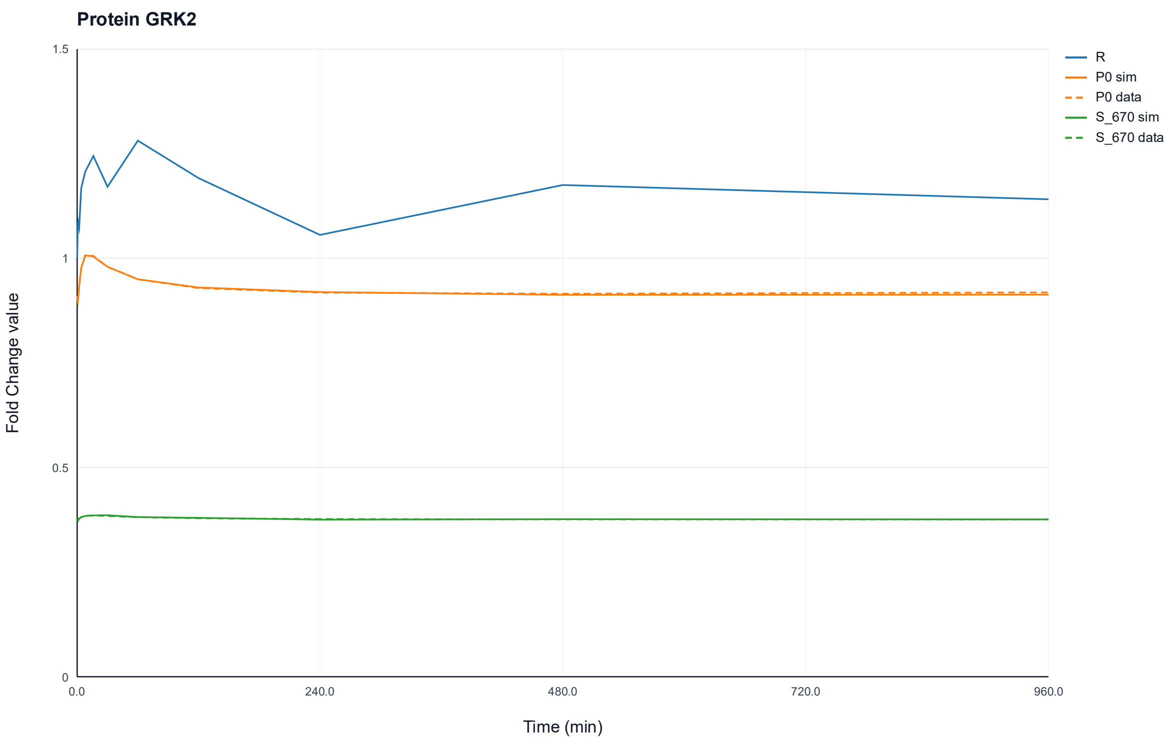

Figure 31: Protein-wise simulated and experimentally observed fold change profiles of protein GRK2 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site S_670. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

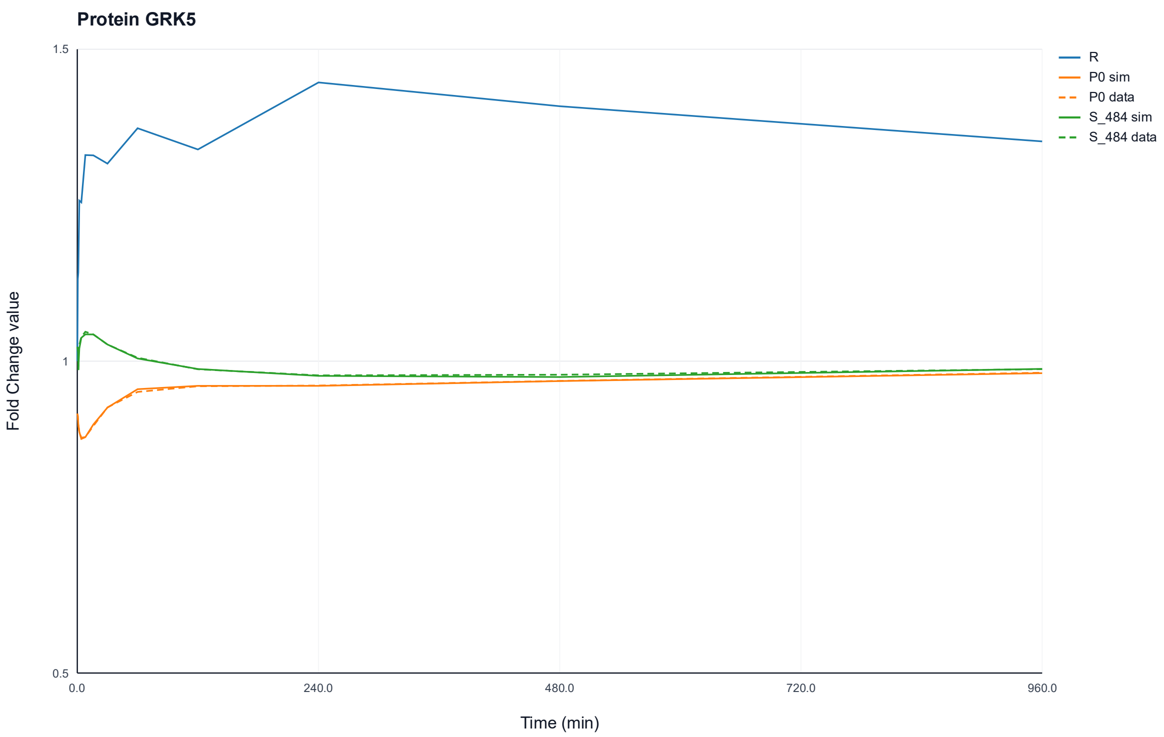

Figure 32: Protein-wise simulated and experimentally observed fold change profiles of protein GRK5 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site S_484. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

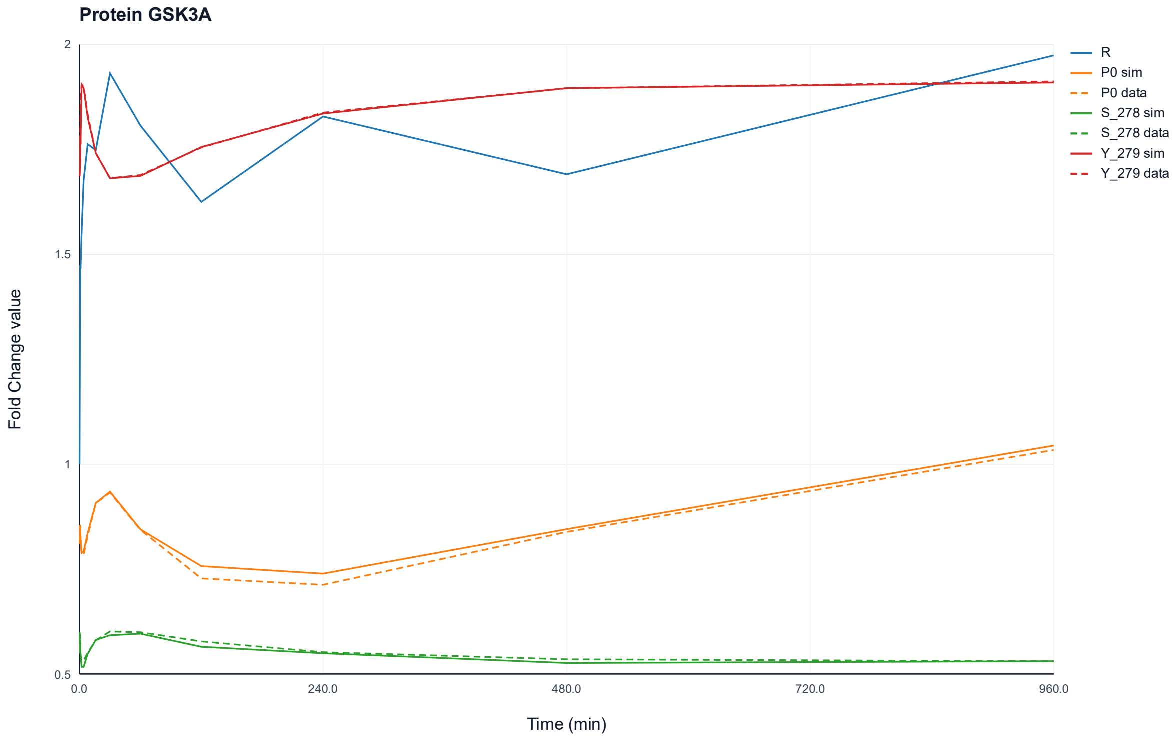

Figure 33: Protein-wise simulated and experimentally observed fold change profiles of protein GSK3A over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_278 and Y_279. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

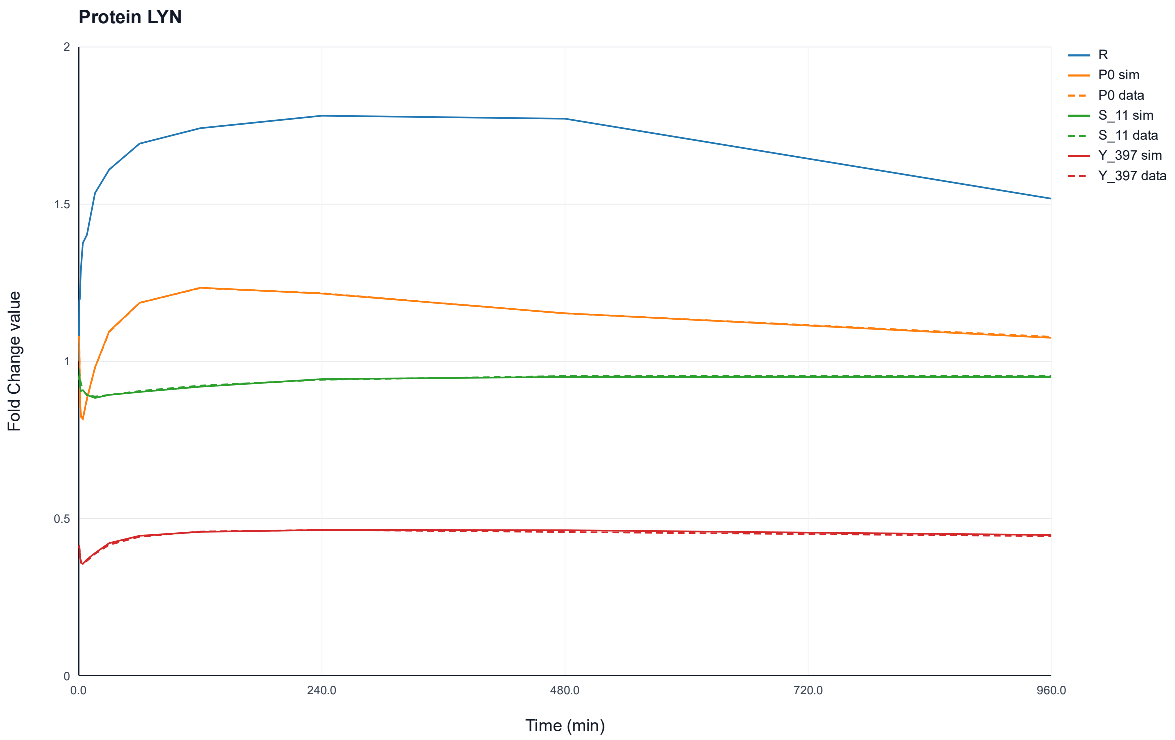

Figure 34: Protein-wise simulated and experimentally observed fold change profiles of protein LYN over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_11 and Y_397. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

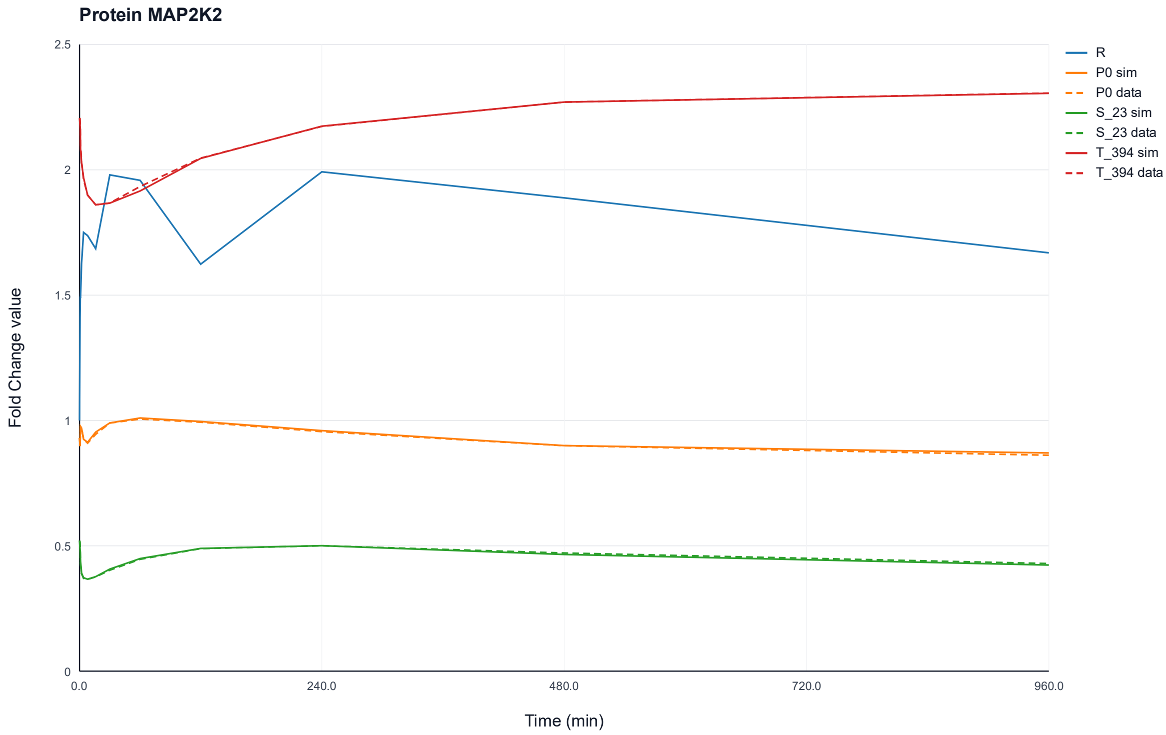

Figure 35: Protein-wise simulated and experimentally observed fold change profiles of protein MAP2K2 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_23 and T_394. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

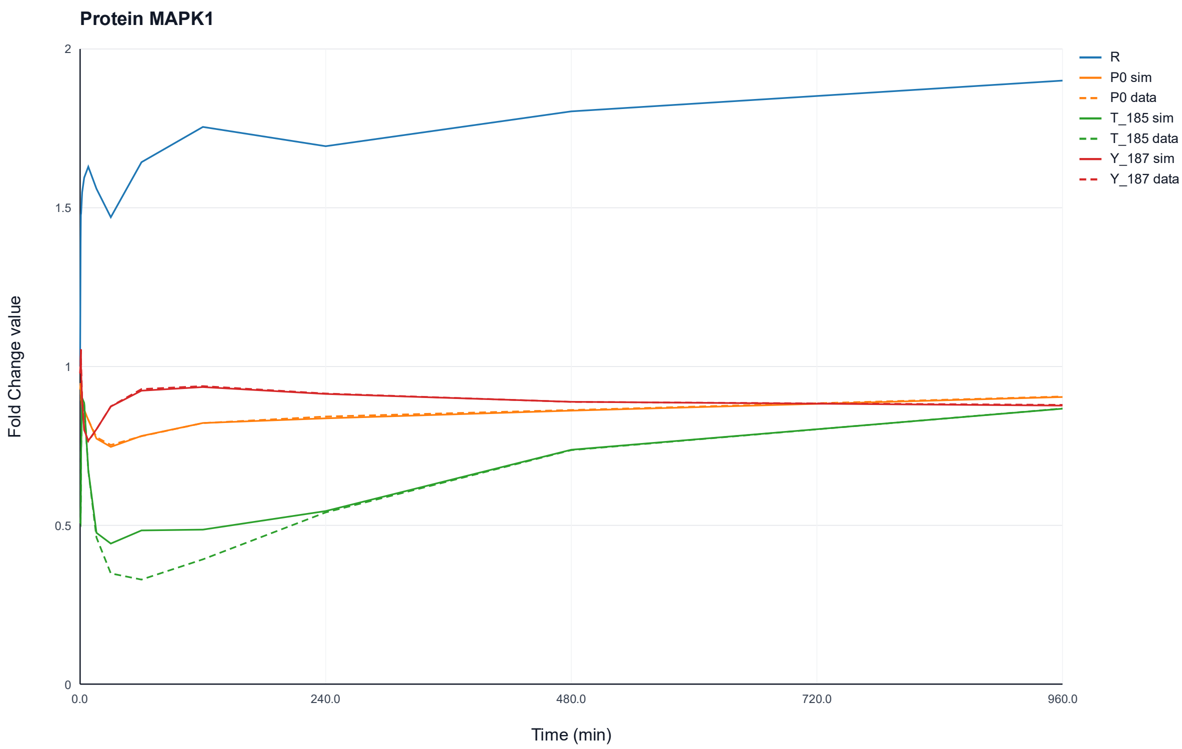

Figure 36: Protein-wise simulated and experimentally observed fold change profiles of protein MAPK1 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites T_185 and Y_187. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

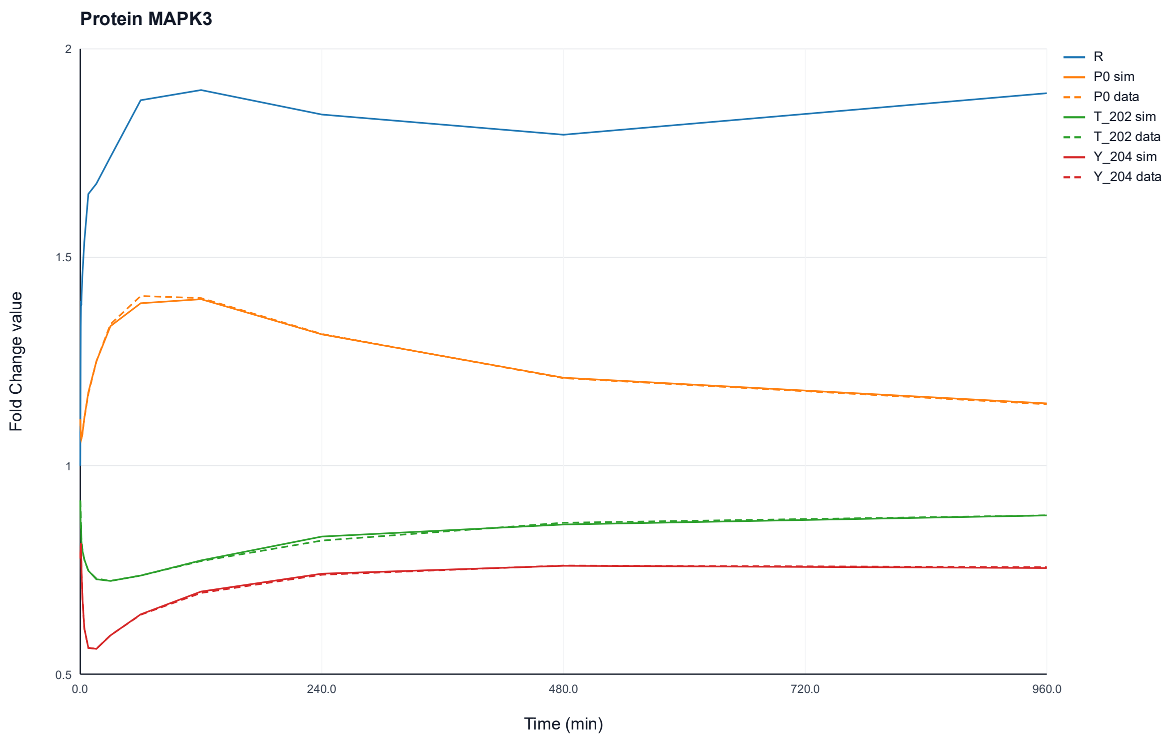

Figure 37: Protein-wise simulated and experimentally observed fold change profiles of protein MAPK3 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites T_202 and Y_204. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

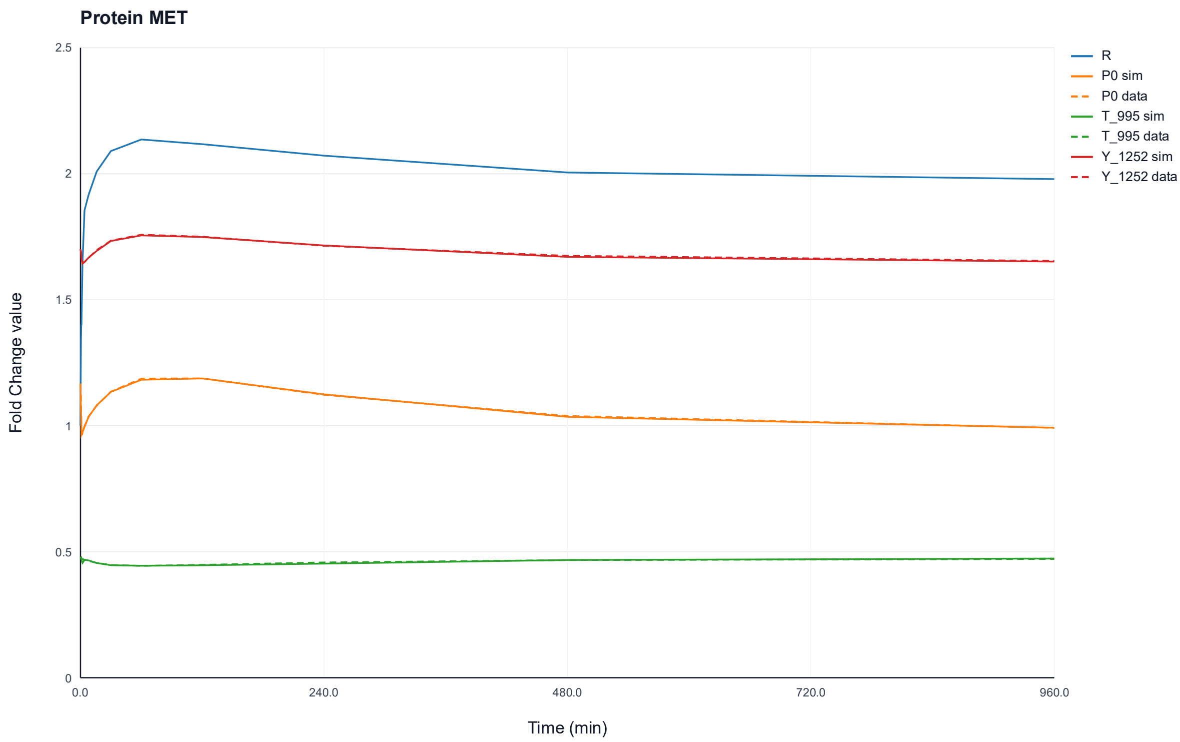

Figure 38: Protein-wise simulated and experimentally observed fold change profiles of protein MET over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites T_995 and Y_1252. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

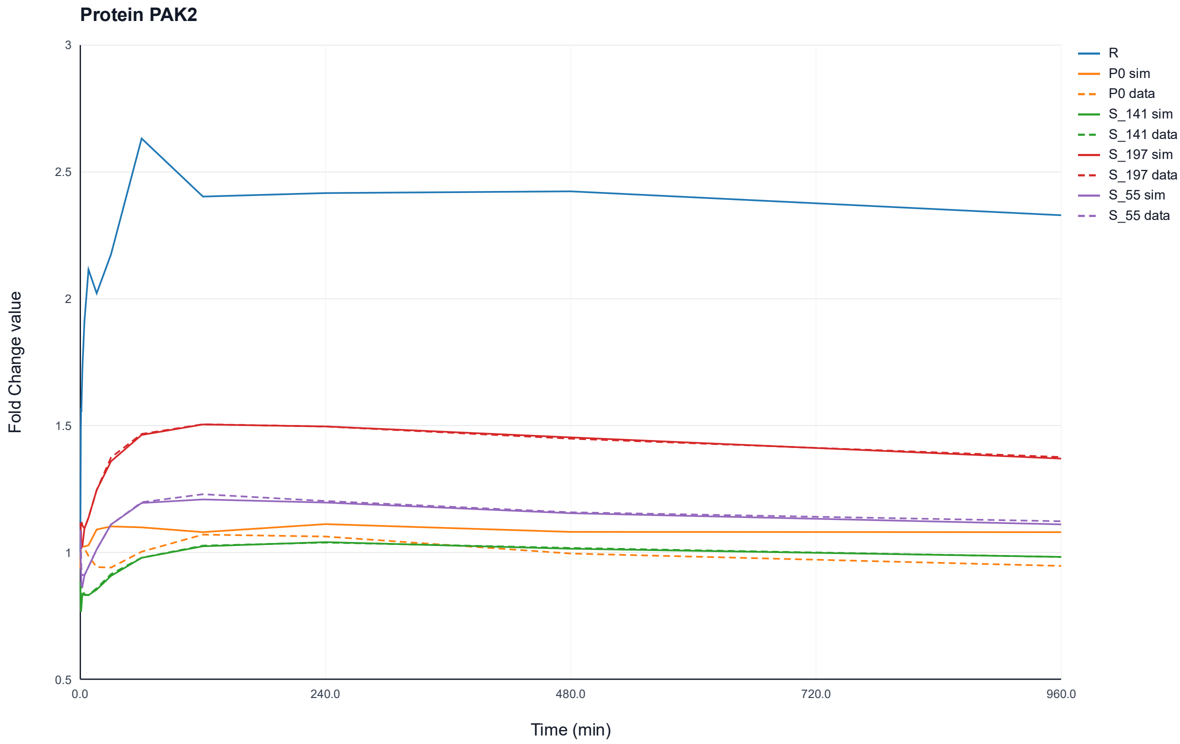

Figure 39: Protein-wise simulated and experimentally observed fold change profiles of protein PAK2 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_141, S_197 and S_55. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

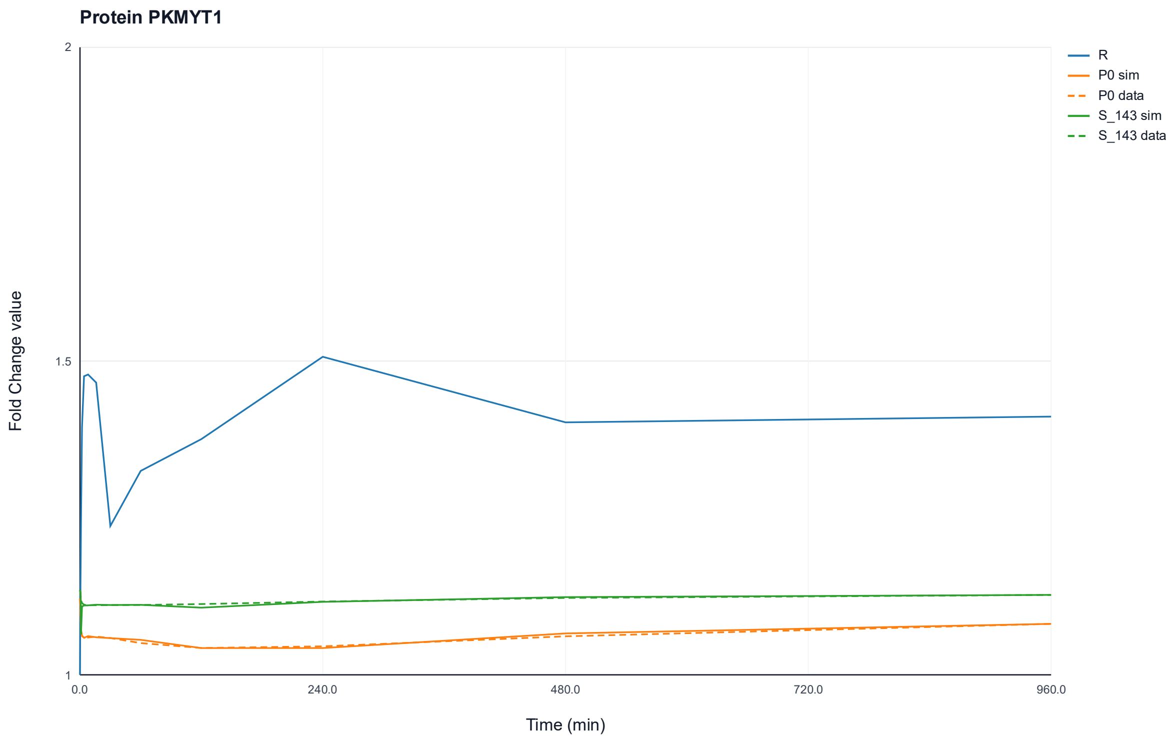

Figure 40: Protein-wise simulated and experimentally observed fold change profiles of protein PKMYT1 over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site S_143. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

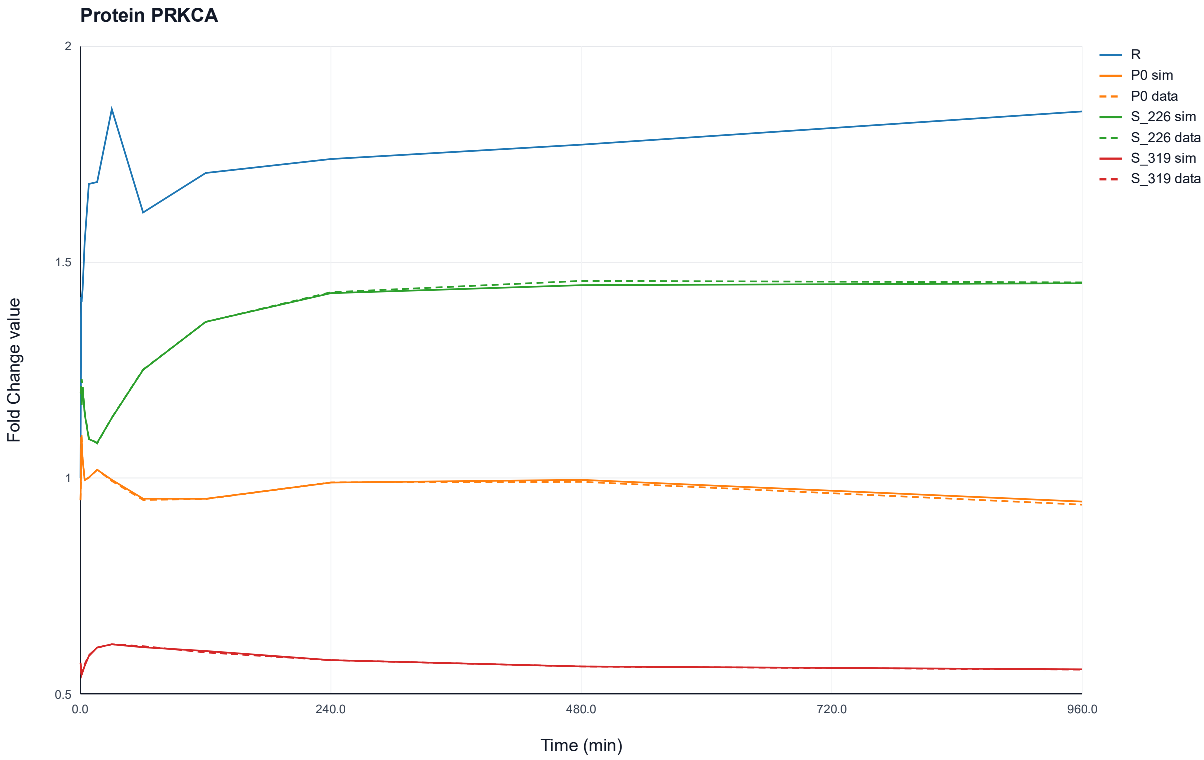

Figure 41: Protein-wise simulated and experimentally observed fold change profiles of protein PRKCA over time. The plot shows the optimized ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_226 and S_319. Solid lines represent model simulations, whereas dashed lines indicate the measured data.

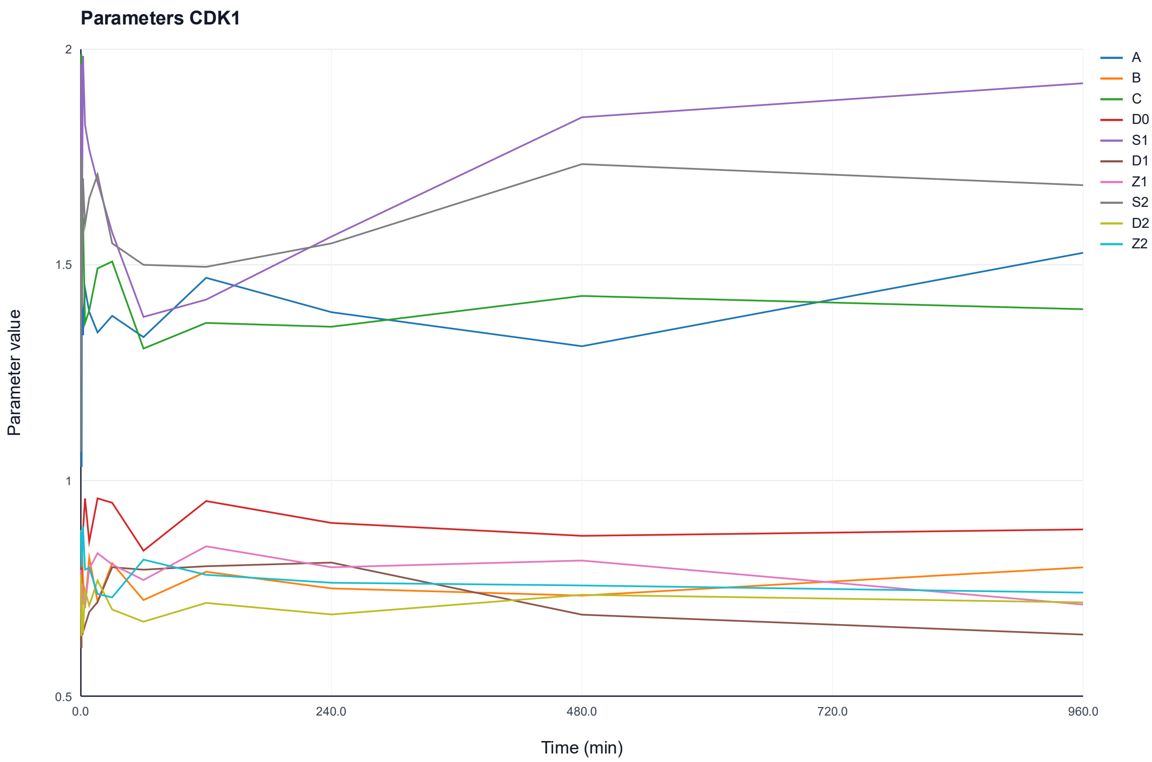

Figure 42: Temporal dynamics of the protein-wise estimated kinetic parameters of protein CDK1. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

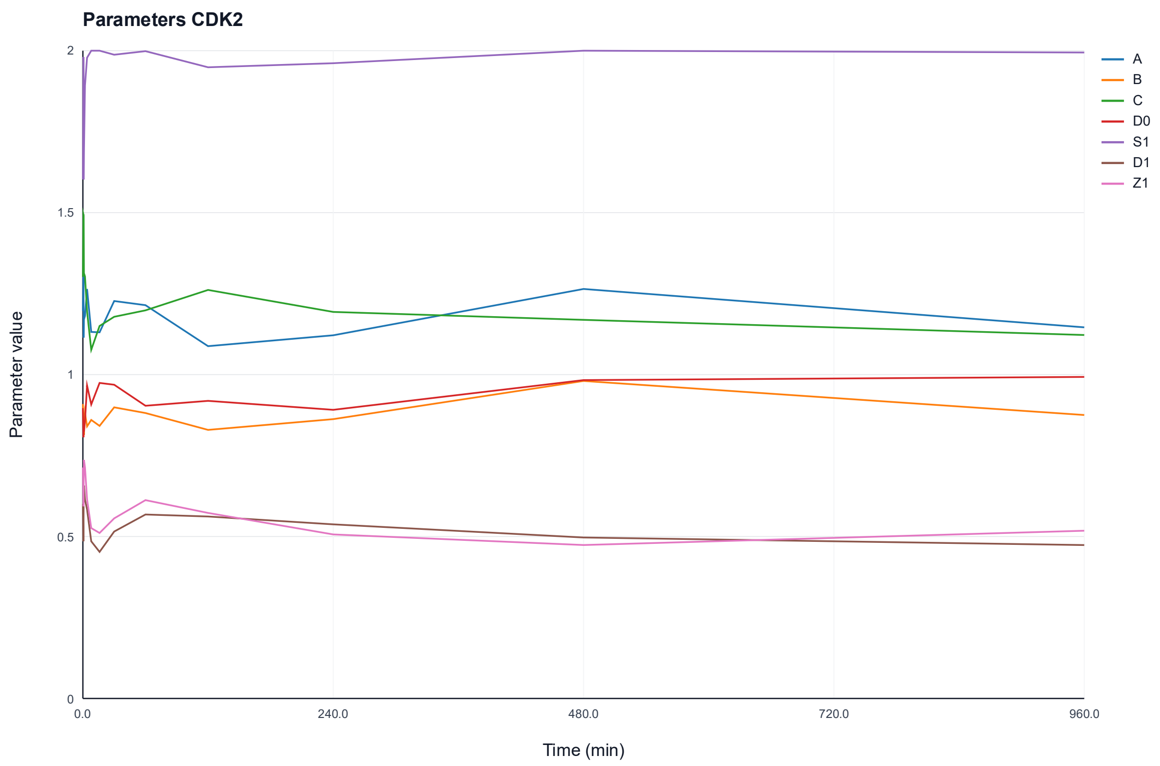

Figure 43: Temporal dynamics of the protein-wise estimated kinetic parameters of protein CDK2. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

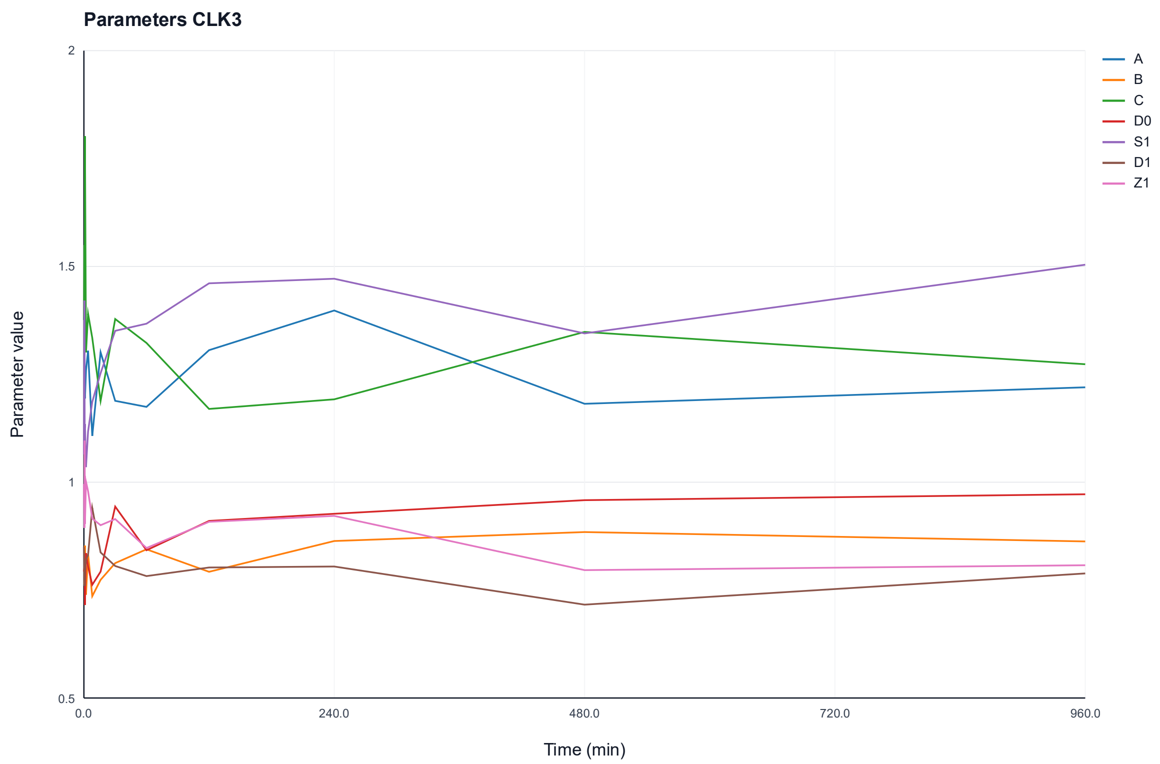

Figure 44: Temporal dynamics of the protein-wise estimated kinetic parameters of protein CLK3. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

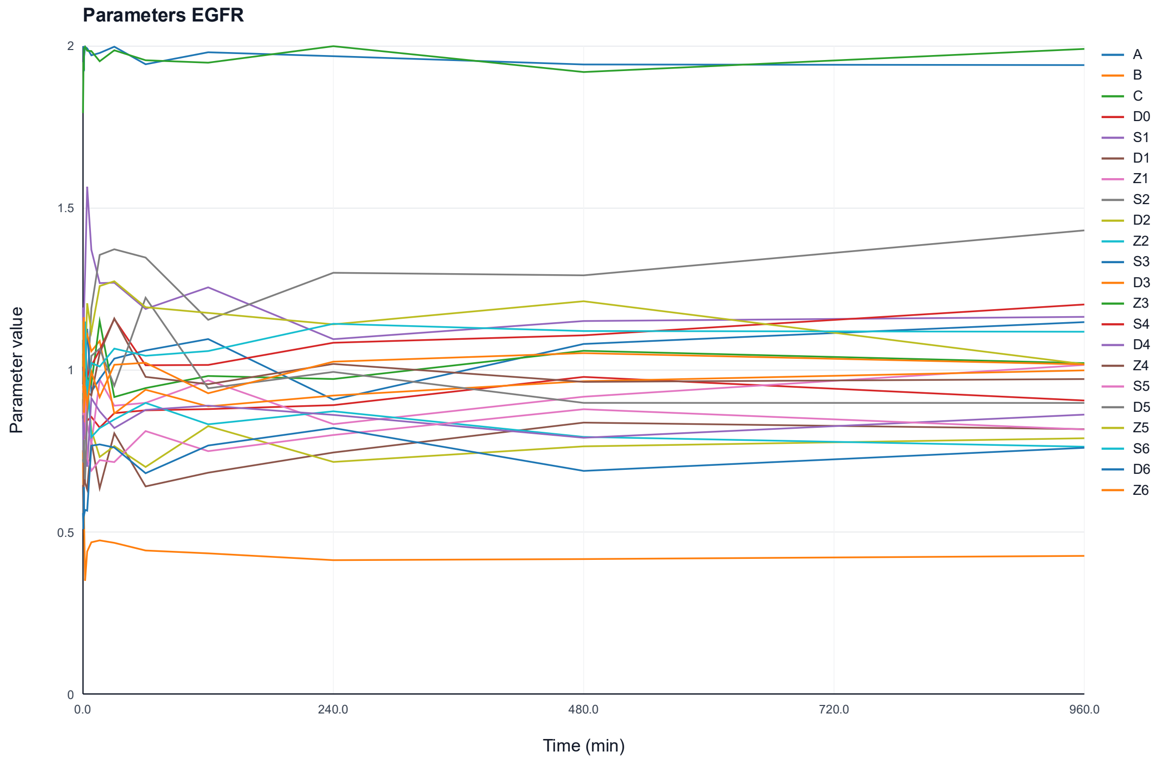

Figure 45: Temporal dynamics of the protein-wise estimated kinetic parameters of protein EGFR. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

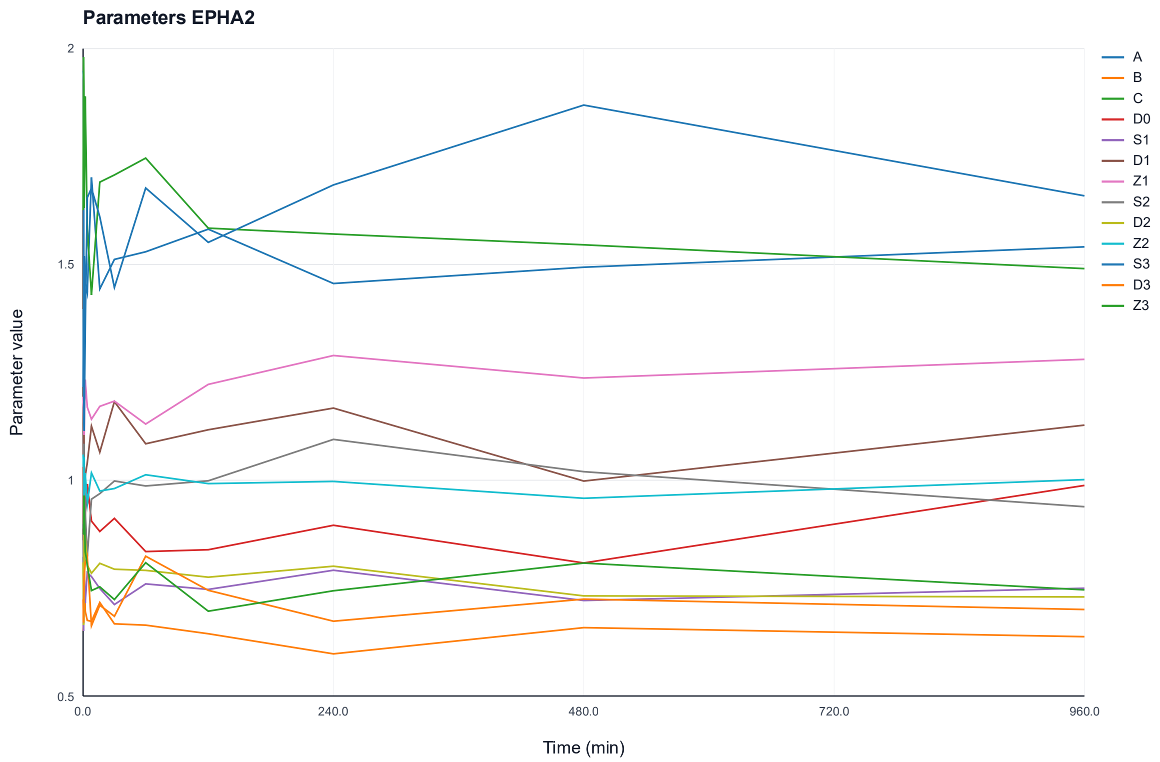

Figure 46: Temporal dynamics of the protein-wise estimated kinetic parameters of protein EPHA2. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

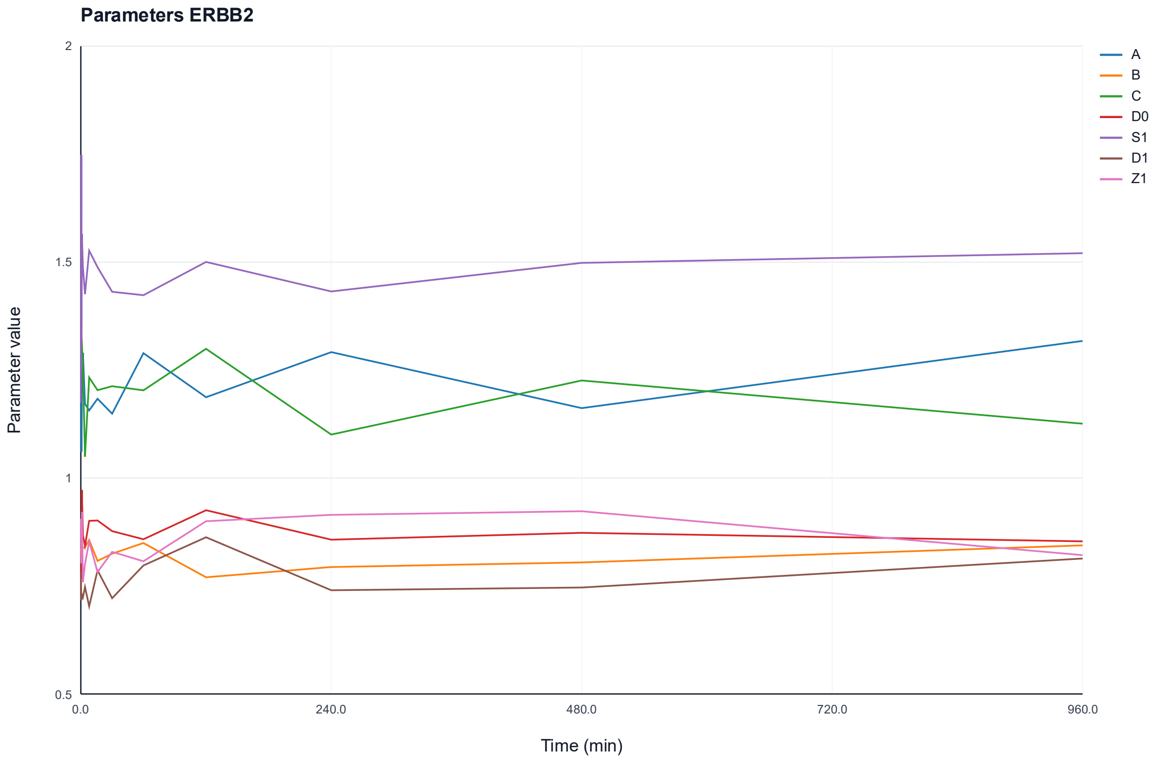

Figure 47: Temporal dynamics of the protein-wise estimated kinetic parameters of protein ERBB2. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

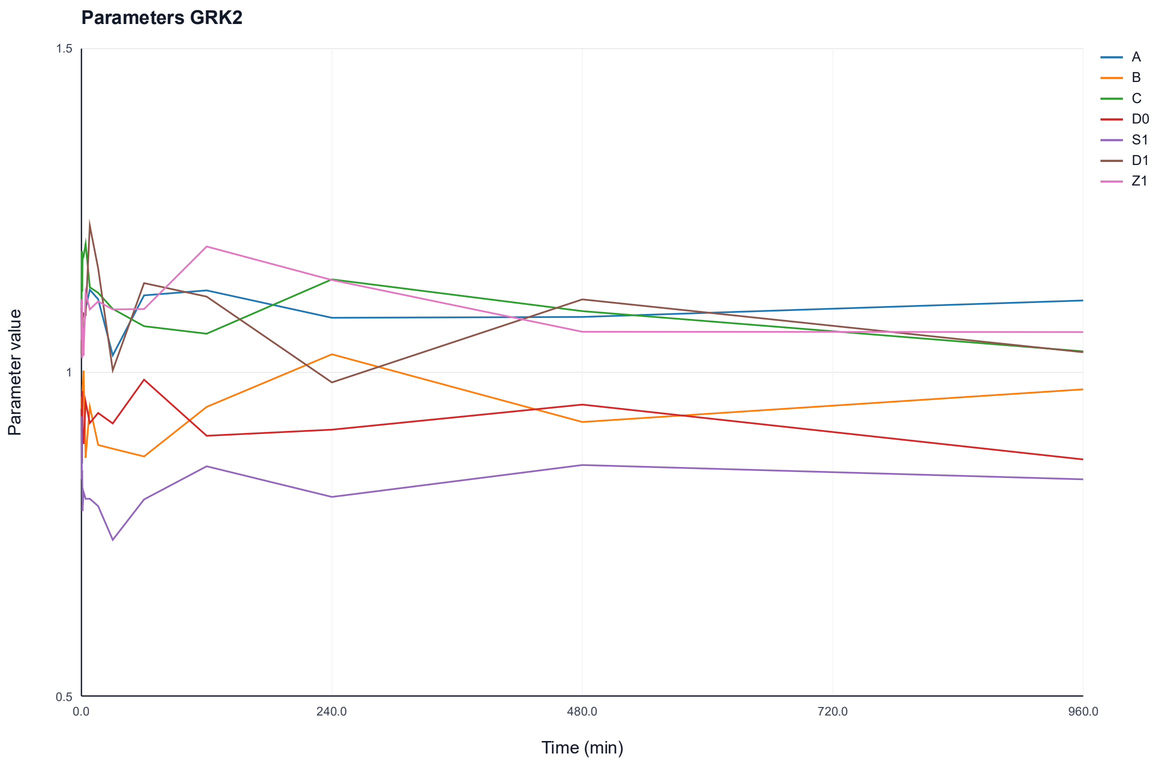

Figure 48: Temporal dynamics of the protein-wise estimated kinetic parameters of protein GRK2. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

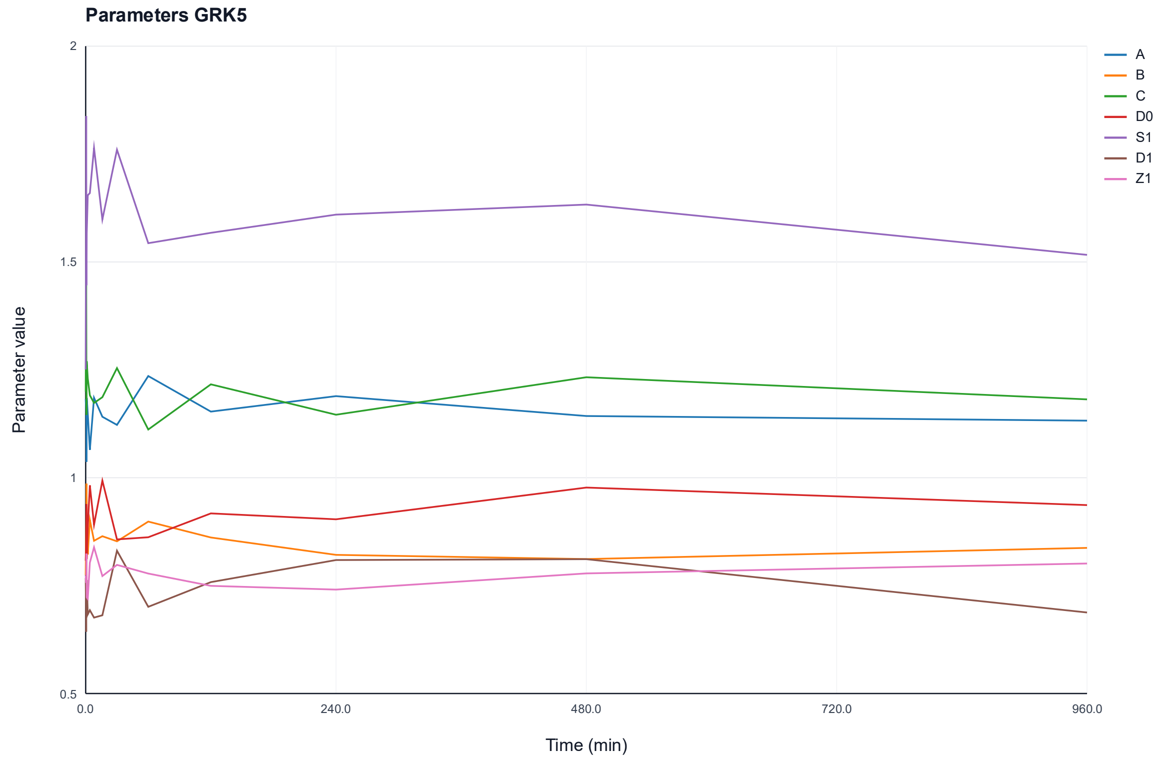

Figure 49: Temporal dynamics of the protein-wise estimated kinetic parameters of protein GRK5. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

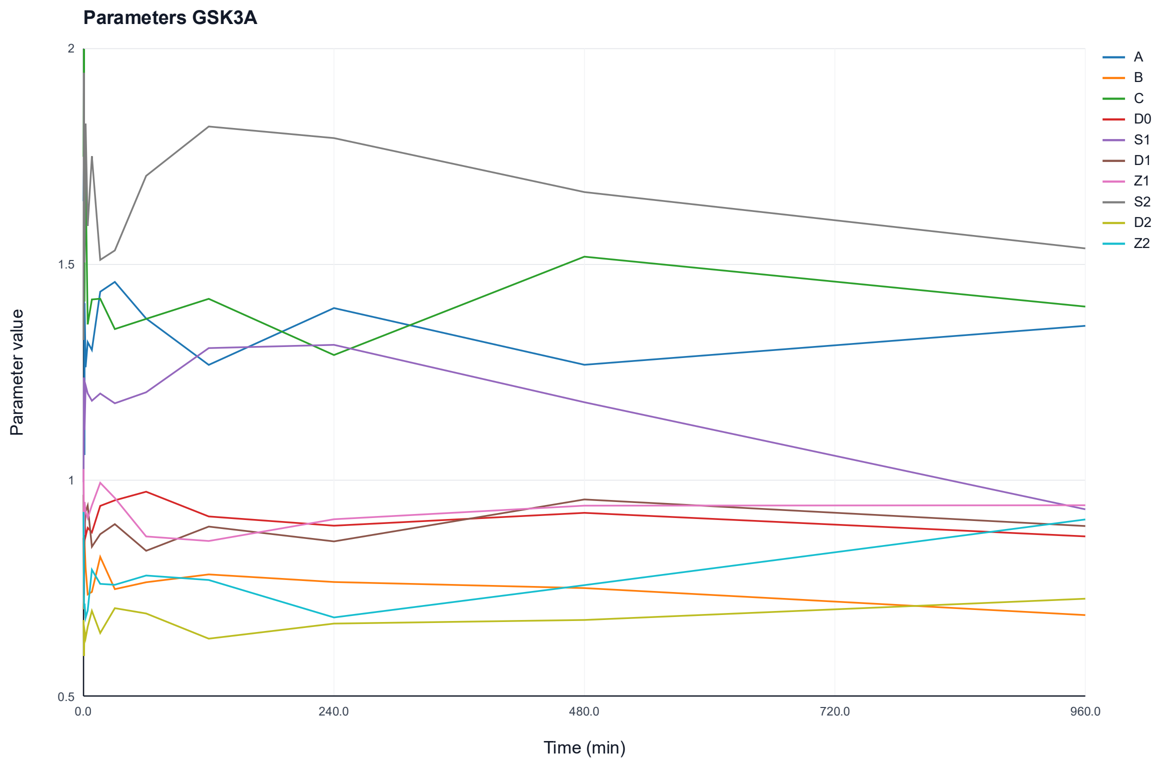

Figure 50: Temporal dynamics of the protein-wise estimated kinetic parameters of protein GSK3A. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

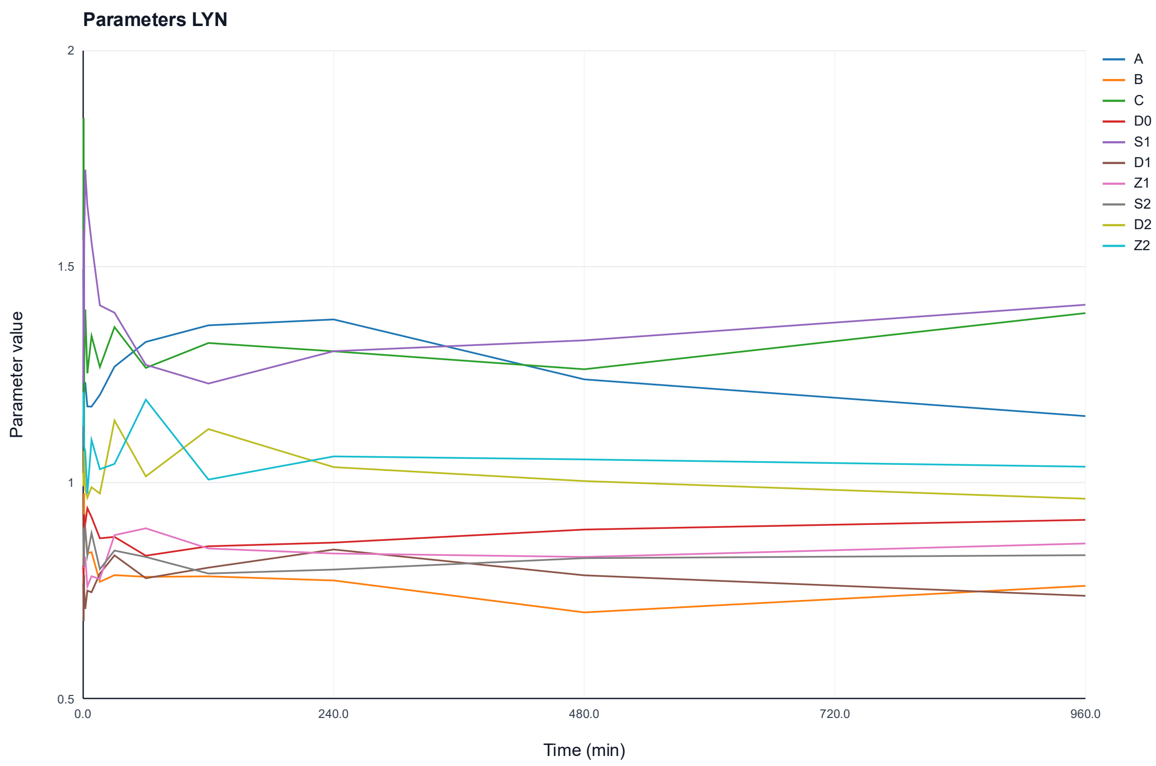

Figure 51: Temporal dynamics of the protein-wise estimated kinetic parameters of protein LYN. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

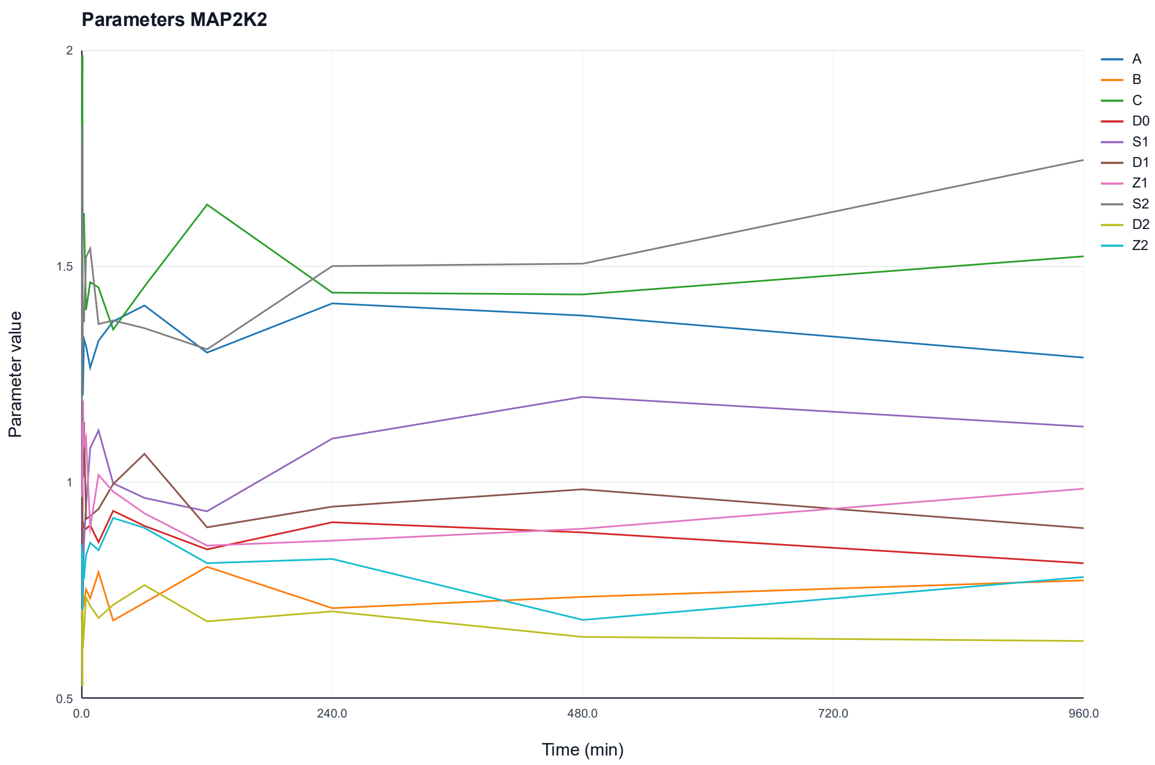

Figure 52: Temporal dynamics of the protein-wise estimated kinetic parameters of protein MAP2K2. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

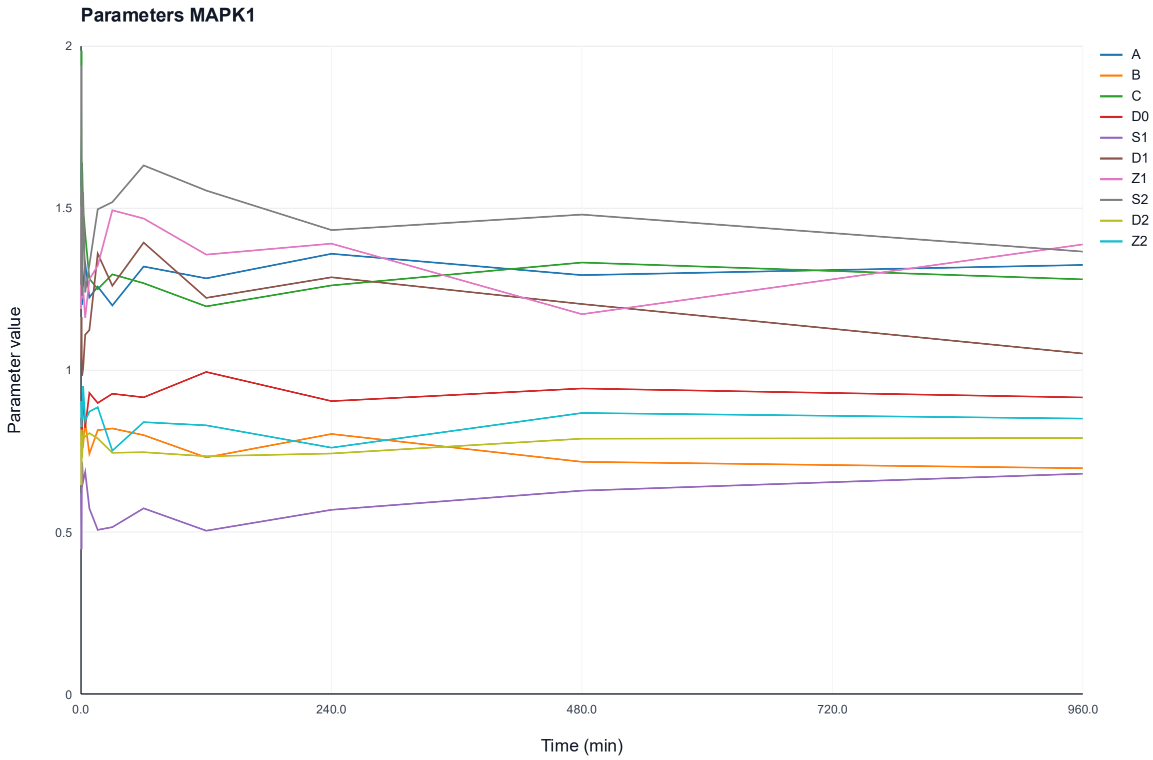

Figure 53: Temporal dynamics of the protein-wise estimated kinetic parameters of protein MAPK1. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

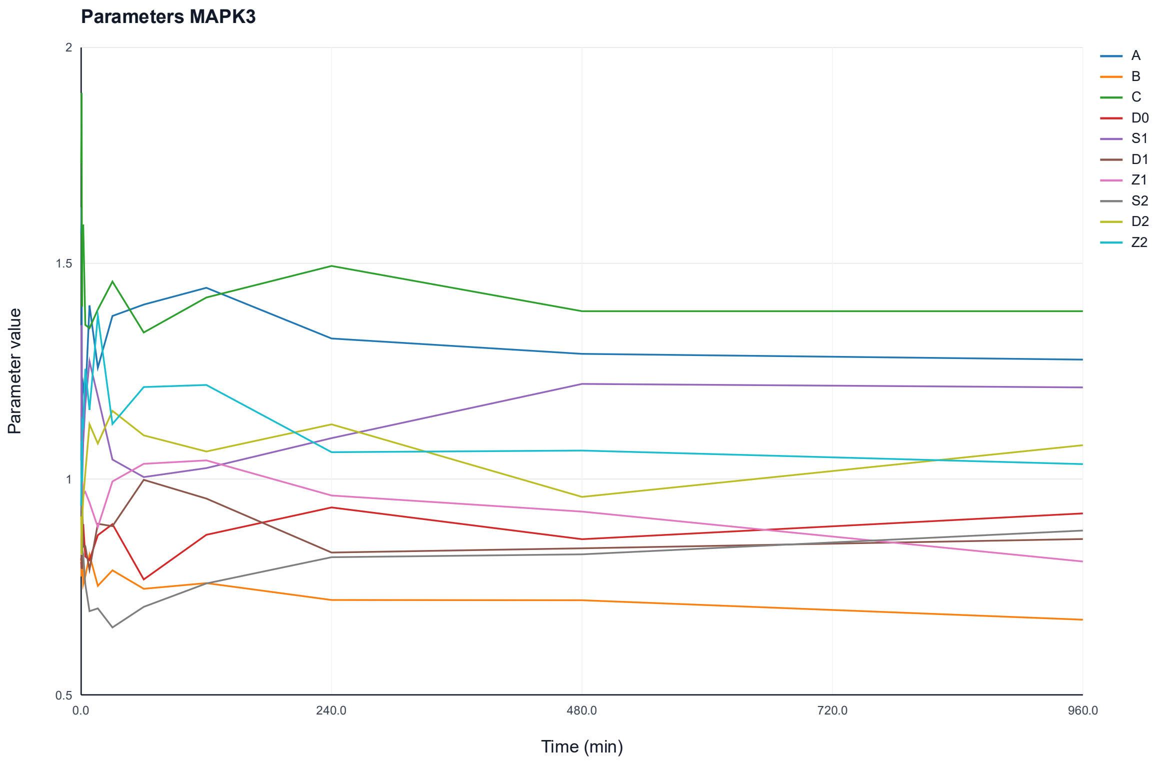

Figure 54: Temporal dynamics of the protein-wise estimated kinetic parameters of protein MAPK3. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

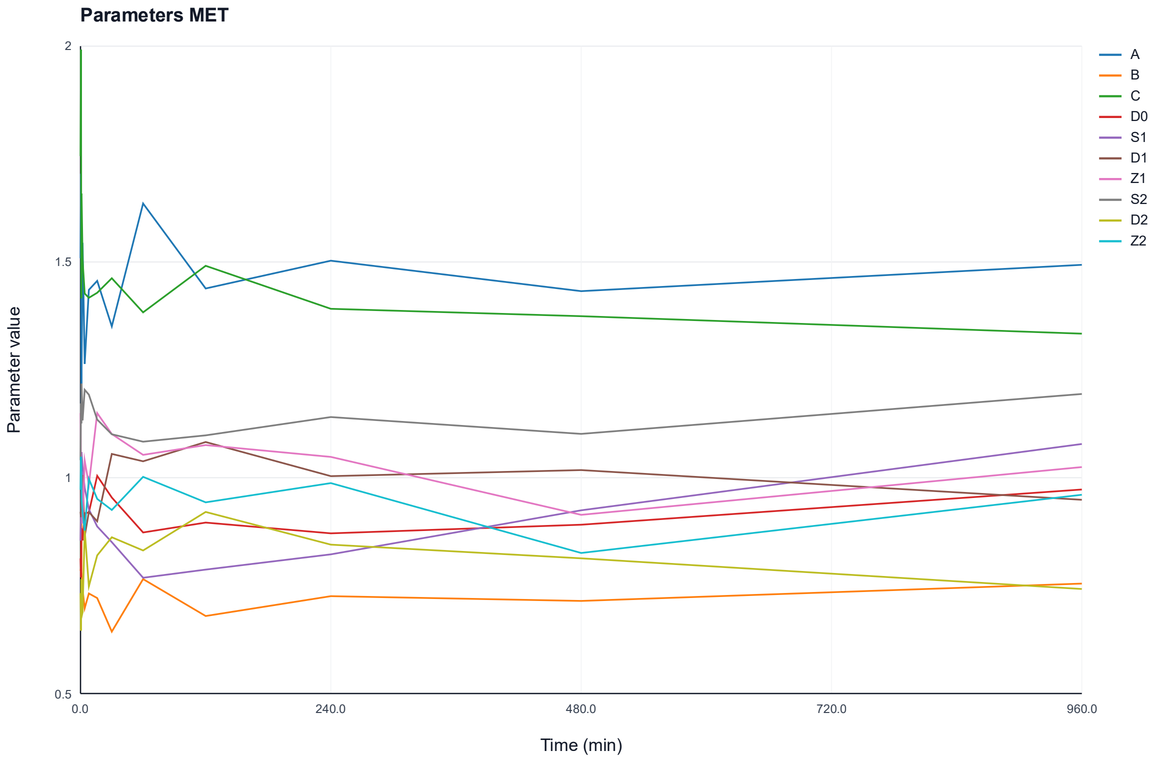

Figure 55: Temporal dynamics of the protein-wise estimated kinetic parameters of protein GSK3A. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

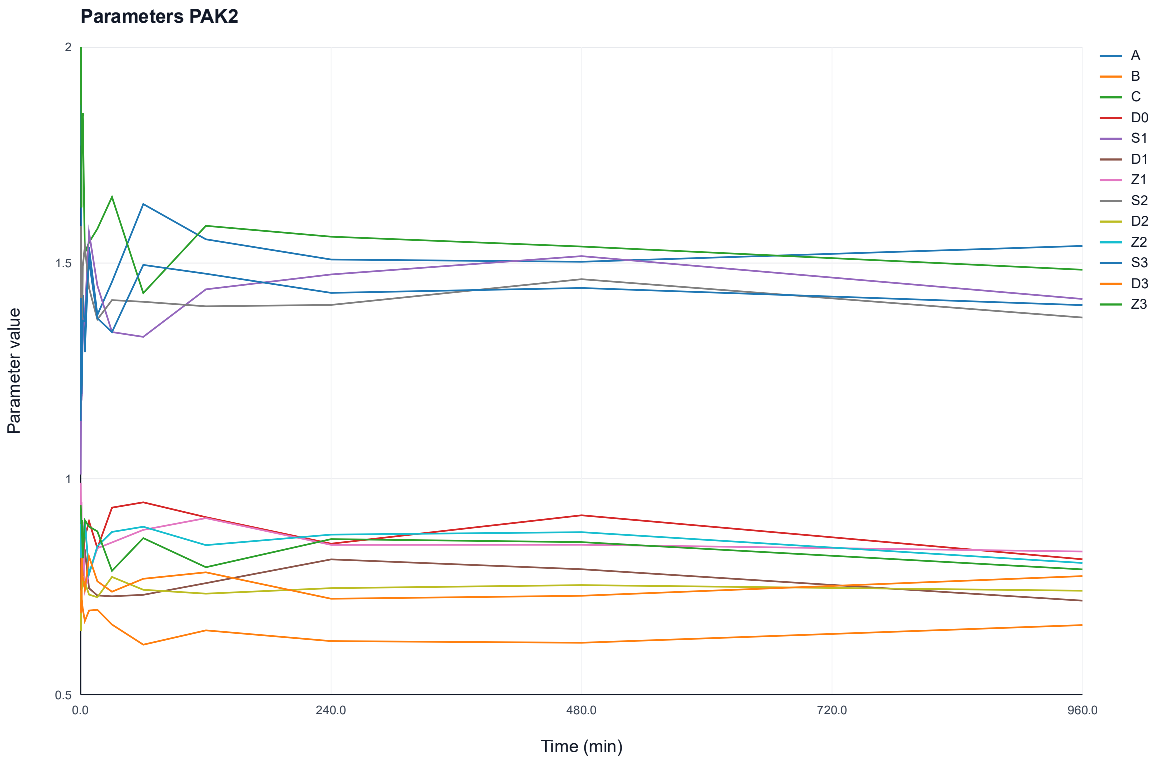

Figure 56: Temporal dynamics of the protein-wise estimated kinetic parameters of protein PAK2. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

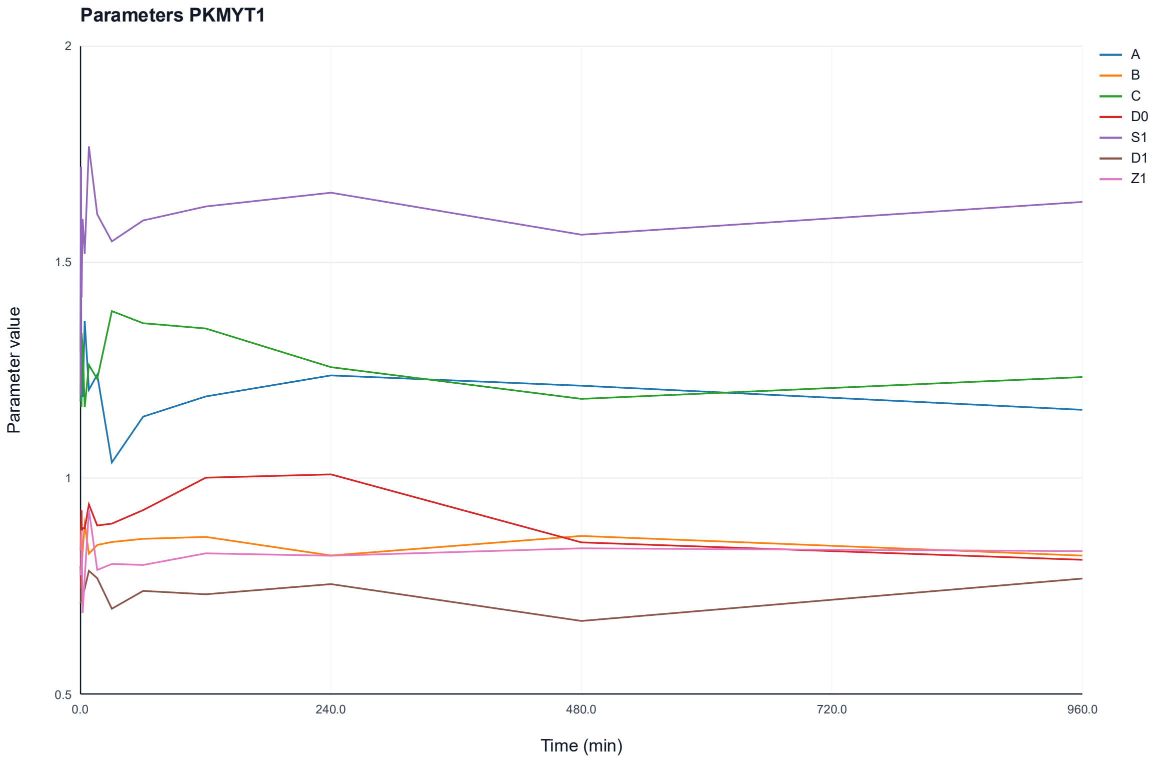

Figure 57: Temporal dynamics of the protein-wise estimated kinetic parameters of protein PKMYT1. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

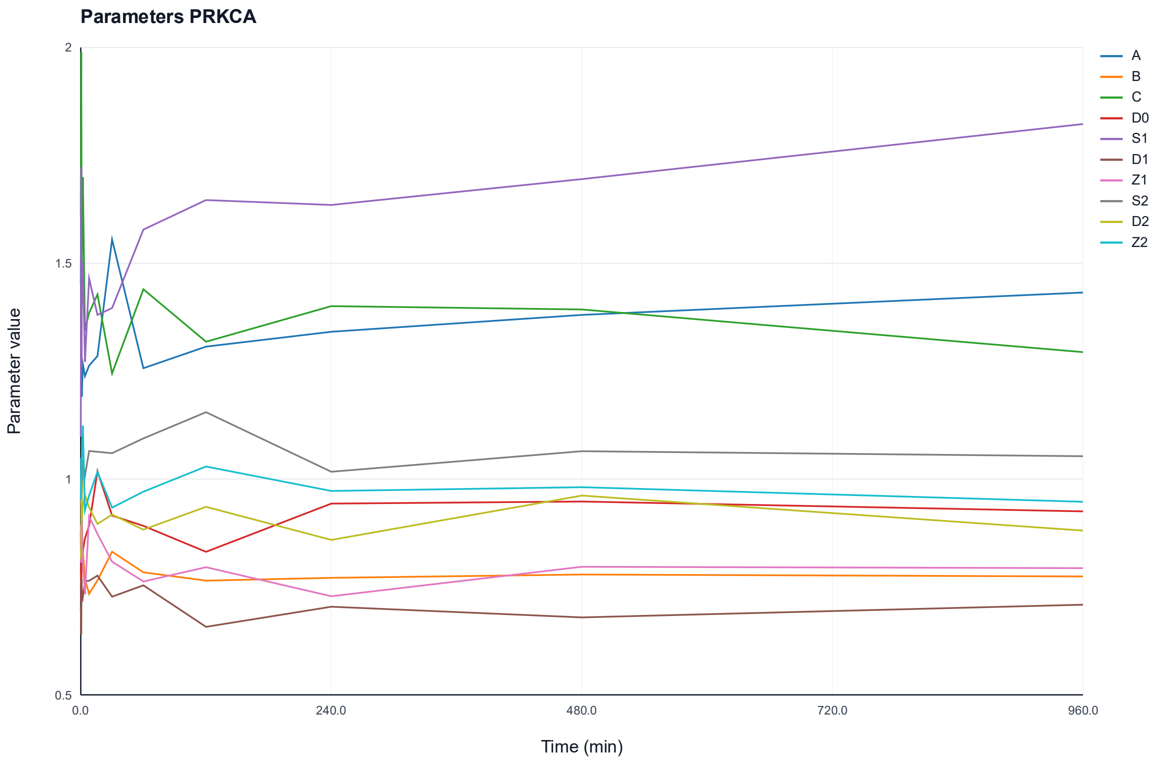

Figure 58: Temporal dynamics of the protein-wise estimated kinetic parameters of protein PRKCA. The plot illustrates the trajectories of parameters A, B, C, D, S, and Z over a 960-minute period.

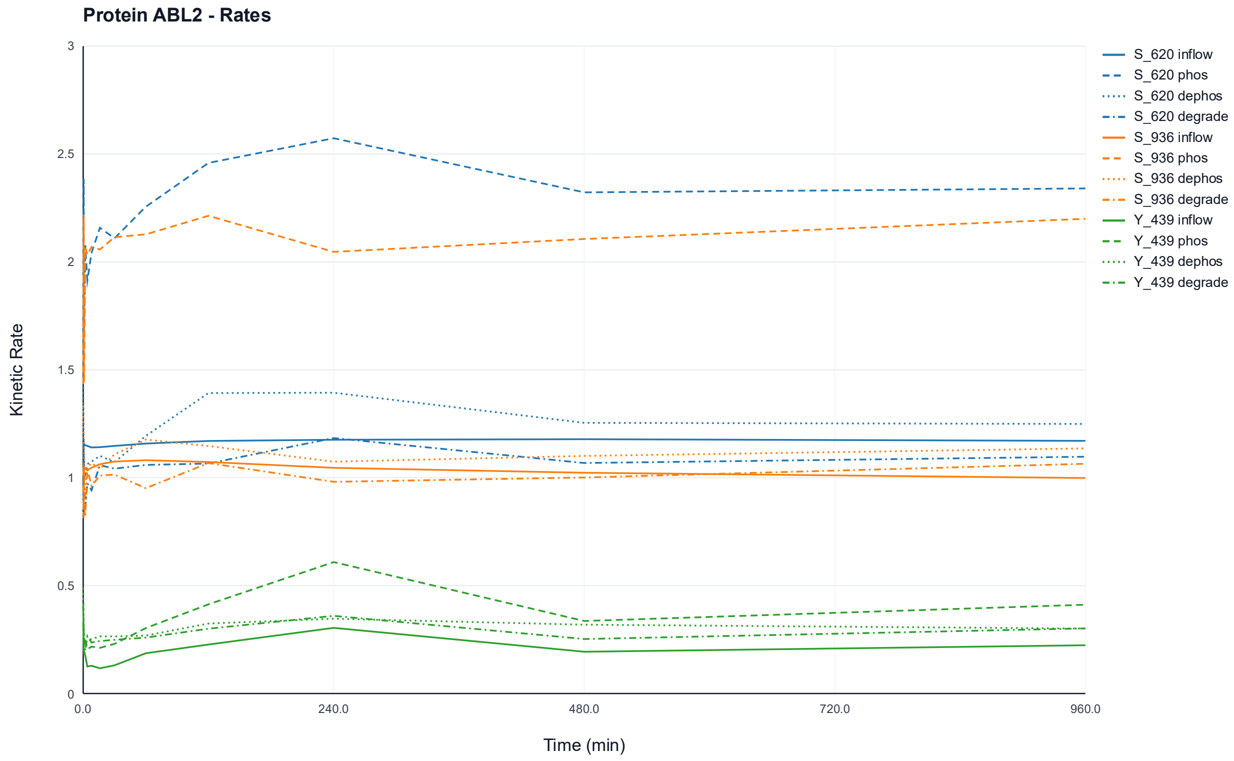

Figure 59: Time-resolved kinetic rate profiles of protein-wise estimation for the ABL2 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites S_620, S_936 and Y_439.

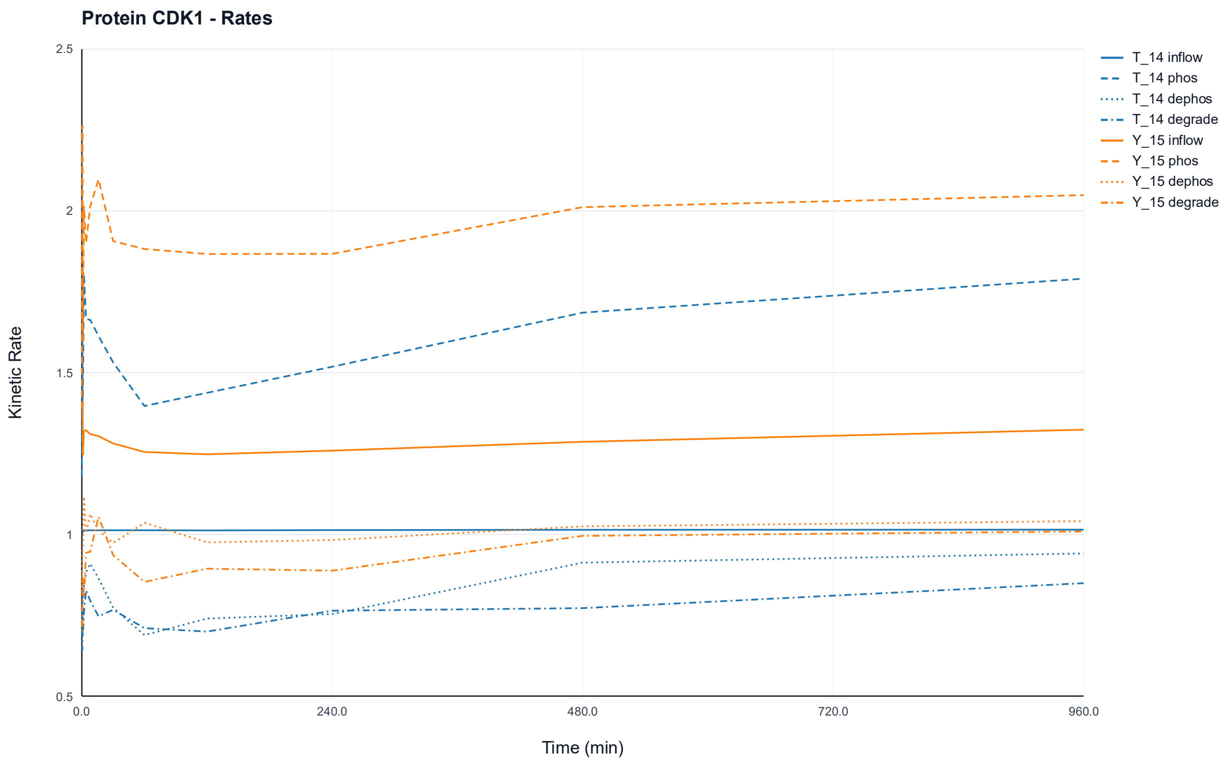

Figure 60: Time-resolved kinetic rate profiles of protein-wise estimation for the CDK1 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites T_14, and Y_15.

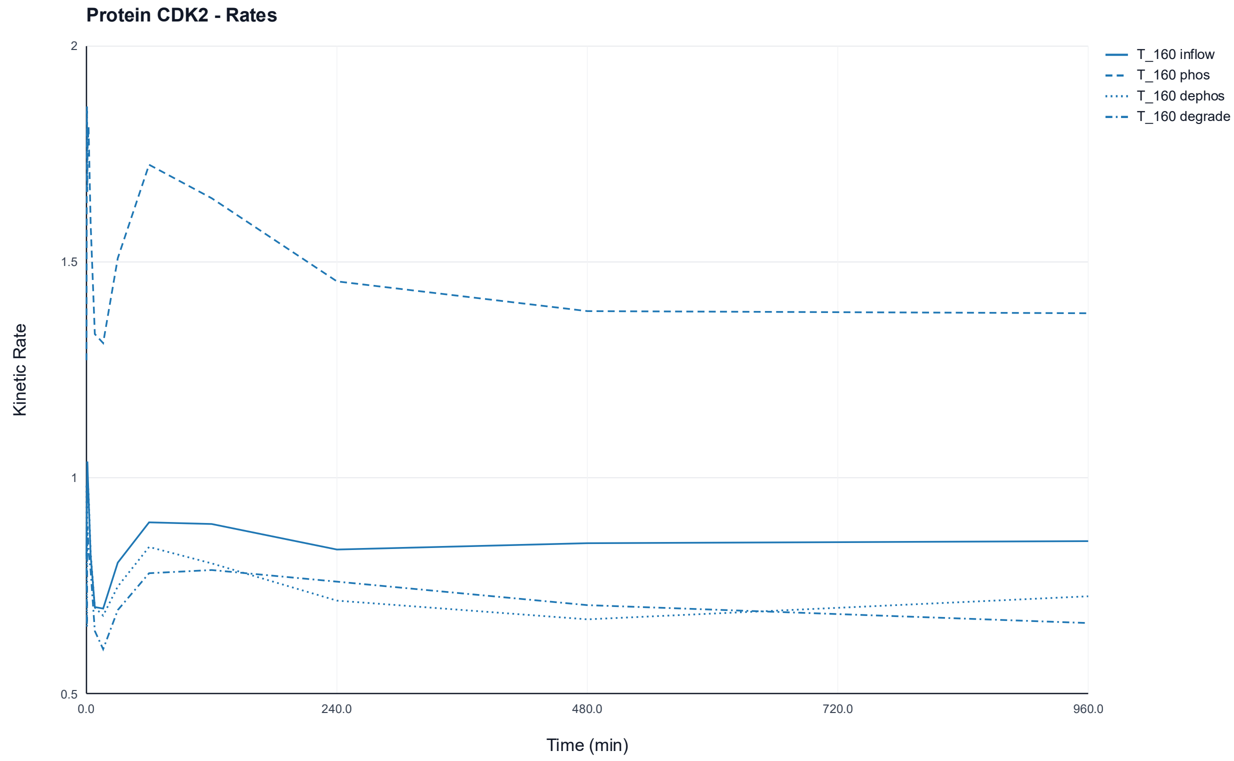

Figure 61: Time-resolved kinetic rate profiles of protein-wise estimation for the CDK2 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at site T_180.

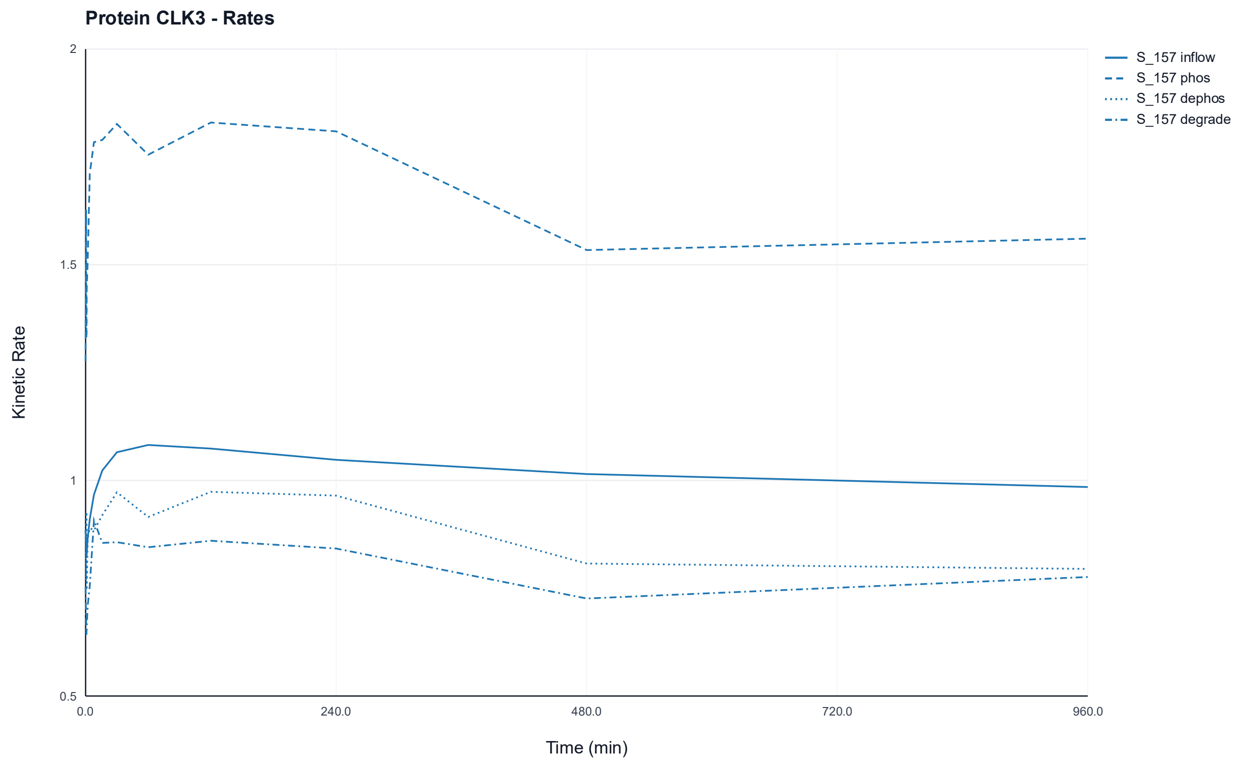

Figure 62: Time-resolved kinetic rate profiles of protein-wise estimation for the CLK3 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at site S_157.

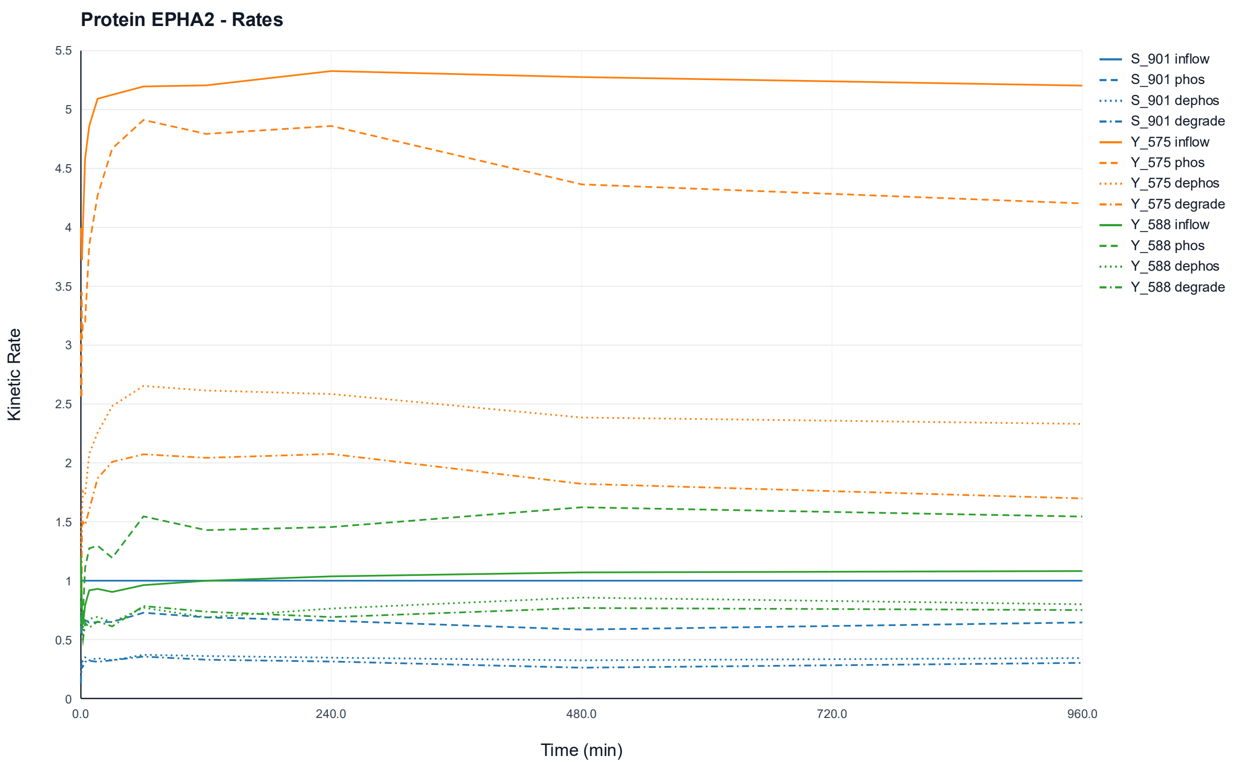

Figure 63: Time-resolved kinetic rate profiles of protein-wise estimation for the EPHA2 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites S_901, Y_575 and Y_588.

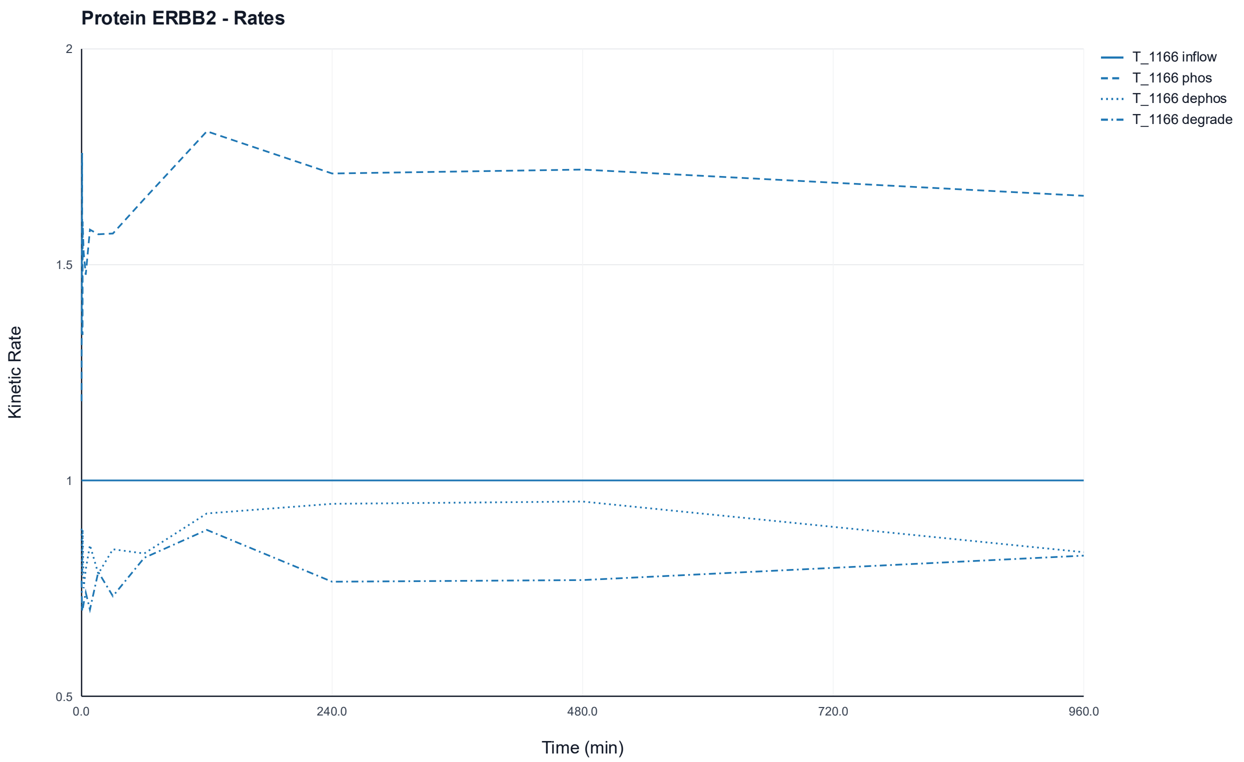

Figure 64: Time-resolved kinetic rate profiles of protein-wise estimation for the ERBB2 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at site T_1166.

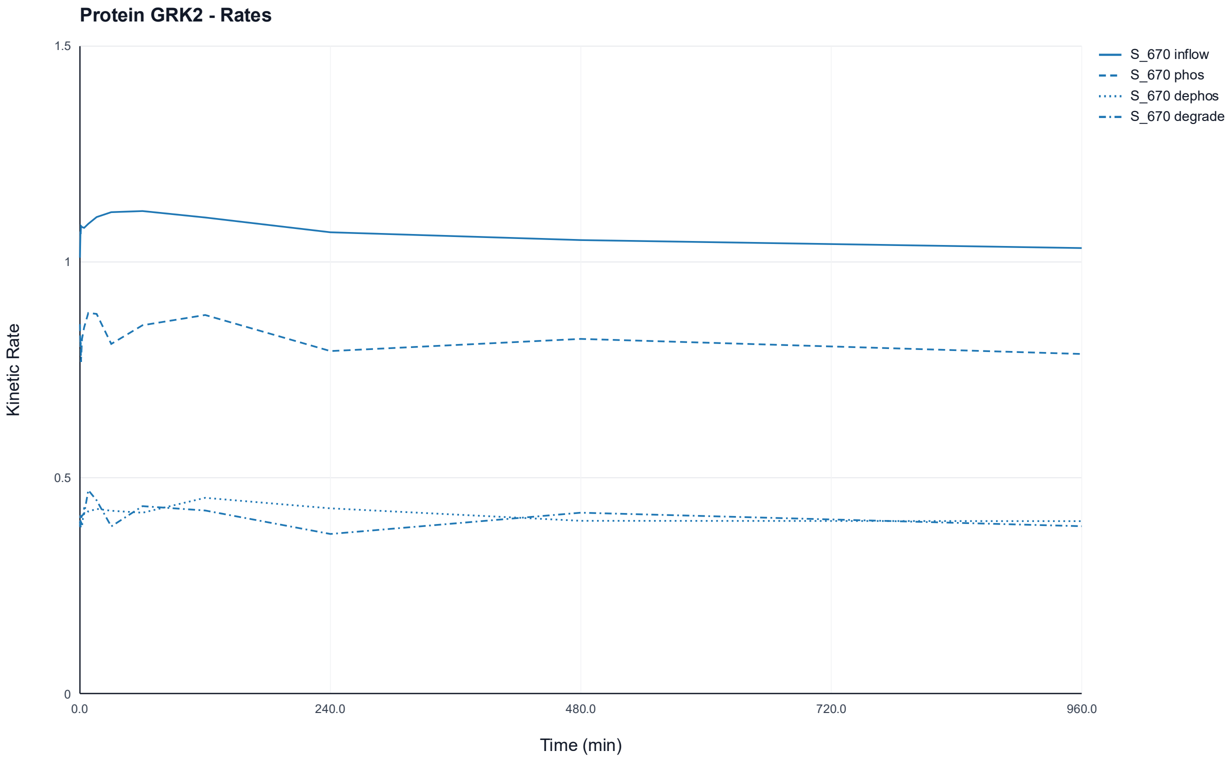

Figure 65: Time-resolved kinetic rate profiles of protein-wise estimation for the GRK2 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at site S_670.

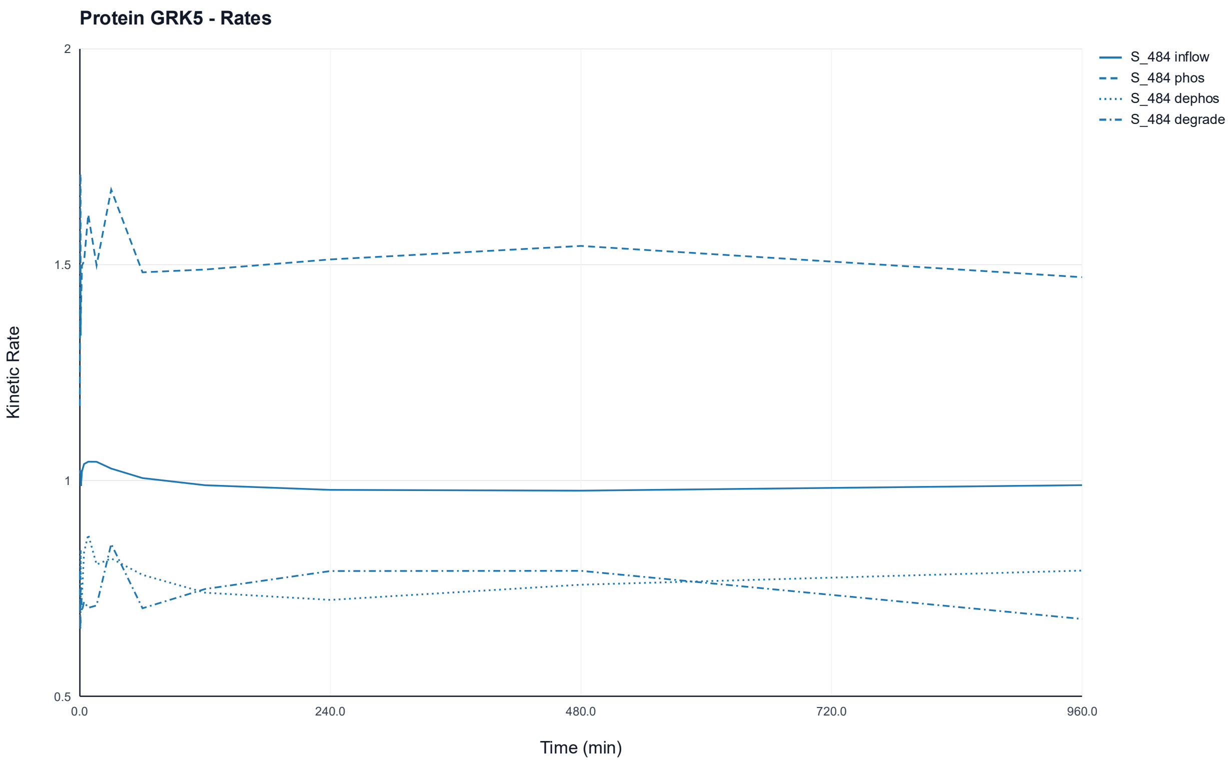

Figure 66: Time-resolved kinetic rate profiles of protein-wise estimation for the GRK5 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at site S_484.

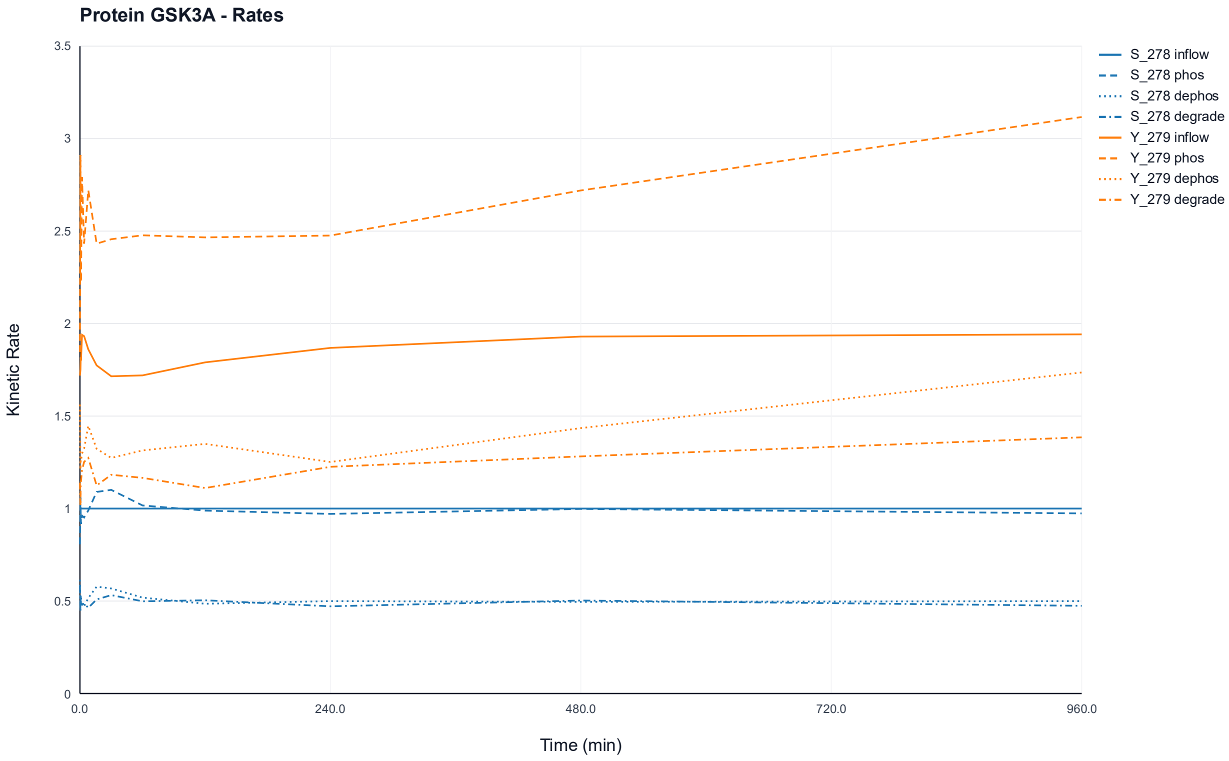

Figure 67: Time-resolved kinetic rate profiles of protein-wise estimation for the GSK3A protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites S_278, and Y_279.

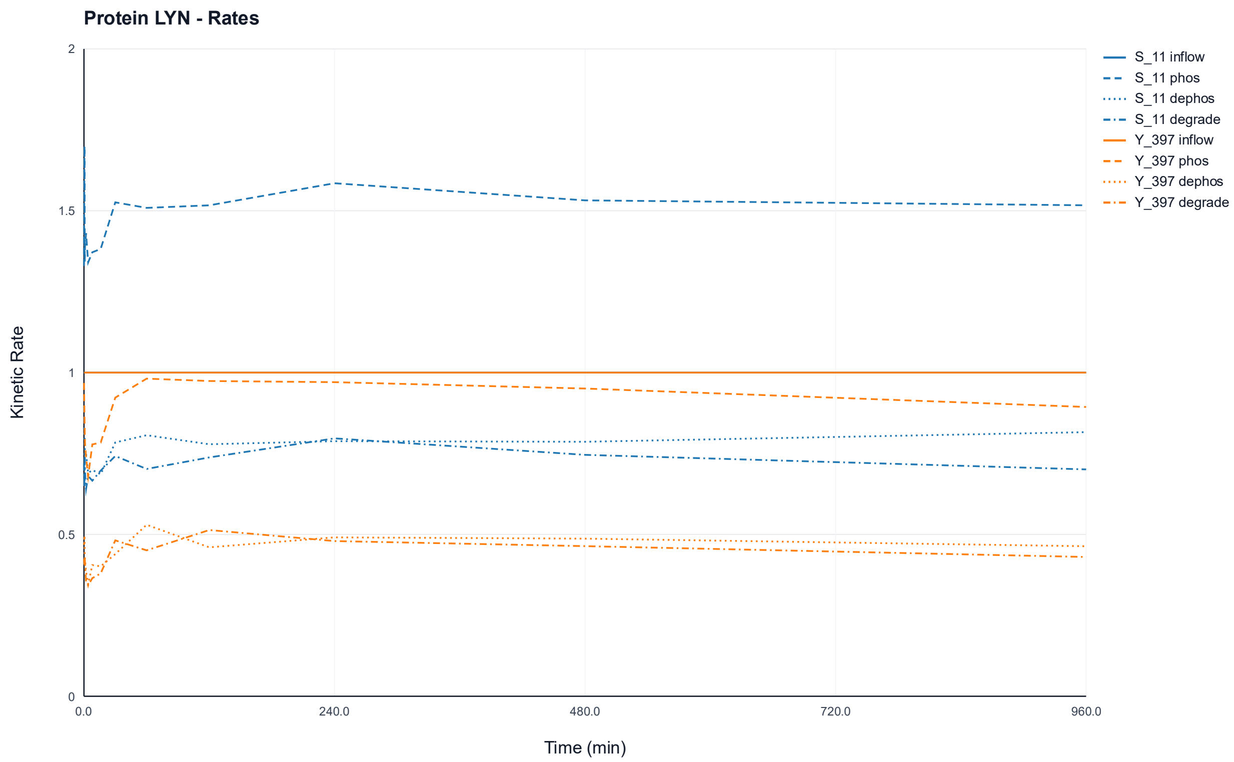

Figure 68: Time-resolved kinetic rate profiles of protein-wise estimation for the LYN protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites S_11 and Y_397.

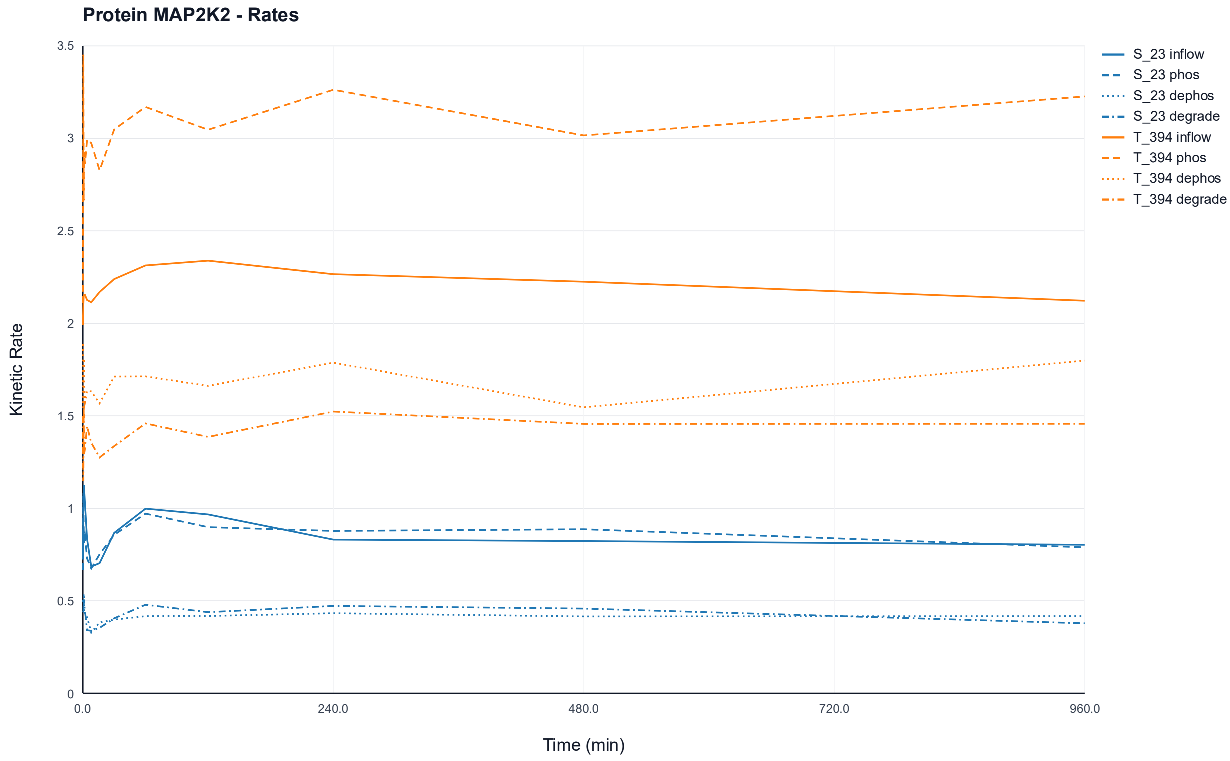

Figure 69: Time-resolved kinetic rate profiles of protein-wise estimation for the MAP2K2 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites S_23 and T_394.

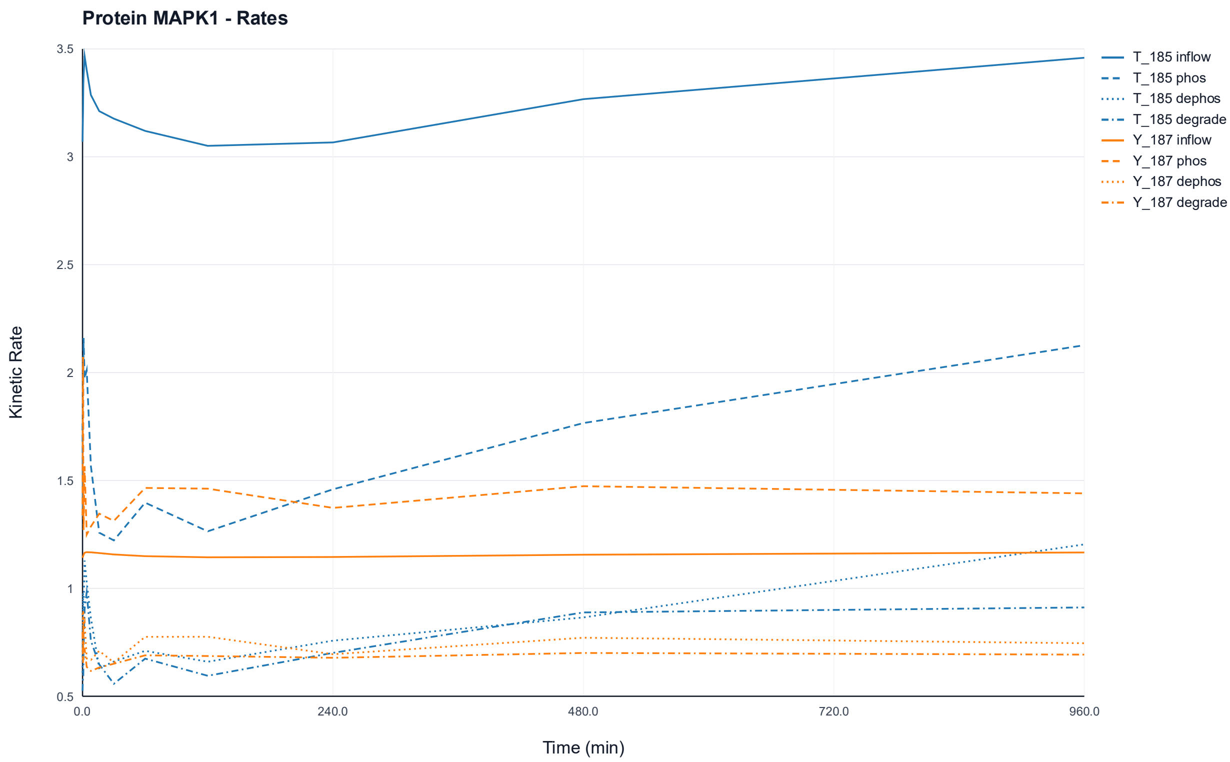

Figure 70: Time-resolved kinetic rate profiles of protein-wise estimation for the MAPK1 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites T_185 and Y_187.

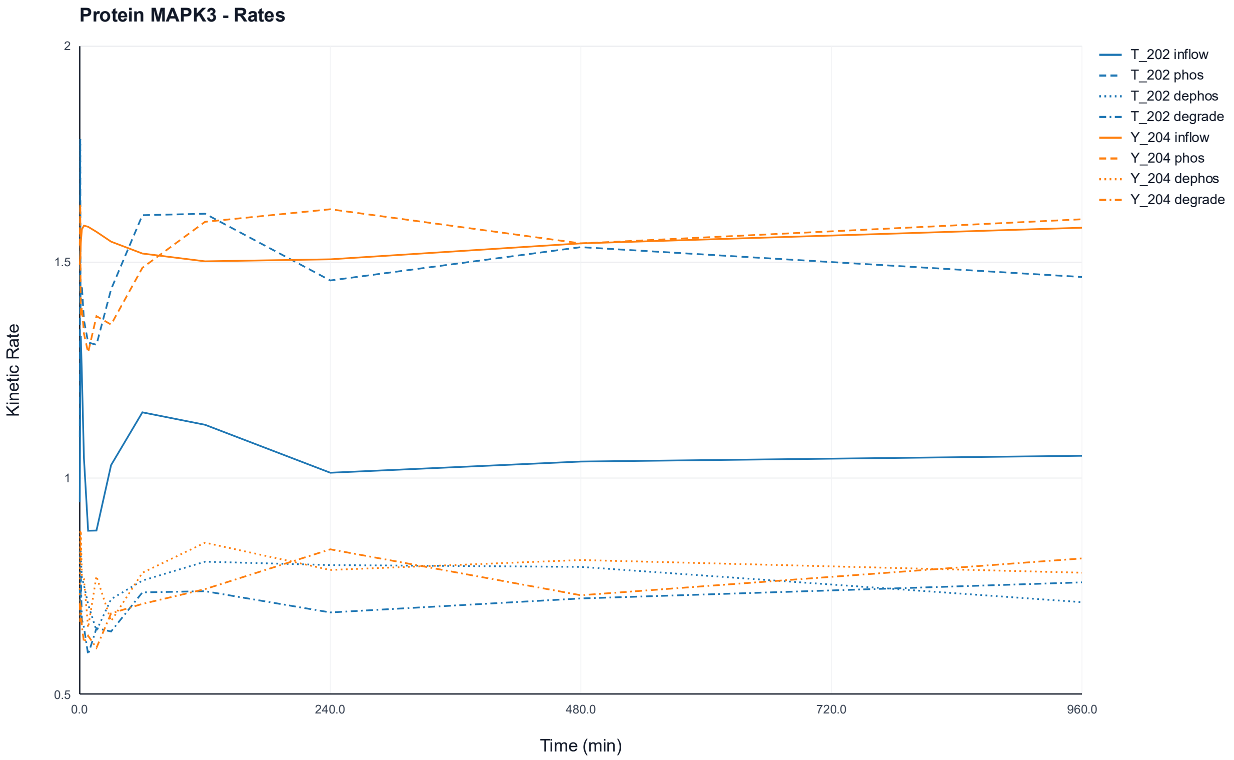

Figure 71: Time-resolved kinetic rate profiles of protein-wise estimation for the MAPK3 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites T_202 and Y_204.

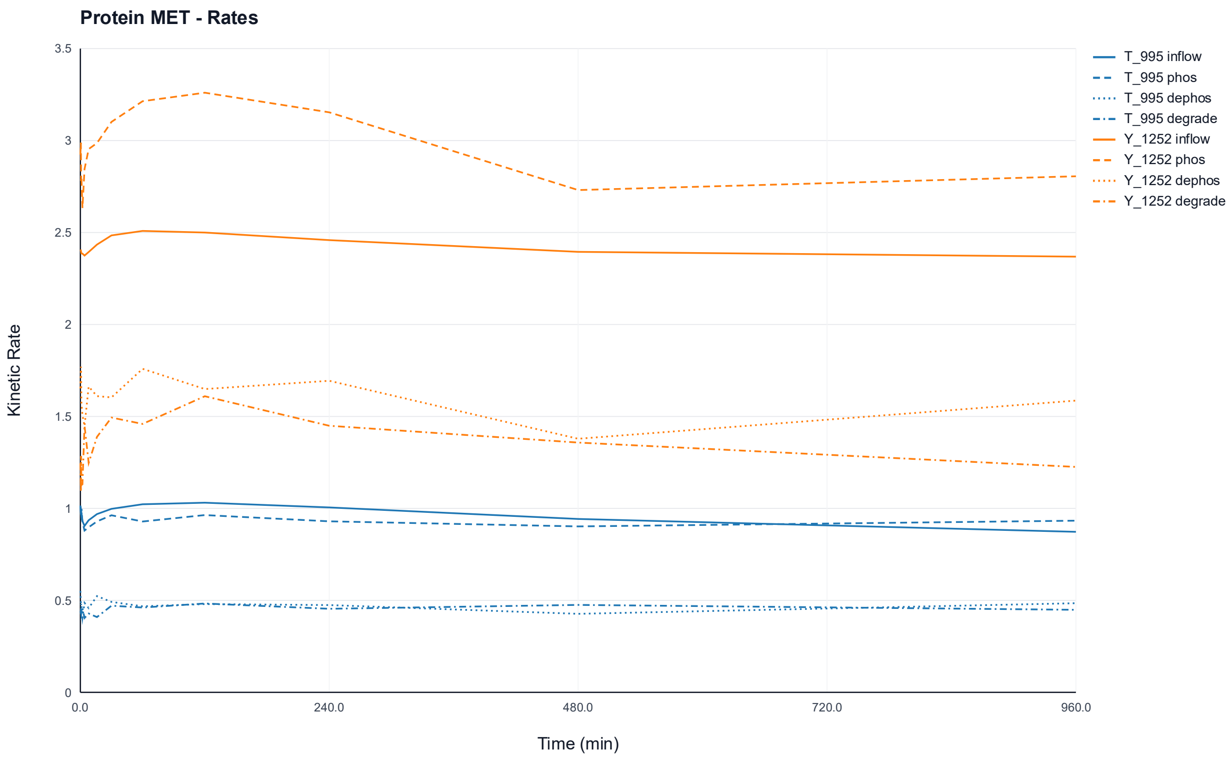

Figure 72: Time-resolved kinetic rate profiles of protein-wise estimation for the MET protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites T_995 and Y_1252.

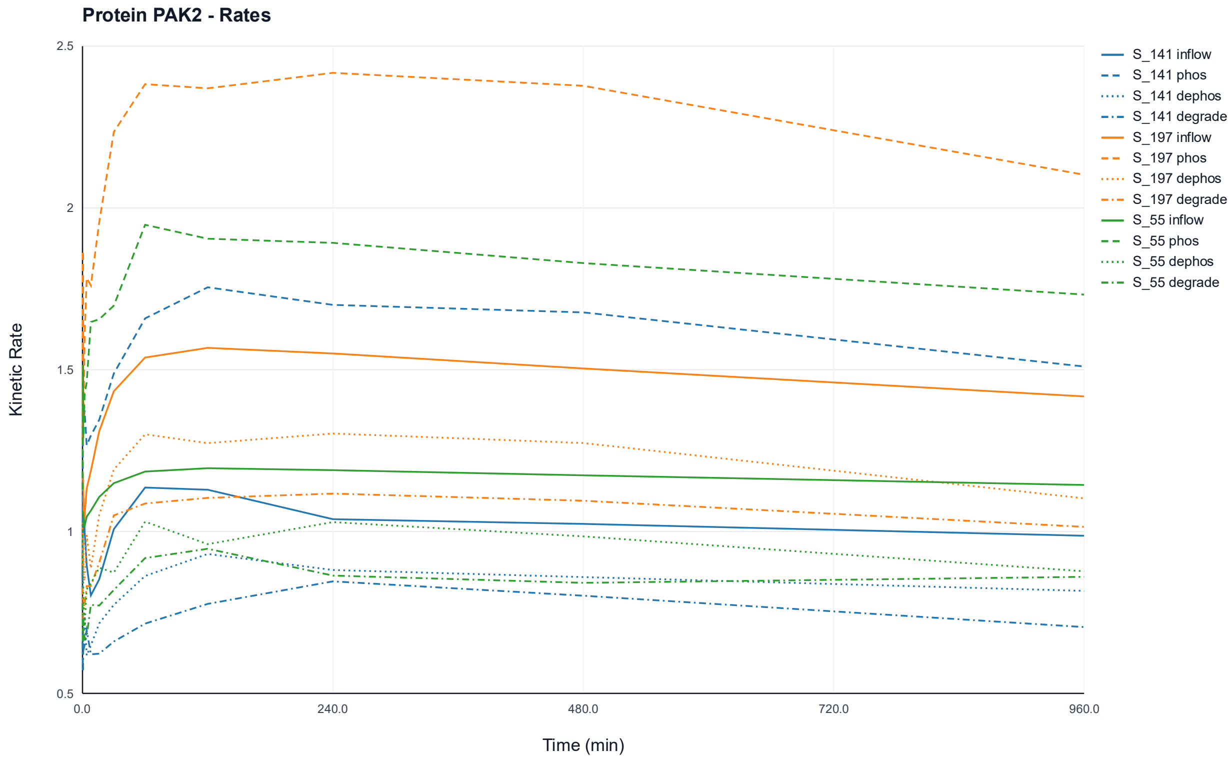

Figure 73: Time-resolved kinetic rate profiles of protein-wise estimation for the PAK2 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites S_141, S_197 and S_55.

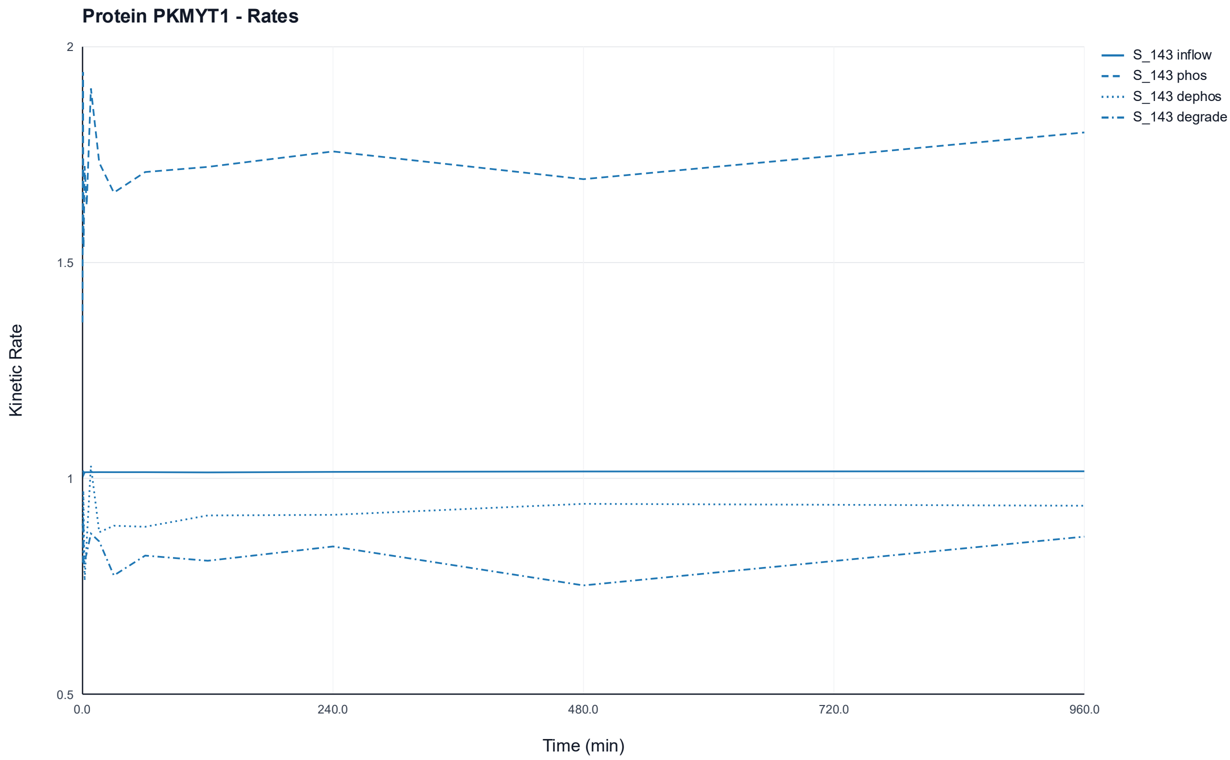

Figure 74: Time-resolved kinetic rate profiles of protein-wise estimation for the PKMYT1 protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at site S_143.

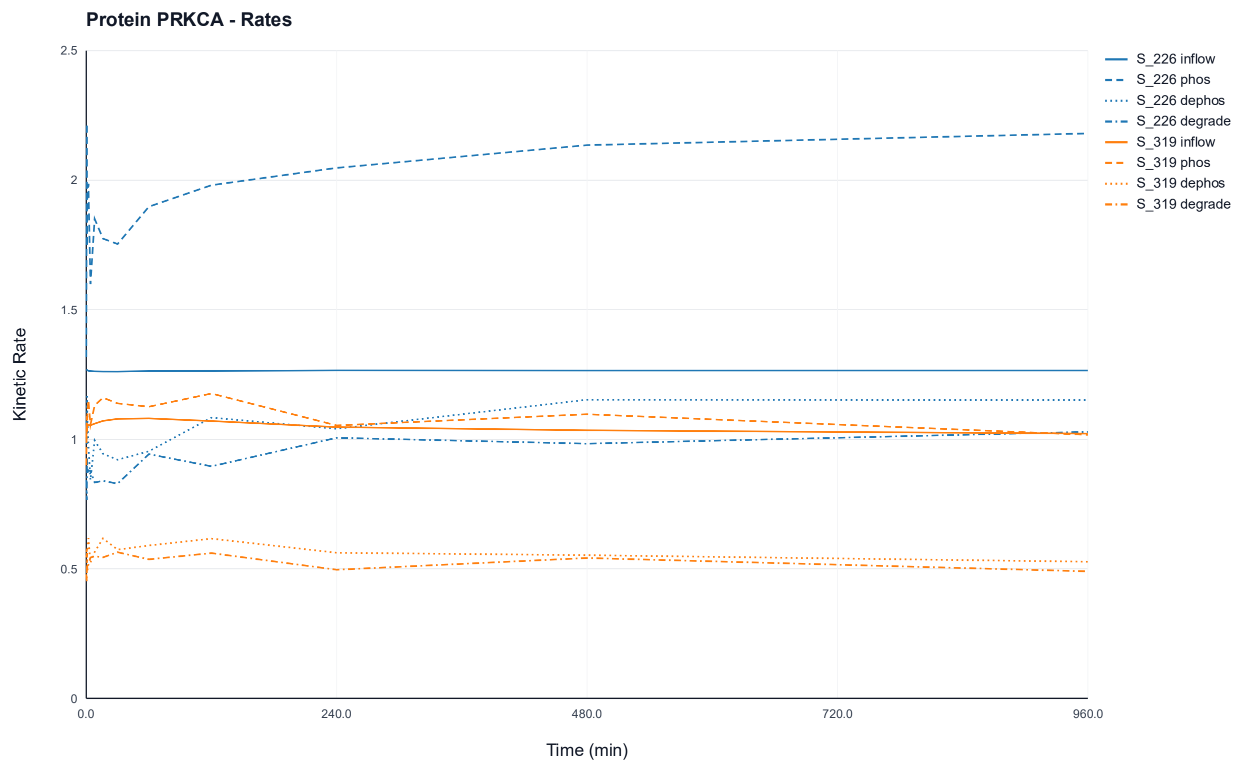

Figure 75: Time-resolved kinetic rate profiles of protein-wise estimation for the PRKCA protein. The activity rate (inflow), phosphorylation rate (phos), dephosphorylation rate (dephos), and degradation rate (degrade) are illustrated for the phosphorylated states at sites S_226 and S_319.

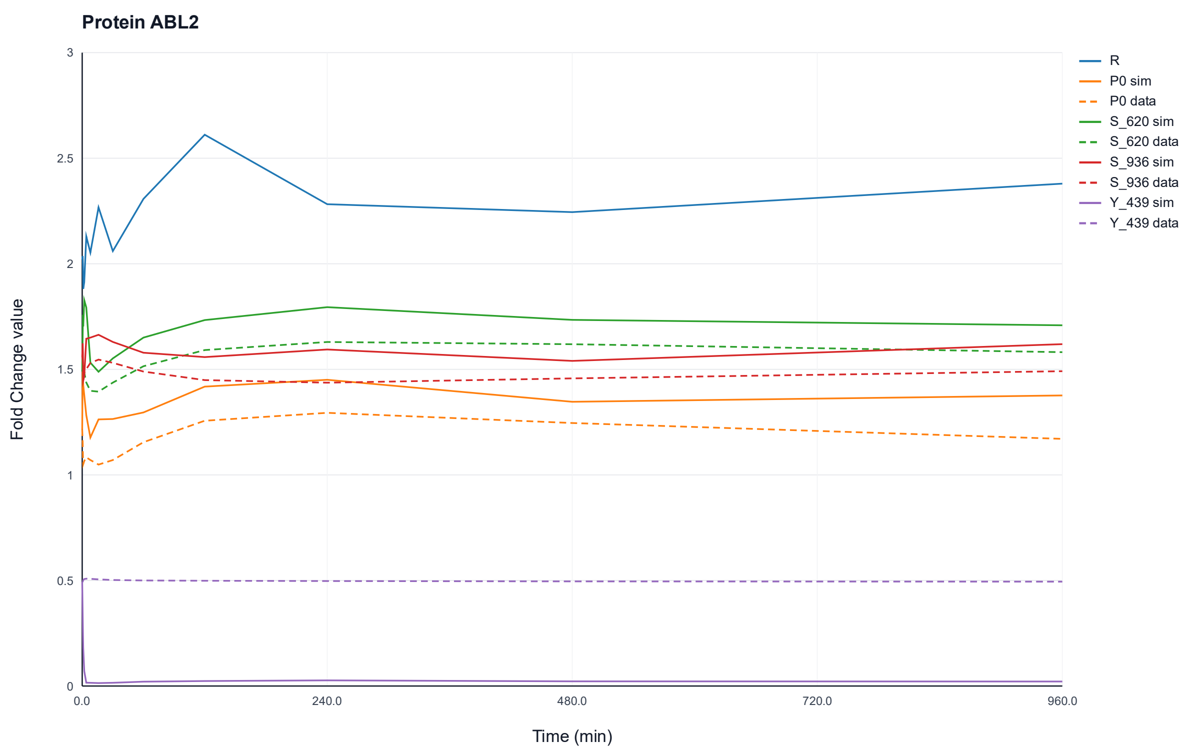

Figure 76: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein ABL2 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_620, S_936 and Y_439. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

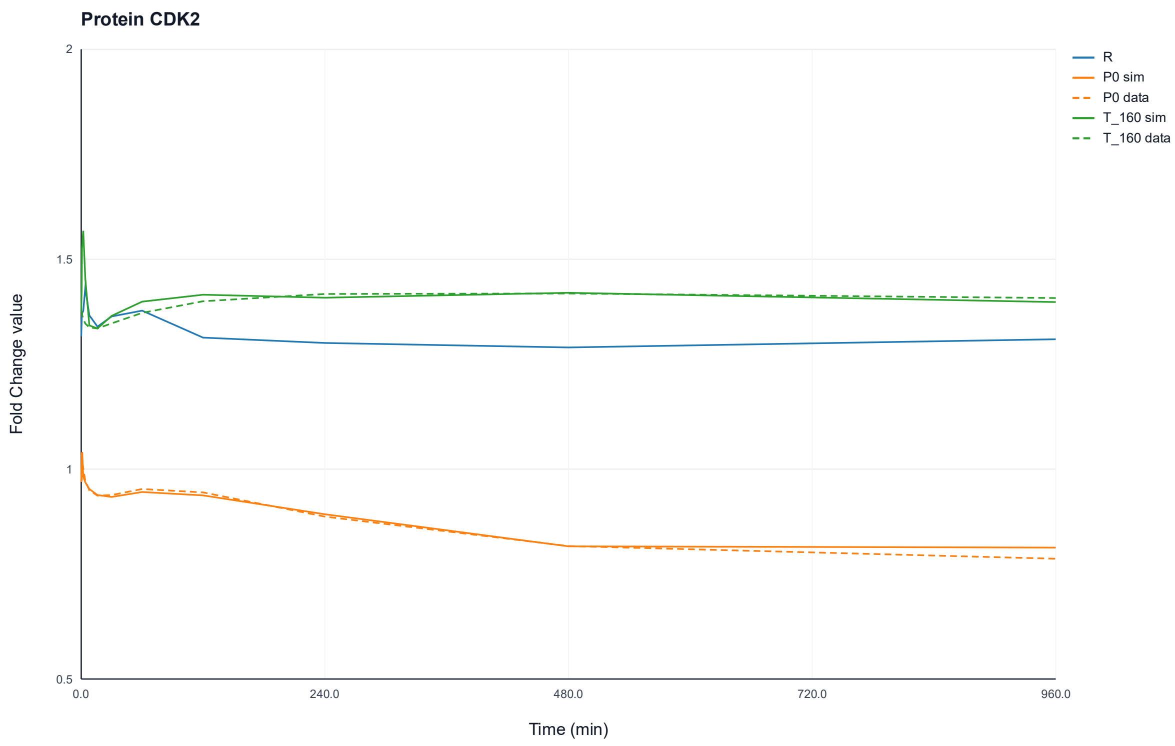

Figure 77: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein CDK2 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site T_160. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

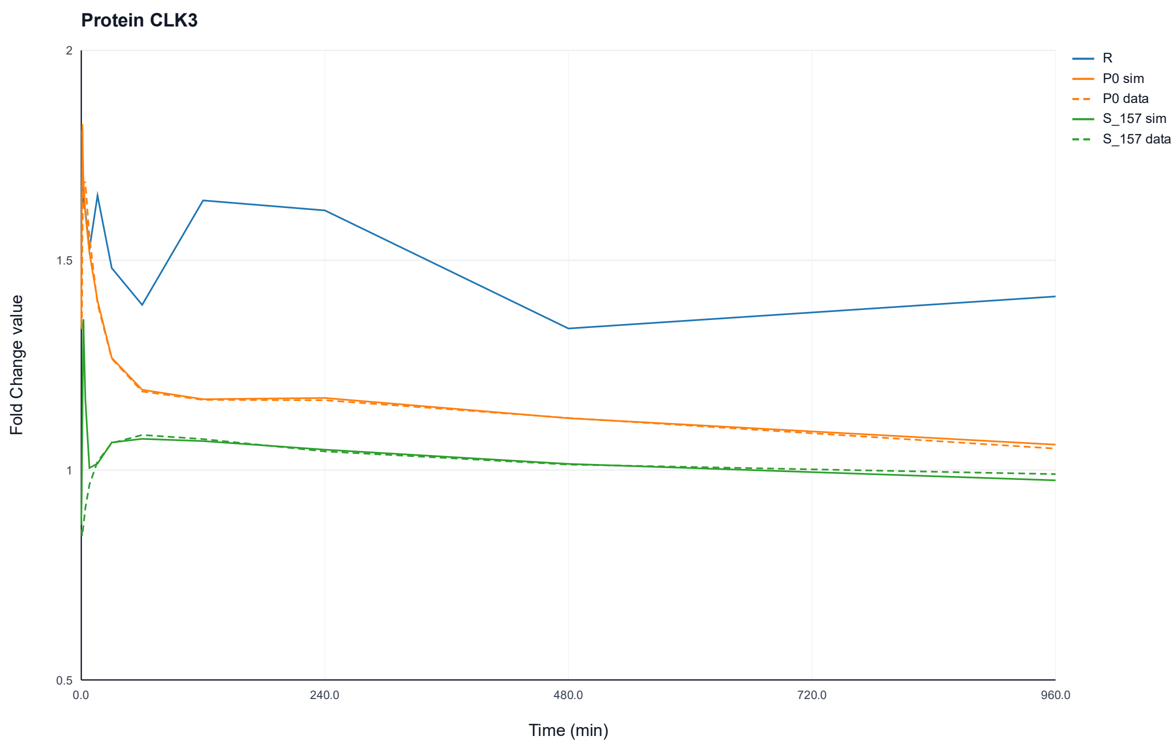

Figure 78: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein CLK3 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site S_157. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

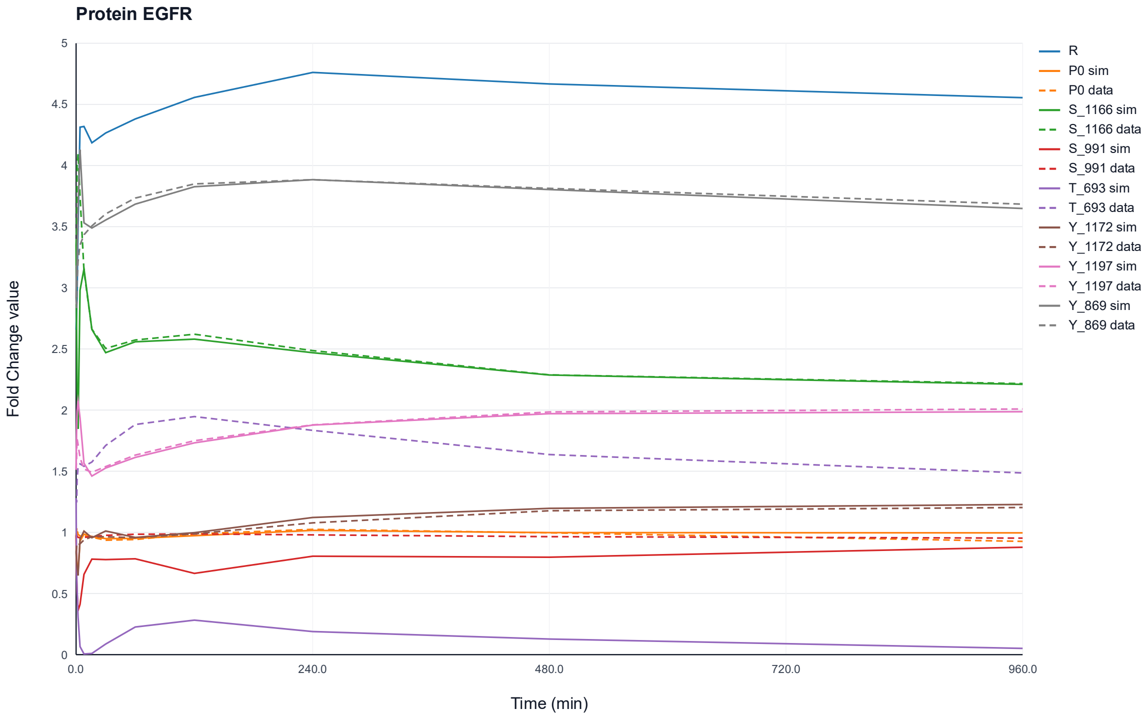

Figure 79: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein EGFR over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_1166, S_991, T_693, Y_1172, Y_1197 and Y_869. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

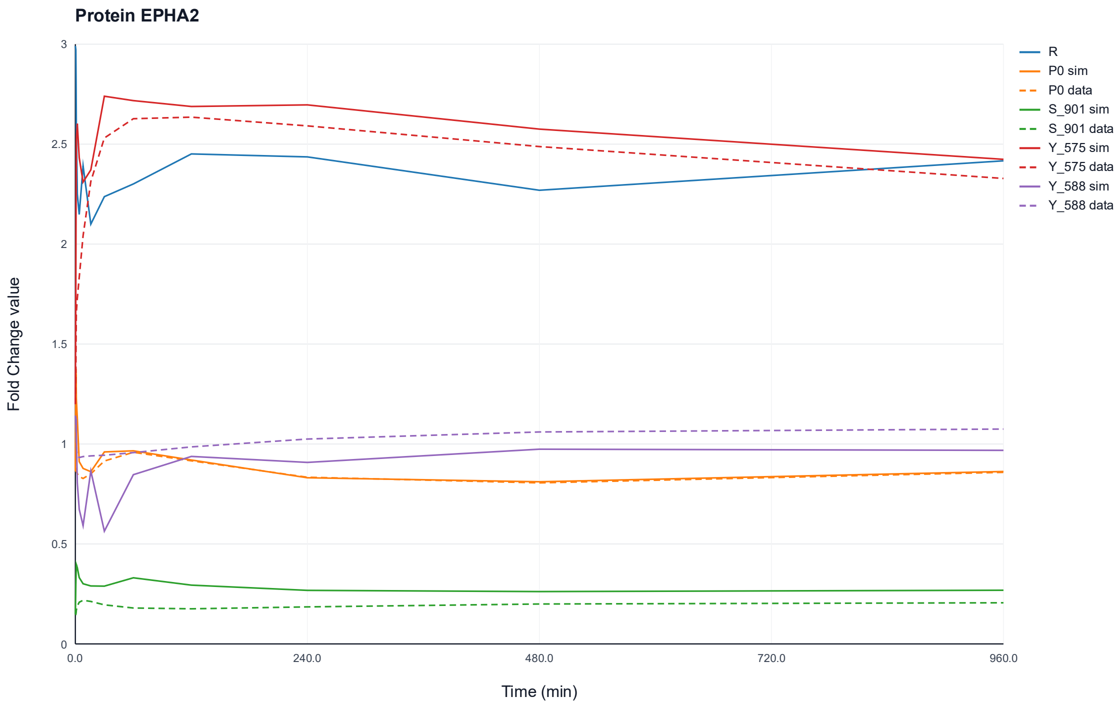

Figure 80: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein EPHA2 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_901, Y_575 and Y_588. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

Figure 81: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein ERBB2 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site T_1166. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

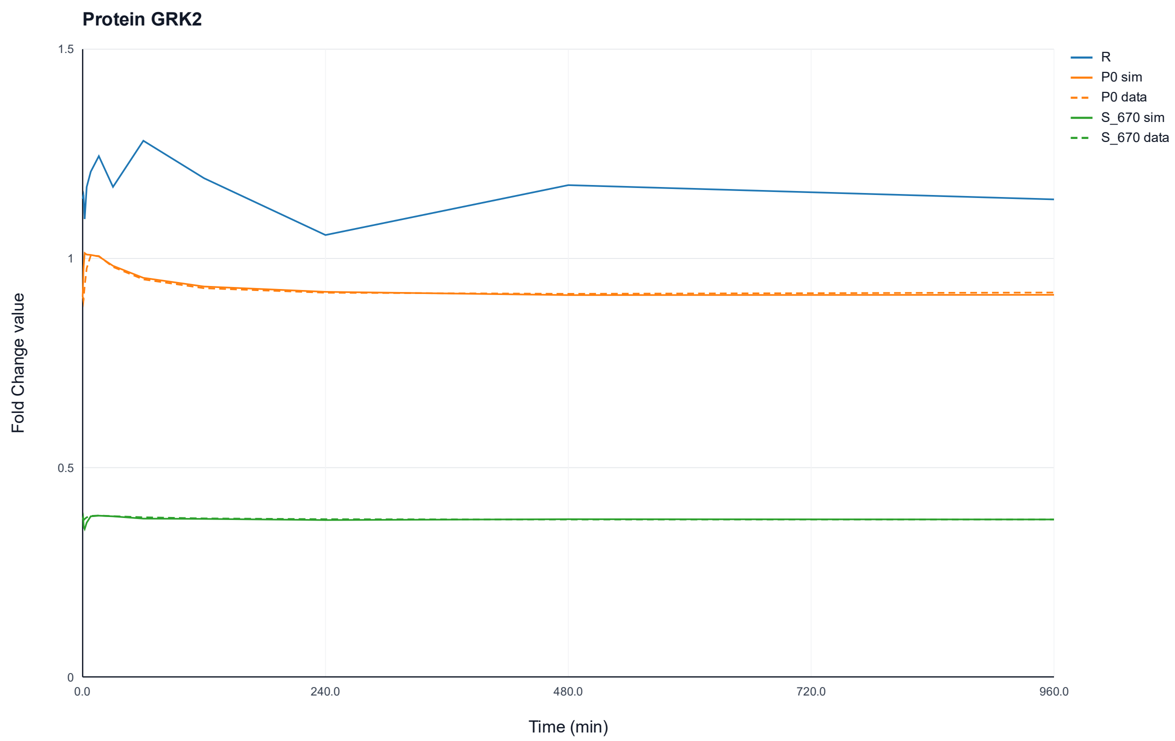

Figure 82: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein GRK2 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site S_670. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

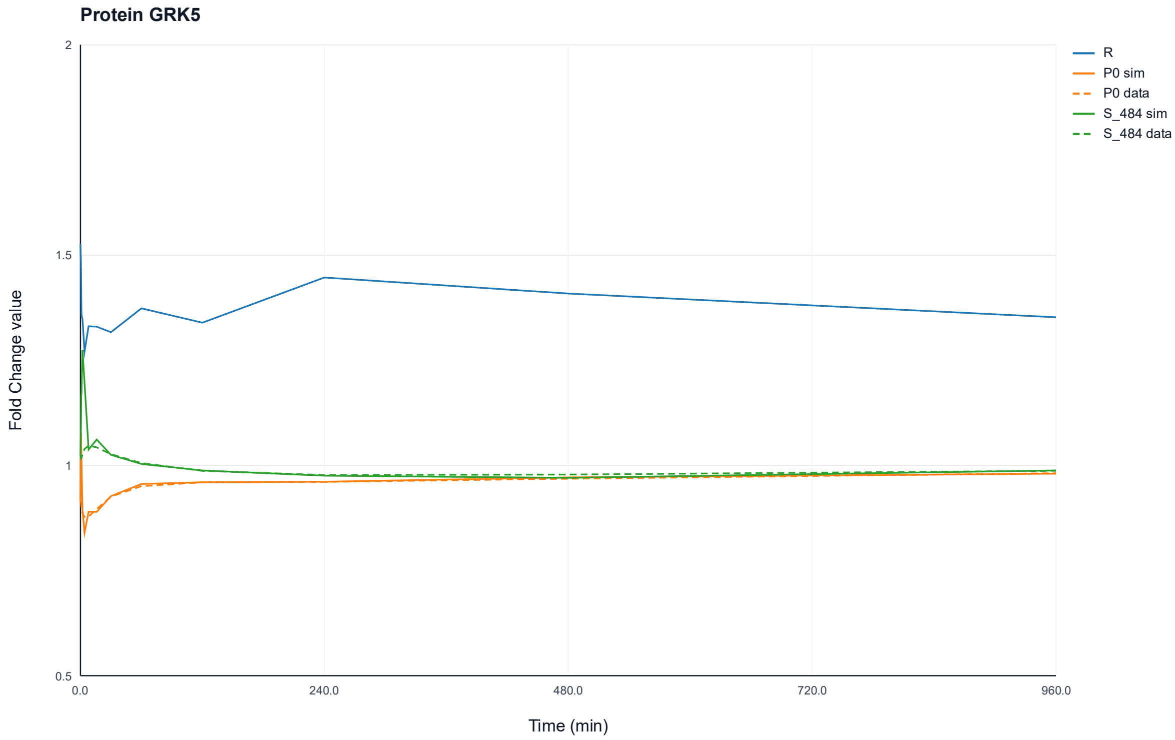

Figure 83: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein GRK5 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site S_484. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

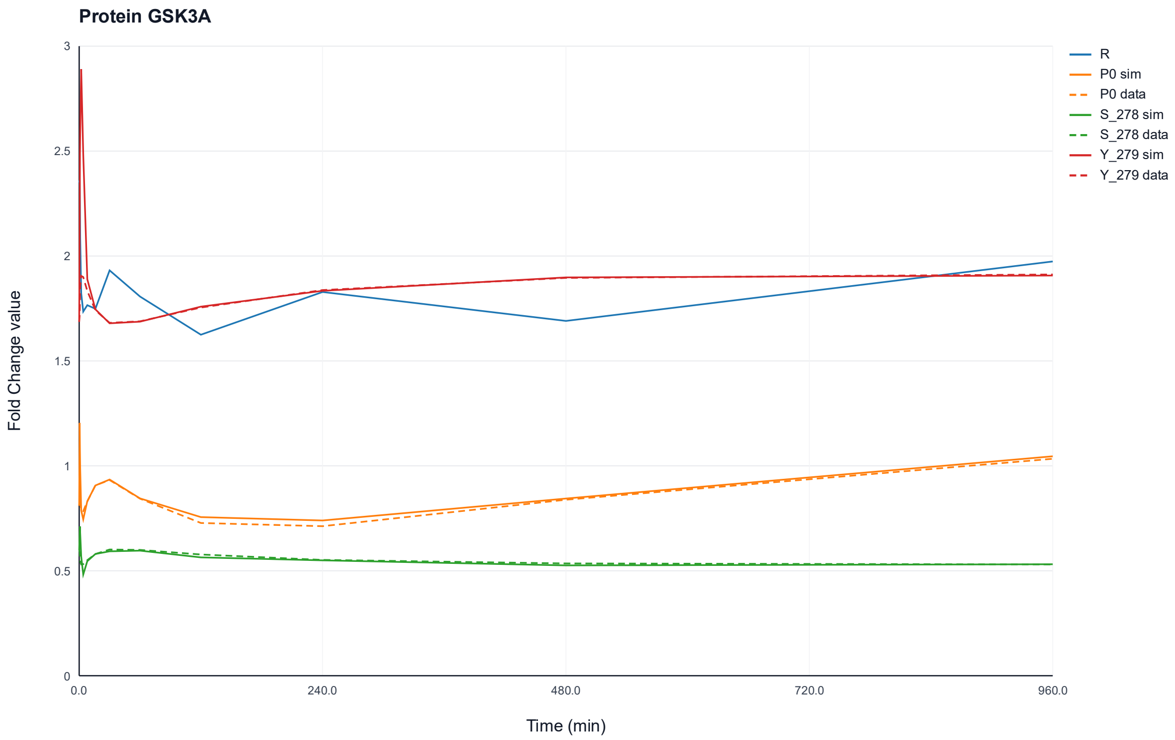

Figure 84: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein GSK3A over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_278 and Y_279. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

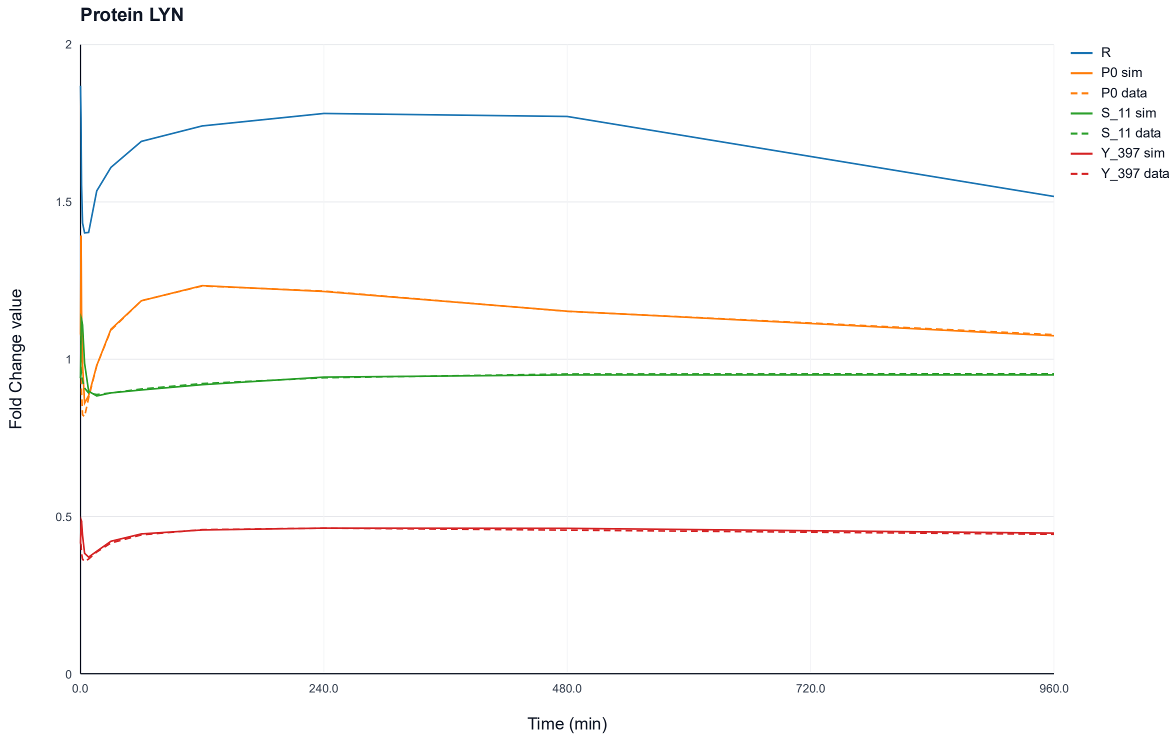

Figure 85: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein LYN over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_11 and Y_397. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

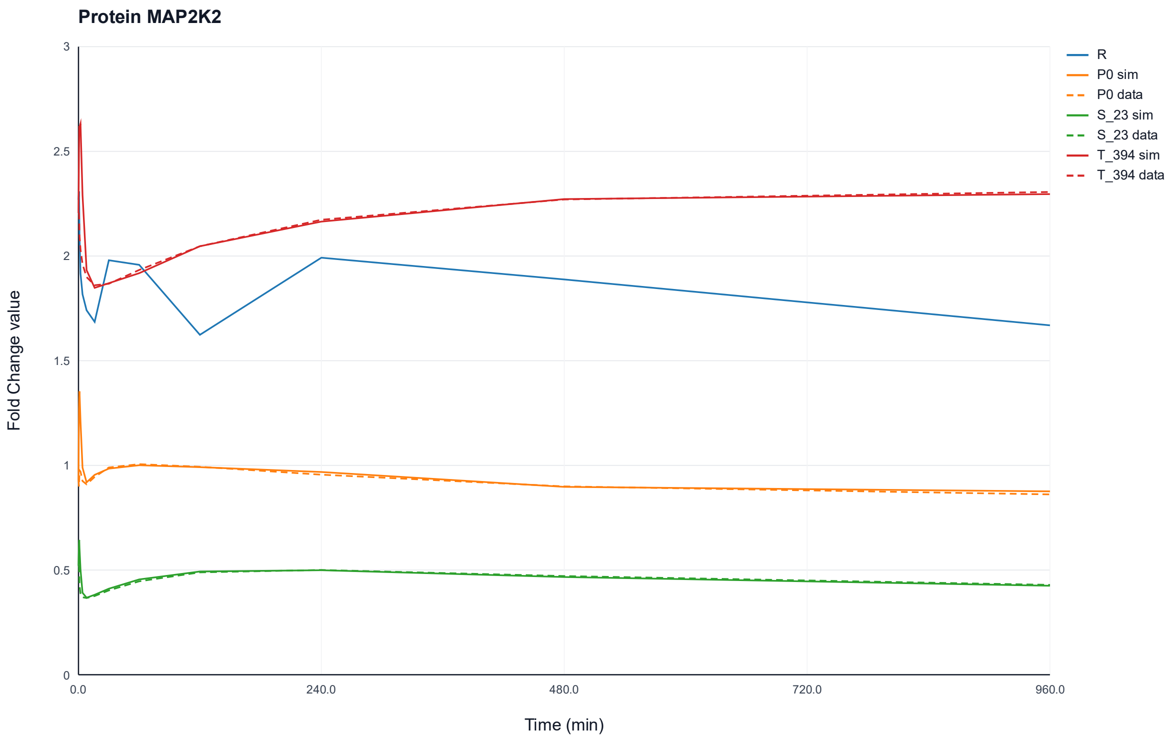

Figure 86: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein MAP2K2 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_23 and T_394. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

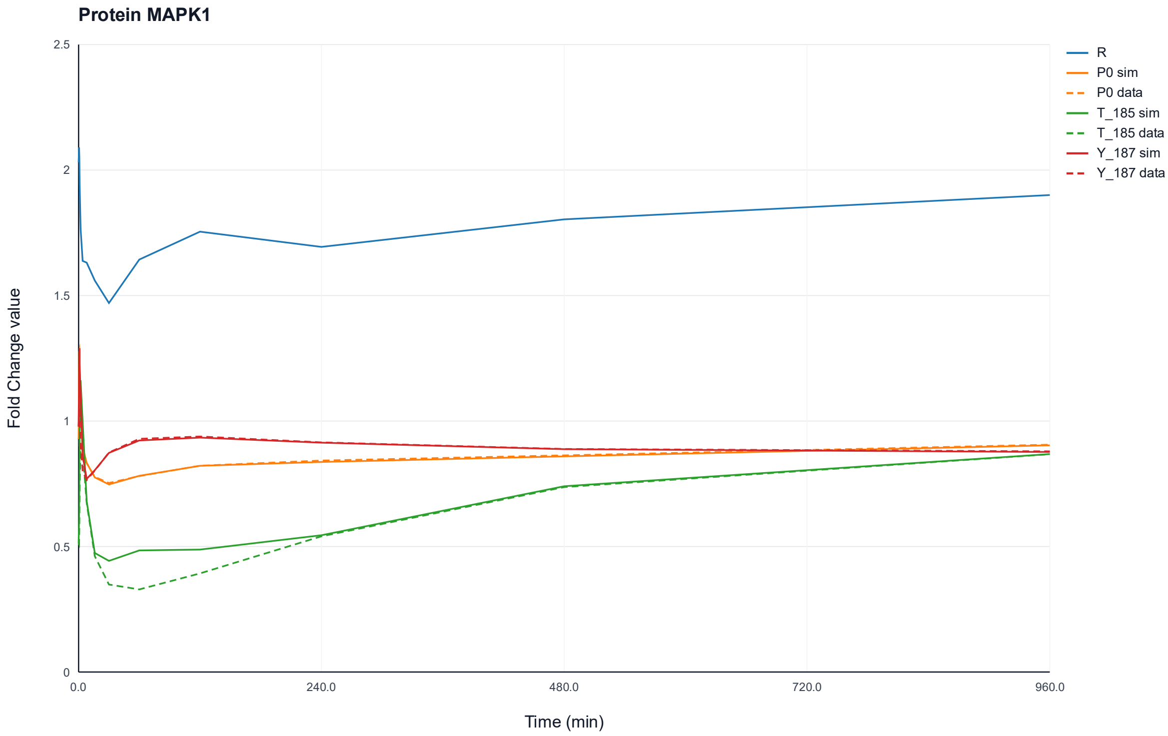

Figure 87: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein MAPK1 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites T_185 and Y_187. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

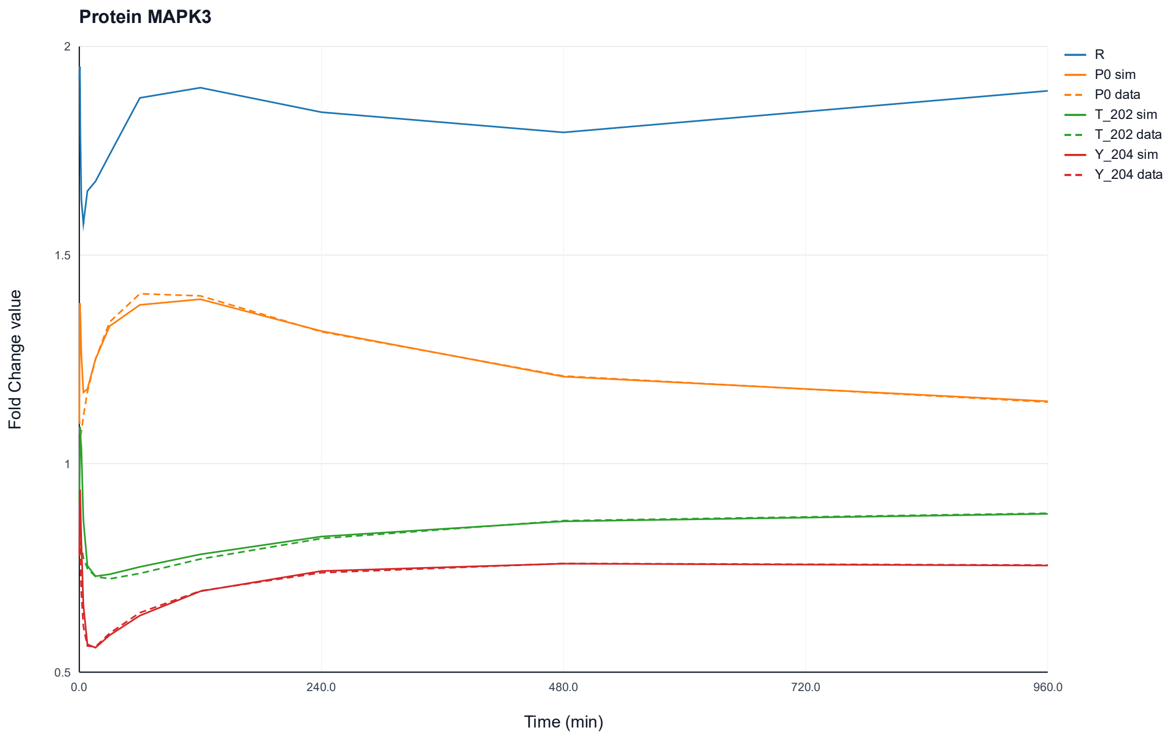

Figure 88: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein MAPK3 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites T_202 and Y_204. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

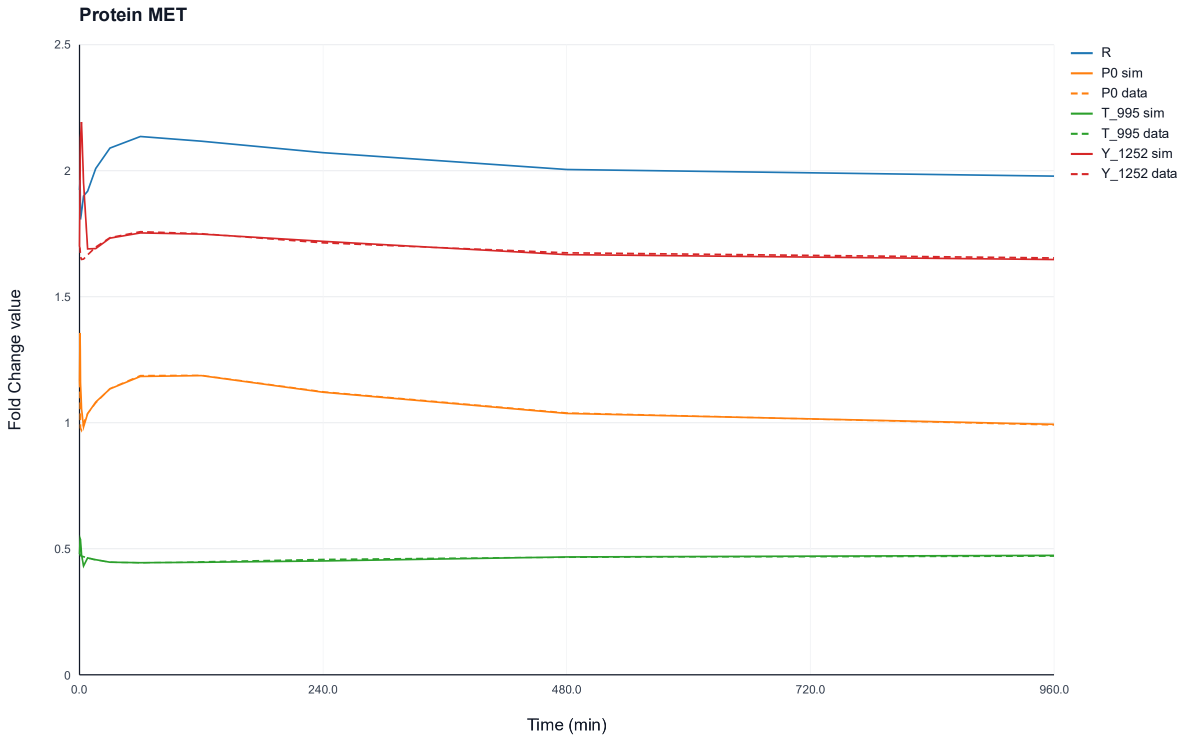

Figure 89: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein MET over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites T_995 and Y_1252. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

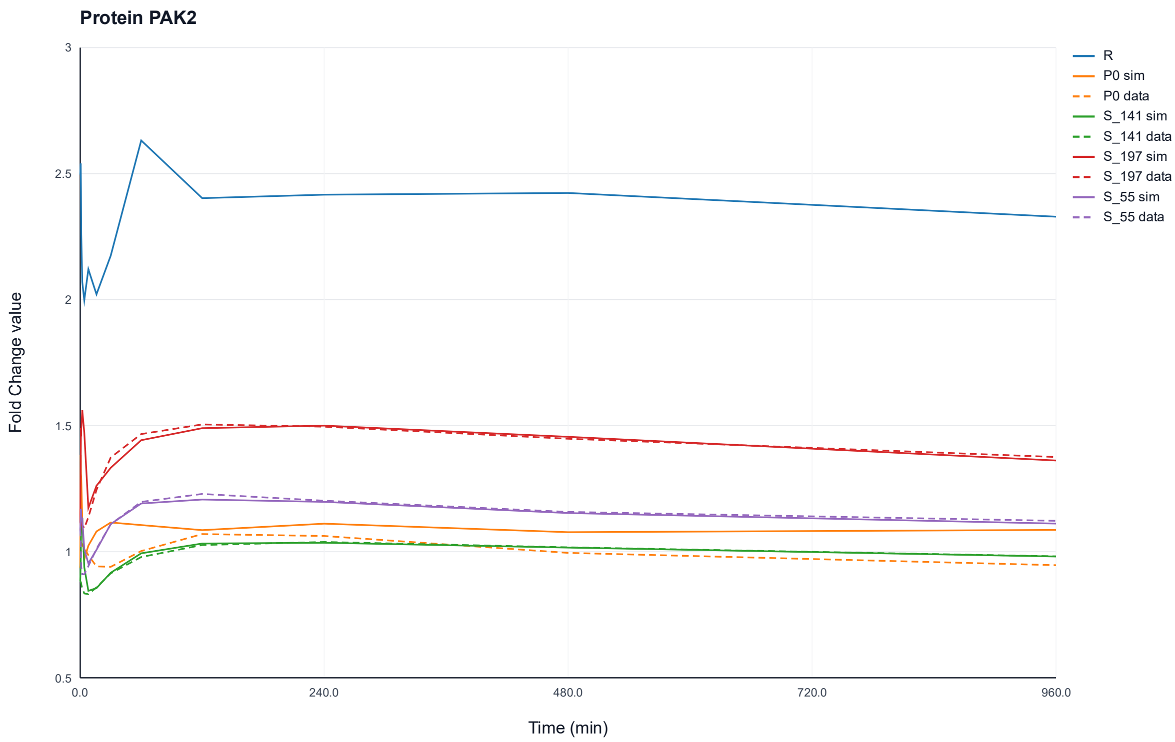

Figure 90: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein PAK2 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_141, S_197 and S_55. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

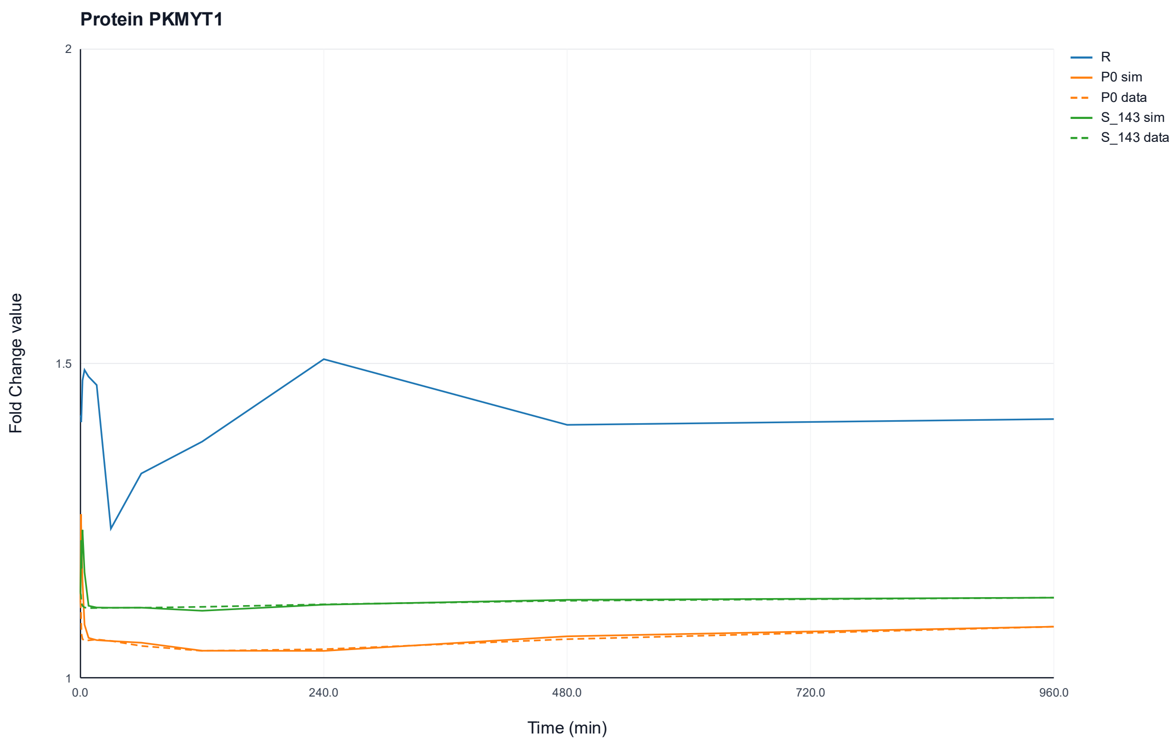

Figure 91: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein PKMYT1 over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at site S_143. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

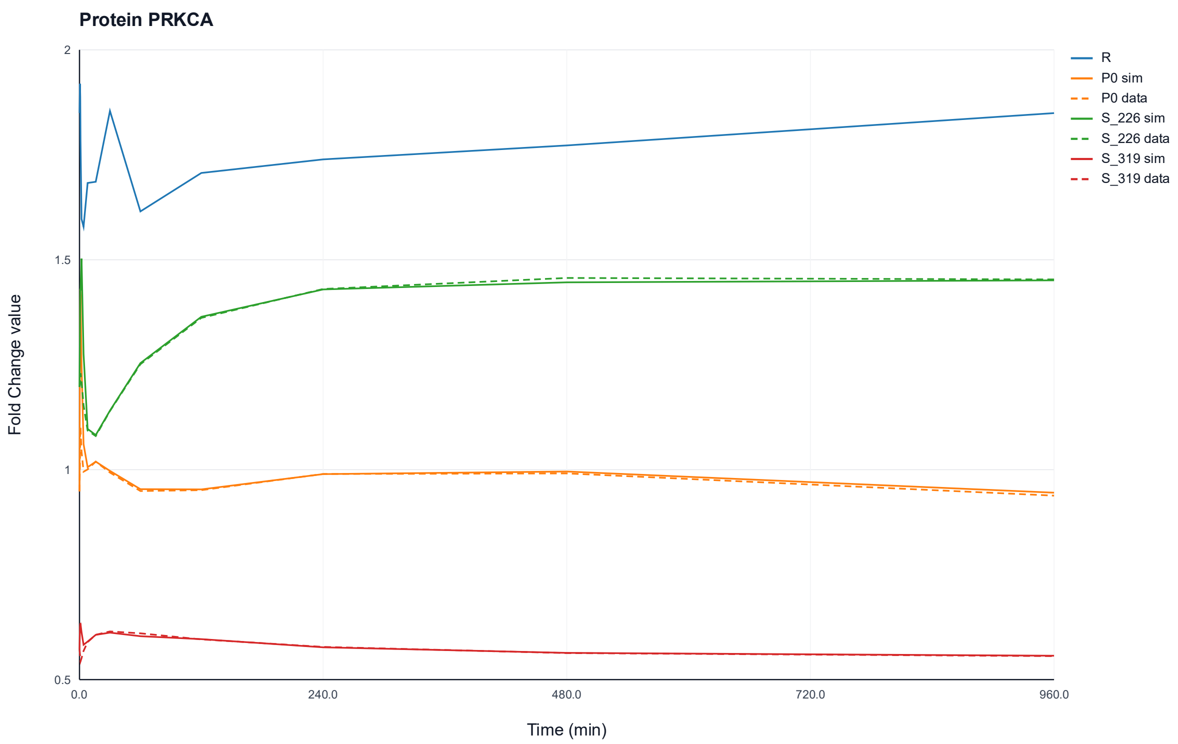

Figure 92: Comparison of experimentally observed fold change profiles and simulated profiles from the dependent ODE network model of protein PRKCA over time. The plot shows the ODE trajectories for the states of the mRNA R, the unphosphorylated protein P_0, and the phosphorylated protein at sites S_226 and S_319. Solid lines represent ODE model simulations, whereas dashed lines indicate the measured data.

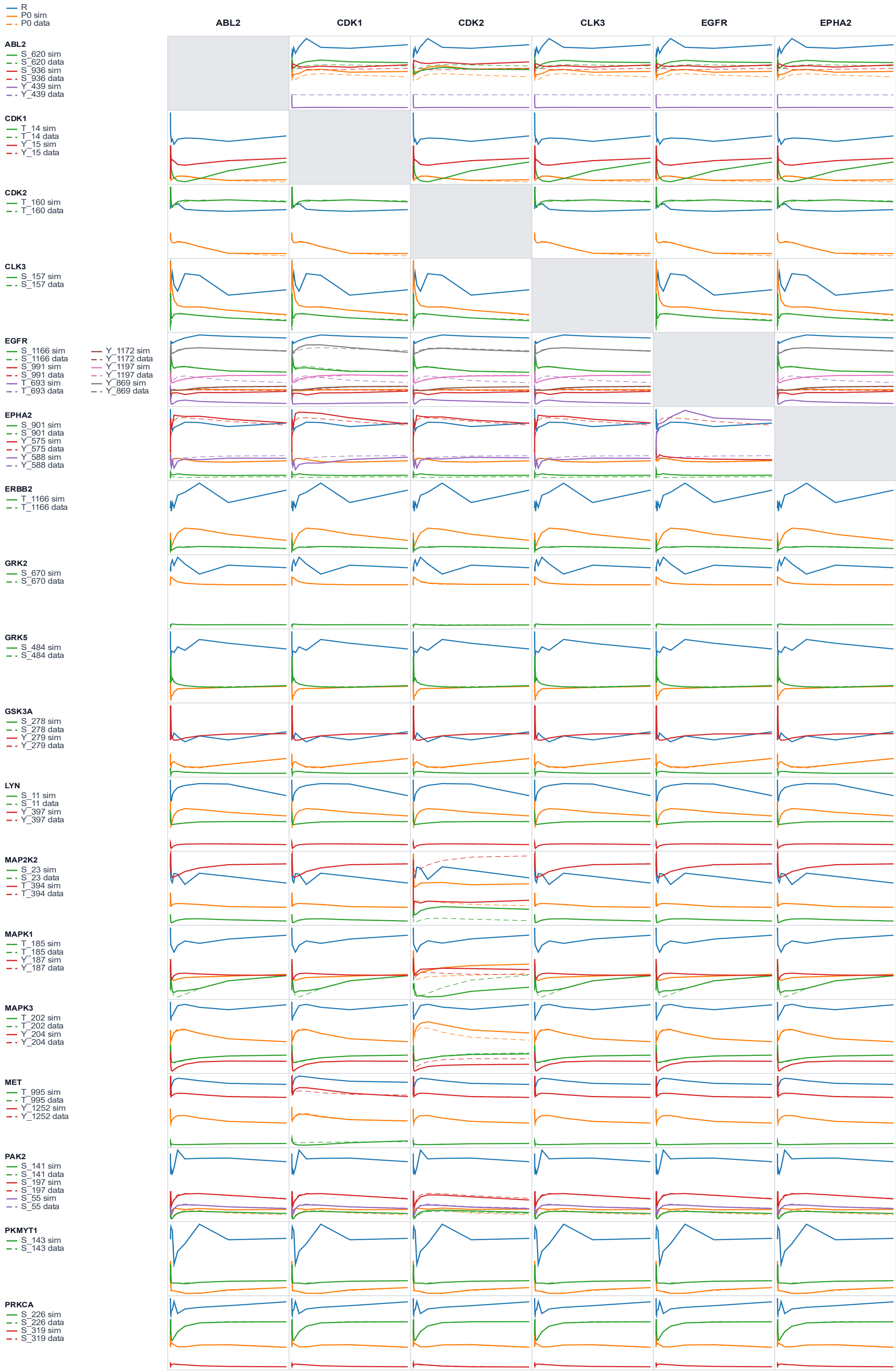

Figure 93: Matrix displaying knockout effects of kinase knockouts of ABL2, CDK1, CDK2, CLK3, EGFR, and EPHA2 on network fold change values. Columns represent the knocked-out kinases, while rows represent the fold change values of the potentially affected kinases in the network. Solid lines represent knockout model simulations, whereas dashed lines indicate the measured data.

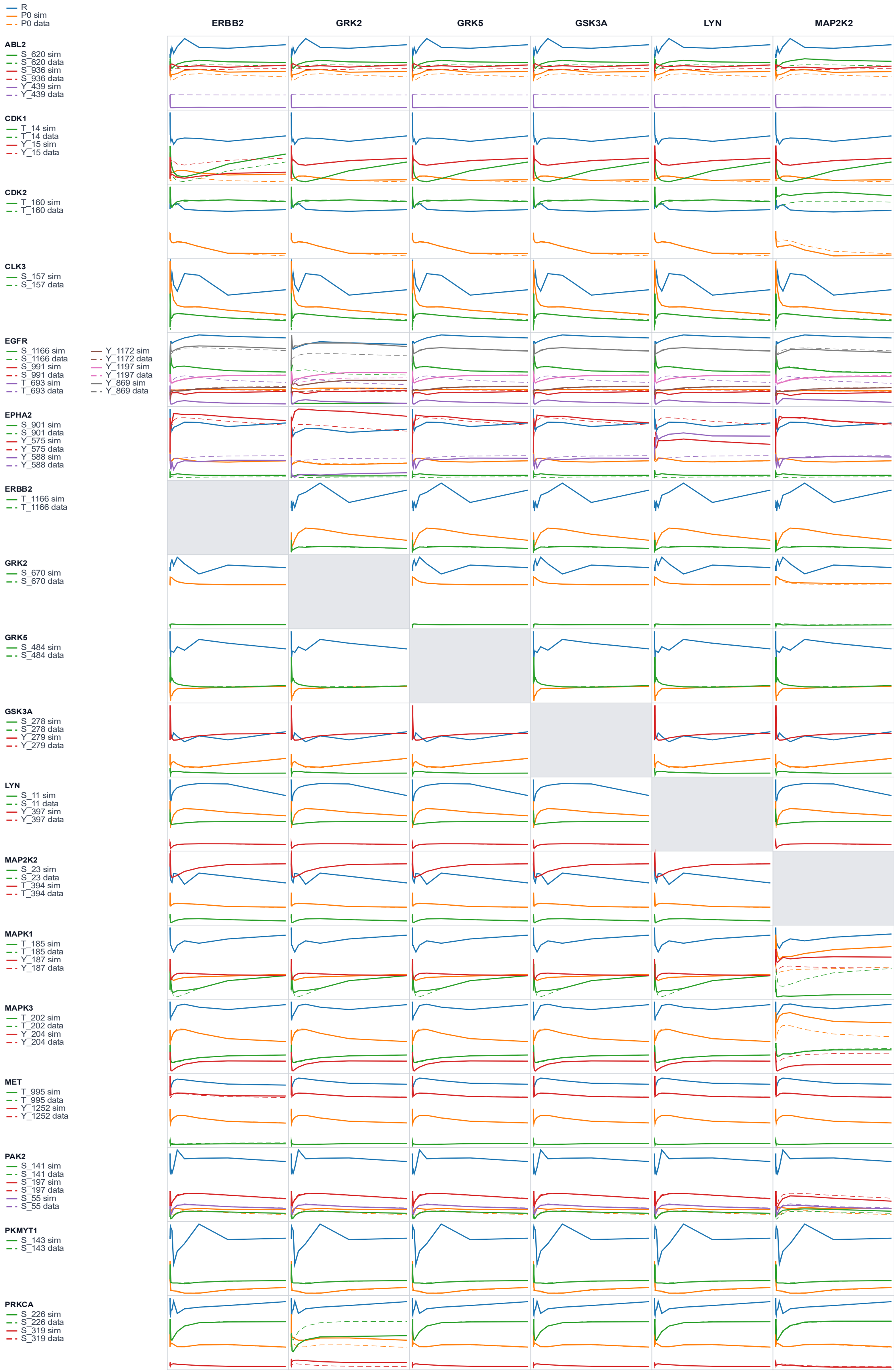

Figure 94: Matrix displaying knockout effects of kinase knockouts of ERBB2, GRK2, GRK5, GSK3A, LYN, and MAP2K2 on network fold change values. Columns represent the knocked-out kinases, while rows represent the fold change values of the potentially affected kinases in the network. Solid lines represent knockout model simulations, whereas dashed lines indicate the measured data.

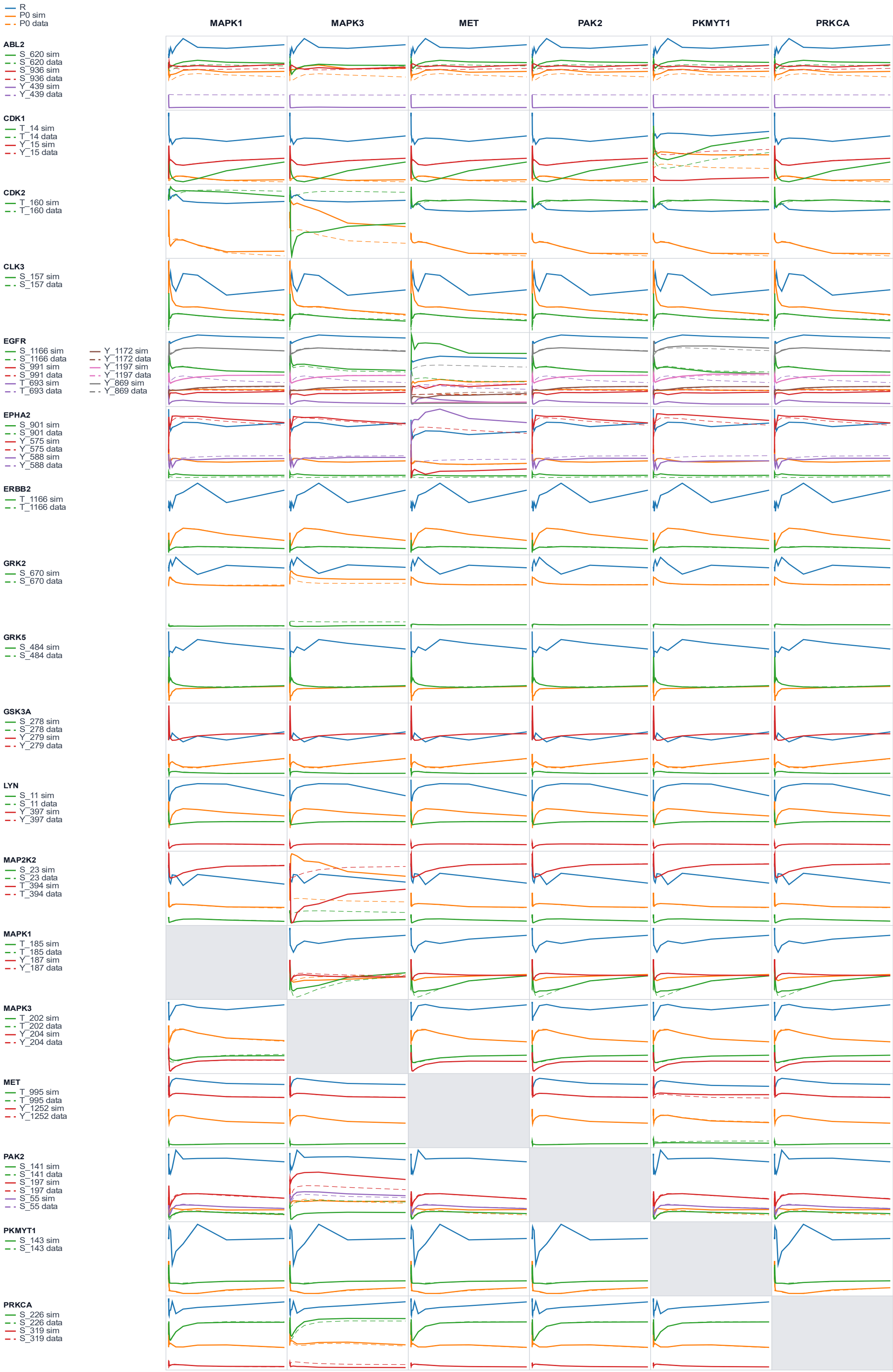

Figure 95: Matrix displaying knockout effects of kinase knockouts of MAPK1, MAPK3, MET, PAK2, PKMYT1, and PRKCA on network fold change values. Columns represent the knocked-out kinases, while rows represent the fold change values of the potentially affected kinases in the network. Solid lines represent knockout model simulations, whereas dashed lines indicate the measured data.

Technical Implementation

The scripts for the dependency parameter optimizer and the ODE network simulation were written in the Julia programming language version 1.12.5 [31]. The code is available in the following GitHub repository: https://github.com/luisfirl/SignalingProject/. The packages utilized in the study are listed below.

Package Name	Version	Reference
CSV.jl	0.10.16	CSV.jl: Utility library for working with CSV and other delimited files in the Julia programming language [32]
DataFrames.jl	1.8.1	DataFrames.jl: Flexible and Fast Tabular Data in Julia [33]
Flux.jl	0.16.9	Fashionable Modelling with Flux [29]
Ipopt.jl	1.14.1	On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming [38]
JuMP.jl	1.30.0	JuMP 1.0: recent improvements to a modeling language for mathematical optimization [37]
MathOptInterface.jl	1.50.1	MathOptInterface: A Data Structure for Mathematical Optimization Problems [39]
Measures.jl	0.3.3	Measures.jl: Unified measure and coordinates types [49]
OrdinaryDiffEq.jl	6.105.0	DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia [44]
Plots.jl	1.41.6	Plots.jl – A User Extendable Plotting API for the Julia Programming Language [50]
Zygote.jl	0.7.10	Don't Unroll Adjoint: Differentiating SSA-Form Programs [30]

Utilized tools based on Artificial Intelligence:

- ChatGPT version 5.3/5.4, OpenAI
    - Formatting thesis structure
    - Definition of terms
    - Literature research
- Codex version 5.4, OpenAI
    - Generating and debugging Julia Code
- DeepL Translate, DeepL SE
    - Synonym research

References

[1] U. Alon, *An introduction to systems biology: design principles of biological circuits*, Second edition. Boca Raton: CRC Press, Taylor & Francis Group, 2020.

[2] K. A. Zielińska and V. L. Katanaev, 'Information Theory: New Look at Oncogenic Signaling Pathways', *Trends in Cell Biology*, vol. 29, no. 11, pp. 862–875, Nov. 2019, doi: 10.1016/j.tcb.2019.08.005.

[3] P. Wee and Z. Wang, 'Epidermal Growth Factor Receptor Cell Proliferation Signaling Pathways', *Cancers*, vol. 9, no. 5, p. 52, May 2017, doi: 10.3390/cancers9050052.

[4] E. S. Welf and J. M. Haugh, 'Signaling pathways that control cell migration: models and analysis', *WIREs Mechanisms of Disease*, vol. 3, no. 2, pp. 231–240, Mar. 2011, doi: 10.1002/wsbm.110.

[5] N. M. Novikov, S. Y. Zolotaryova, A. M. Gautreau, and E. V. Denisov, 'Mutational drivers of cancer cell migration and invasion', *Br J Cancer*, vol. 124, no. 1, pp. 102–114, Jan. 2021, doi: 10.1038/s41416-020-01149-0.

[6] F. G. Giancotti, 'Deregulation of cell signaling in cancer', *FEBS Letters*, vol. 588, no. 16, pp. 2558–2570, Aug. 2014, doi: 10.1016/j.febslet.2014.02.005.

[7] R. Sever and J. S. Brugge, 'Signal Transduction in Cancer', *Cold Spring Harbor Perspectives in Medicine*, vol. 5, no. 4, pp. a006098–a006098, Apr. 2015, doi: 10.1101/cshperspect.a006098.

[8] L. K. Nguyen and B. N. Kholodenko, 'Feedback regulation in cell signalling: Lessons for cancer therapeutics', *Seminars in Cell & Developmental Biology*, vol. 50, pp. 85–94, Feb. 2016, doi: 10.1016/j.semcdb.2015.09.024.

[9] C. Sun and R. Bernards, 'Feedback and redundancy in receptor tyrosine kinase signaling: relevance to cancer therapies', *Trends in Biochemical Sciences*, vol. 39, no. 10, pp. 465–474, Oct. 2014, doi: 10.1016/j.tibs.2014.08.010.

[10] J. A. Ubersax and J. E. Ferrell Jr, 'Mechanisms of specificity in protein phosphorylation', *Nat Rev Mol Cell Biol*, vol. 8, no. 7, pp. 530–541, Jul. 2007, doi: 10.1038/nrm2203.

[11] L. N. Johnson, 'The regulation of protein phosphorylation', *Biochemical Society Transactions*, vol. 37, no. 4, pp. 627–641, Aug. 2009, doi: 10.1042/BST0370627.

[12] M. Köhn, 'Turn and Face the Strange: A New View on Phosphatases', *ACS Cent. Sci.*, vol. 6, no. 4, pp. 467–477, Apr. 2020, doi: 10.1021/acscentsci.9b00909.

[13] F. Ardito, M. Giuliani, D. Perrone, G. Troiano, and L. L. Muzio, 'The crucial role of protein phosphorylation in cell signaling and its use as targeted therapy (Review)', *International Journal of Molecular Medicine*, vol. 40, no. 2, pp. 271–280, Aug. 2017, doi: 10.3892/ijmm.2017.3036.

[14] H. Nishi, A. Shaytan, and A. R. Panchenko, 'Physicochemical mechanisms of protein regulation by phosphorylation', *Front. Genet.*, vol. 5, Aug. 2014, doi: 10.3389/fgene.2014.00270.

[15] P. P. Roux and J. Blenis, 'ERK and p38 MAPK-Activated Protein Kinases: a Family of Protein Kinases with Diverse Biological Functions', *Microbiol Mol Biol Rev*, vol. 68, no. 2, pp. 320–344, Jun. 2004, doi: 10.1128/MMBR.68.2.320-344.2004.

[16] B. N. Kholodenko, 'Spatially distributed cell signalling', *FEBS Letters*, vol. 583, no. 24, pp. 4006–4012, Dec. 2009, doi: 10.1016/j.febslet.2009.09.045.

[17] B. N. Kholodenko and M. R. Birtwistle, 'Four-dimensional dynamics of MAPK information-processing systems', *WIREs Mechanisms of Disease*, vol. 1, no. 1, pp. 28–44, Jul. 2009, doi: 10.1002/wsbm.16.

[18] B. N. Kholodenko, 'Cell-signalling dynamics in time and space', *Nat Rev Mol Cell Biol*, vol. 7, no. 3, pp. 165–176, Mar. 2006, doi: 10.1038/nrm1838.

[19] J. E. Ferrell and S. H. Ha, 'Ultrasensitivity part III: cascades, bistable switches, and oscillators', *Trends in Biochemical Sciences*, vol. 39, no. 12, pp. 612–618, Dec. 2014, doi: 10.1016/j.tibs.2014.10.002.

[20] M. Engel, J. Longden, J. Ferkinghoff-Borg, X. Robin, G. Saginc, and R. Linding, 'Bowhead: Bayesian modelling of cell velocity during concerted cell migration', *PLoS Comput Biol*, vol. 14, no. 1, p. e1005900, Jan. 2018, doi: 10.1371/journal.pcbi.1005900.

[21] K. Subik *et al.*, ‘The Expression Patterns of ER, PR, HER2, CK5/6, EGFR, Ki-67 and AR by Immunohistochemical Analysis in Breast Cancer Cell Lines’, *Breast Cancer (Auckl)*, vol. 4, p. 117822341000400, Jan. 2010, doi: 10.1177/117822341000400004.

[22] A. Patsialou *et al.*, ‘Intravital multiphoton imaging reveals multicellular streaming as a crucial component of in vivo cell migration in human breast tumors’, *IntraVital*, vol. 2, no. 2, p. e25294, Apr. 2013, doi: 10.4161/intv.25294.

[23] C. C. Santini *et al.*, ‘Global view of the RAF-MEK-ERK module and its immediate downstream effectors’, *Sci Rep*, vol. 9, no. 1, p. 10865, Jul. 2019, doi: 10.1038/s41598-019-47245-x.

[24] X. Robin *et al.*, ‘Probability-based detection of phosphoproteomic uncertainty reveals rare signaling events driven by oncogenic kinase gene fusion’, Apr. 29, 2019, *Systems Biology*. doi: 10.1101/621961.

[25] A. Appert-Collin, P. Hubert, G. Crémel, and A. Bennasroune, ‘Role of ErbB Receptors in Cancer Cell Migration and Invasion’, *Front. Pharmacol.*, vol. 6, Nov. 2015, doi: 10.3389/fphar.2015.00283.

[26] H. Horn *et al.*, ‘KinomeXplorer: an integrated platform for kinome biology studies’, *Nat Methods*, vol. 11, no. 6, pp. 603–604, Jun. 2014, doi: 10.1038/nmeth.2968.

[27] M. Kõivomägi *et al.*, ‘Multisite phosphorylation networks as signal processors for Cdk1’, *Nat Struct Mol Biol*, vol. 20, no. 12, pp. 1415–1424, Dec. 2013, doi: 10.1038/nsmb.2706.

[28] R. Roskoski, ‘A historical overview of protein kinases and their targeted small molecule inhibitors’, *Pharmacological Research*, vol. 100, pp. 1–23, Oct. 2015, doi: 10.1016/j.phrs.2015.07.010.

[29] M. Innes *et al.*, ‘Fashionable Modelling with Flux’, Nov. 10, 2018, *arXiv*: arXiv:1811.01457. doi: 10.48550/arXiv.1811.01457.

[30] M. Innes, ‘Don’t Unroll Adjoint: Differentiating SSA-Form Programs’, Mar. 09, 2019, *arXiv*: arXiv:1810.07951. doi: 10.48550/arXiv.1810.07951.

[31] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, ‘Julia: A Fresh Approach to Numerical Computing’, *SIAM Rev.*, vol. 59, no. 1, pp. 65–98, Jan. 2017, doi: 10.1137/141000671.

[32] Jacob Quinn, *CSV.jl: Utility library for working with CSV and other delimited files in the Julia programming language*. (2024). Julia. [Online]. Available: https://github.com/JuliaData/CSV.jl

[33] M. Bouchet-Valat and B. Kamiński, ‘DataFrames.jl : Flexible and Fast Tabular Data in Julia’, *J. Stat. Soft.*, vol. 107, no. 4, 2023, doi: 10.18637/jss.v107.i04.

[34] J. S. Bridle, ‘Probabilistic Interpretation of Feedforward Classification Network Outputs, with Relationships to Statistical Pattern Recognition’, in *Neurocomputing*, F. F. Soulié and J. Hérault, Eds, Berlin, Heidelberg: Springer Berlin Heidelberg, 1990, pp. 227–236. doi: 10.1007/978-3-642-76153-9_28.

[35] P.-J. Meyer, ‘Reachability analysis of neural networks using mixed monotonicity’, *IEEE Control Syst. Lett.*, vol. 6, pp. 3068–3073, 2022, doi: 10.1109/LCSYS.2022.3182547.

[36] D. P. Kingma and J. Ba, ‘Adam: A Method for Stochastic Optimization’, Jan. 30, 2017, *arXiv*: arXiv:1412.6980. doi: 10.48550/arXiv.1412.6980.

[37] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, ‘JuMP 1.0: Recent improvements to a modeling language for mathematical optimization’, Mar. 19, 2023, *arXiv*: arXiv:2206.03866. doi: 10.48550/arXiv.2206.03866.

[38] A. Wächter and L. T. Biegler, ‘On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming’, *Math. Program.*, vol. 106, no. 1, pp. 25–57, Mar. 2006, doi: 10.1007/s10107-004-0559-y.

[39] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, ‘MathOptInterface: A Data Structure for Mathematical Optimization Problems’, *INFORMS Journal on Computing*, vol. 34, no. 2, pp. 672–689, Mar. 2022, doi: 10.1287/ijoc.2021.1067.

[40] J. Nocedal and S. J. Wright, *Numerical optimization*. in Springer series in operations research. New York: Springer, 1999.

[41] G. Bingham and R. Miikkulainen, ‘Discovering Parametric Activation Functions’, *Neural Networks*, vol. 148, pp. 48–65, Apr. 2022, doi: 10.1016/j.neunet.2022.01.001.

[42] V. Nair and G. E. Hinton, ‘Rectified Linear Units Improve Restricted Boltzmann Machines’.

[43] C. Dugas, Y. Bengio, F. Bélisle, C. Nadeau, and R. Garcia, ‘Incorporating Second-Order Functional Knowledge for Better Option Pricing’.

[44] C. Rackauckas and Q. Nie, ‘DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia’, *JORS*, vol. 5, no. 1, p. 15, May 2017, doi: 10.5334/jors.151.

[45] C. Rackauckas, 'OrdinaryDiffEq', OrdinaryDiffEq. [Online]. Available: https://docs.sciml.ai/DiffEqDocs/stable/api/ordinarydiffeq/explicit/Tsit5/

[46] E. Pauwels, C. Lajaunie, and J.-P. Vert, 'A Bayesian active learning strategy for sequential experimental design in systems biology', *BMC Syst Biol*, vol. 8, no. 1, p. 102, Dec. 2014, doi: 10.1186/s12918-014-0102-6.

[47] J. C. Spall, 'Multivariate stochastic approximation using a simultaneous perturbation gradient approximation', *IEEE Trans. Automat. Contr.*, vol. 37, no. 3, pp. 332–341, Mar. 1992, doi: 10.1109/9.119632.

[48] M. Almanstötter, R. Vetter, and D. Iber, 'PINNverse: Accurate parameter estimation in differential equations from noisy data with constrained physics-informed neural networks', Apr. 07, 2025, *arXiv*: arXiv:2504.05248. doi: 10.48550/arXiv.2504.05248.

[49] Shashi Gowda, Ben J. Arthur, James Wringley, Tamas Nagy, and Daniel C. Jones, *Measures.jl: Unified measure and coordinates types*. (2024). Julia. [Online]. Available: https://github.com/JuliaGraphics/Measures.jl

[50] S. Christ, D. Schwabeneder, C. Rackauckas, M. K. Borregaard, and T. Breloff, 'Plots.jl – A User Extendable Plotting API for the Julia Programming Language', *Journal of Open Research Software*, vol. 11, p. 5, Feb. 2023, doi: 10.5334/jors.431.

Statutory Declaration

First Name: Luis

Last Name: Firl

Matriculation Number: 573870

Course of Study: Life Science Engineering

I herewith formally declare that I have written the submitted master thesis independently. I did not use any outside support except for the quoted literature and other sources mentioned in the thesis.

I clearly marked and separately listed all of the literature and all of the other sources which I employed when producing this academic work, either literally or in content.

In the appendix of my work, I have listed all resources based on Artificial Intelligence that were utilized in the production of my thesis. I assure that I have not used any AI-based tools whose use has been explicitly excluded in writing by the examiner. I understand that the use of texts or other content and products generated by AI-based tools does not guarantee their quality. I am fully responsible for the adoption of any machine-generated passages used by me and bear the responsibility for any erroneous or distorted content, faulty references, violations of data protection and copyright law or plagiarism generated by the AI. I also affirm that my creative influence predominates in the present work.

Berlin, 15.04.2026

Place, Date

L. Firl

Signature